



Strathmore University

@iLabAfrica Centre

Applied Machine Learning

From First Principles to Production Systems

Javier Serrano
Emmanuel Lee Bundi

Master in Data Science and Analytics
DSA 8401

Nairobi, 2026

**Applied Machine Learning: From First Principles to Production Systems in the
African Fintech Ecosystem**

Javier Serrano & Emmanuel Lee Bundi
iLabAfrica — Strathmore University, Nairobi, Kenya

Lecture notes for DSA 8401 — Applied Machine Learning,
Master in Data Science and Analytics.

© 2026 The authors. All rights reserved.
Draft edition — for enrolled students. Not for redistribution.

Typeset with L^AT_EX using the book class.
Figures produced with TikZ; code listings with listings.

Contact: jgarcia@strathmore.edu

Version 0.1 — August 31, 2026

Contents

Preface	xi
I Foundations and Classical Machine Learning	1
1 Introduction to Machine Learning	3
1.1 Why This Chapter Comes First	4
1.2 What Is Machine Learning?	5
1.2.1 An operational definition	5
1.2.2 ML inverts the direction of programming	6
1.2.3 When ML earns its keep — and when it does not . . .	7
1.2.4 The formal core: empirical risk minimization	7
1.2.5 The Bayes-optimal reference	10
1.2.6 Notation for the rest of the book	11
1.3 The Machine-Learning Landscape	11
1.3.1 The four learning paradigms	11
1.3.2 Two orthogonal operational axes	13
1.3.3 No free lunch, and what it licenses	15
1.4 Why ML Systems Fail in Production	16
1.4.1 The small green box	16
1.4.2 The notebook-to-production gap, localized	17
1.5 The End-to-End ML Lifecycle	18
1.5.1 From CRISP-DM to a monitoring-driven loop	18
1.5.2 It is a loop, not a waterfall	19
1.6 Framing the Problem and Choosing the Metric	20
1.6.1 Step 1 is where projects are won	20
1.6.2 Regression metrics: how the business pays for errors	21
1.6.3 Classification under imbalance: why accuracy lies . .	21
1.7 A First Look at the Data	24
1.7.1 The raw material	24
1.7.2 A feature dictionary, declared up front	24
1.7.3 What to interrogate, and why	25
1.8 Data, Splits, and Leakage	27

1.8.1	Splits: the discipline that protects your estimate of $R(f)$	27
1.8.2	Data leakage: the silent killer of fintech models	28
1.9	Baselines, Evaluation, and Generalization	30
1.9.1	Why generalization is hard: the bias–variance decomposition	30
1.9.2	Always ship a baseline first	31
1.9.3	Cross-validation: cheap insurance on the estimate . .	31
1.10	Presenting and Deciding	33
1.10.1	The decision memo	33
1.10.2	Champion, challenger, and shadow mode	33
1.11	Fairness, Explainability, and Regulation	34
1.11.1	Making “fair” precise enough to test	34
1.11.2	Explainability as a deployment requirement	35
1.12	From Model to System: MLOps and the Agentic Outlook . .	35
1.12.1	The MLOps spine of this course	36
1.12.2	Where LLMs and agents fit — and where they do not	37
1.13	Chapter Summary	38
	Exercises	41
	Key Terms	42
	Further Reading	42
2	Data Preparation and Feature Engineering	45
2.1	Introduction: From Raw Data to Model-Ready Data	46
2.1.1	The Six Dimensions of Data Quality	47
2.1.2	The Unit of Prediction	48
2.2	Missing Data	48
2.2.1	Rubin’s Taxonomy of Missingness	49
2.2.2	Diagnosis Before Treatment	50
2.2.3	Imputation I: Simple Statistics with Indicators	50
2.2.4	Imputation II: Model-Based Methods	51
2.2.5	A Decision Framework	52
2.3	Outliers and Duplicates	53
2.3.1	Detecting Outliers	53
2.3.2	Treating Outliers, and When Not To	54
2.3.3	Duplicates and Entity Resolution	54
2.4	Scaling and Transformations	55
2.4.1	Why Scale, and With What	55
2.4.2	Taming Skew: Log and Power Transforms	56
2.5	Encoding Categorical Variables	57
2.5.1	Low Cardinality: One-Hot and Ordinal	57
2.5.2	High Cardinality I: Frequency Encoding and the Hashing Trick	58
2.5.3	High Cardinality II: Target Encoding	58
2.6	Feature Engineering	59

2.6.1	Engineered versus Learned Features	59
2.6.2	Domain Features for Mobile-Money Credit Scoring	60
2.6.3	Datetime Features and Cyclical Encoding	61
2.6.4	Interactions, Binning, and Crosses	61
2.6.5	What Makes a Feature Good?	62
2.7	Data Leakage	63
2.7.1	Definition and Why It Is Dangerous	63
2.7.2	Six Common Causes	64
2.7.3	Temporal and Group-Aware Splits	64
2.7.4	A Detection Protocol	65
2.7.5	The Structural Antidote: Pipelines	66
2.8	From Pipelines to Feature Platforms	67
	Assignment 2 (graded, 20 marks)	68
	Exercises	69
	Key Terms	70
	Further Reading	70
3	Classical Machine Learning and Rigorous Evaluation	71
3.1	Introduction	72
3.2	The Classical Toolbox	73
3.2.1	Why classical models still run production tabular ML	73
3.2.2	Logistic regression	73
3.2.3	Softmax regression for multiclass problems	75
3.2.4	Maximum-margin classification: support vector machines	75
3.2.5	Neighbours and Bayes: two more baselines	76
3.2.6	Choosing among the classical learners	77
3.3	Beyond Accuracy: Metrics that Match the Problem	78
3.3.1	The accuracy trap	78
3.3.2	The confusion matrix	78
3.3.3	Precision, recall, and the F_β family	79
3.3.4	From metrics to money: cost-sensitive evaluation	79
3.4	Ranking Quality: ROC and Precision–Recall Analysis	80
3.4.1	Scores, not labels: the threshold as a free parameter	80
3.4.2	The ROC curve and AUC	81
3.4.3	Precision–recall curves: the imbalance-honest view	82
3.5	Calibration: Do the Probabilities Mean Anything?	82
3.6	Cross-Validation and Honest Uncertainty	84
3.6.1	Why one split is not enough	84
3.6.2	Cross-validating the pipeline	85
3.6.3	Nested cross-validation for model selection	86
3.6.4	Comparing models honestly	86
3.7	Learning from Imbalanced Data	87
3.7.1	Three families of remedies	87

3.7.2	SMOTE: synthesising minority examples	88
3.7.3	The cardinal sin: resampling before splitting	89
3.8	Case Study: Evaluation Design for the Thin-File Credit Scorer	90
3.9	Chapter Summary	91
4	Decision Trees, Ensembles and Hyperparameter Tuning	95
4.1	Introduction	96
4.2	Decision Trees: The Building Block	98
4.2.1	A tree is a sequence of questions	98
4.2.2	The same object, drawn as a partition	98
4.2.3	How a tree is grown	99
4.2.4	Controlling complexity: stopping and pruning	100
4.2.5	The flaw that motivates the rest of the chapter	101
4.3	Why Ensembles Work	102
4.3.1	The wisdom of independent crowds	102
4.3.2	The variance of an average	103
4.4	Voting Classifiers	104
4.5	Bagging and Random Forests	106
4.5.1	Bootstrap aggregating	106
4.5.2	Out-of-bag evaluation	106
4.5.3	Random Forests: decorrelating the trees	107
4.6	Boosting	109
4.6.1	The idea, before the algebra	109
4.6.2	A worked trace	110
4.6.3	From residuals to gradients	111
4.6.4	AdaBoost, the historical special case	112
4.6.5	Shrinkage and early stopping	113
4.7	Regularised Gradient Boosting: XGBoost, LightGBM, CatBoost	113
4.7.1	Regularisation inside the objective	114
4.7.2	The library landscape	115
4.7.3	Monotonic constraints and regulated deployment	115
4.8	Stacked Generalisation	117
4.9	Hyperparameter Optimisation	118
4.9.1	What is a hyperparameter, and why is it different?	118
4.9.2	A ladder of strategies	119
4.9.3	Grid search versus random search	119
4.9.4	Learning from previous trials: Bayesian search and TPE	121
4.9.5	Spending less per trial: successive halving and Hyperband	122
4.9.6	Designing the search space	123
4.9.7	Honest evaluation of a tuned model	123
4.10	Case Study: Round Three of the Credit Scorer	125
	Exercises	127
	Key Terms	128

Further Reading	129
5 Unsupervised Learning and Dimensionality Reduction	131
5.1 Introduction: Learning Without Labels	132
5.2 The Unsupervised Landscape	134
5.3 Clustering with K -Means	136
5.3.1 The objective	136
5.3.2 Choosing k	137
5.3.3 Implementation	139
5.4 Density-Based Clustering: DBSCAN	140
5.5 Gaussian Mixtures and EM	142
5.5.1 The model	142
5.5.2 Fitting by Expectation–Maximisation	143
5.5.3 Selecting k by information criteria	145
5.5.4 Mixtures as anomaly detectors	146
5.6 The Curse of Dimensionality	147
5.7 Principal Component Analysis	148
5.7.1 Variance maximisation	148
5.7.2 Reconstruction-error minimisation	149
5.7.3 Computation and the choice of d	149
5.7.4 Variants you will actually need	151
5.8 Case Study: The Segmentation Pipeline in Production	152
5.8.1 Profiling: turning clusters into personas	152
5.8.2 Stability: a segment that moves is not a segment	153
Exercises	156
Key Terms	157
Further Reading	158
 II Deep Learning and Generative AI	 159
6 Deep Learning Fundamentals	161
6.1 Introduction: from engineered features to learned representations	162
6.1.1 When a network is the right instrument	164
6.2 The artificial neuron and the multi-layer perceptron	165
6.3 Activation functions	168
6.4 The training objective	170
6.4.1 Binary cross-entropy from the Bernoulli likelihood	170
6.4.2 Choosing the output layer and loss together	172
6.5 Gradient descent	172
6.5.1 Why the learning rate dominates everything else	173
6.6 Backpropagation	174
6.7 Optimisers beyond plain gradient descent	177

6.7.1	Momentum	177
6.7.2	Adam	177
6.7.3	Learning-rate schedules	178
6.8	Pathologies of deep training	179
6.9	Initialisation	180
6.10	Normalisation layers	181
6.11	Regularisation	182
6.11.1	Dropout	182
6.11.2	Early stopping and weight decay	183
6.12	Input scaling and conditioning	184
6.13	Entity embeddings	185
6.14	Implementation in TensorFlow and Keras	187
6.14.1	The training loop, written out	187
6.14.2	The baseline network	188
6.14.3	The embedding challenger	189
6.14.4	Hyperparameter search and debugging	189
6.15	Evaluation, calibration and governance	190
6.15.1	Ranking versus calibration	190
6.15.2	Explanation	191
6.15.3	Fairness and segment-level performance	191
	Exercises	194
	Key Terms	196
	Further Reading	197
7	Computer Vision with Convolutional Networks	199
7.1	Introduction	199
7.2	Core Theory	199
7.3	Implementation	199
8	Sequence Models and Time Series	201
8.1	Introduction	201
8.2	Core Theory	201
8.3	Implementation	201
9	Transformers, Large Language Models and RAG	203
9.1	Introduction	203
9.2	Core Theory	203
9.3	Implementation	203
III	MLOps and Production Systems	205
10	MLOps: Packaging, Serving, Tracking, CI/CD	207
10.1	Introduction	207

10.2 Core Theory	207
10.3 Implementation	207
11 Model Monitoring, Drift and ML System Design	209
11.1 Introduction	209
11.2 Core Theory	209
11.3 Implementation	209
 Appendices	 213
A Capstone Project	213
B Solutions to Exercises	215
Chapter 1	215
Chapter 2	222
Chapter 3	227

Preface

This book collects the lecture notes of **DSA 8401 — Applied Machine Learning**, taught in the Master in Data Science and Analytics at iLabAfrica, Strathmore University. It is written for graduate students who already program and know basic statistics, and who want to go beyond training models in a notebook: the goal throughout is *production-grade* machine learning — data pipelines, rigorous evaluation, deployment, and monitoring.

Three commitments shape every chapter:

1. **An end-to-end view.** Models are treated as components of systems. MLOps (Docker, FastAPI, MLflow, CI/CD) is not an afterthought but the destination of the whole arc.
2. **African context as pedagogy.** Worked examples are anchored in the African technology and financial ecosystem: thin-file mobile-money credit scoring (the running thread of the course), trade-corridor underwriting, clinical pathway modelling, and streaming fraud detection. This is a deliberate instructional choice, not decoration.
3. **Modern architectures.** Transformers, LLMs, RAG pipelines, and an outlook on agentic systems are integrated where they matter for applied work.

How this book maps to the course

Each chapter corresponds to one teaching week (three hours: a 1.5-hour lecture plus a 1.5-hour practical lab). The two MLOps weeks are consolidated into a single chapter, and the capstone project brief lives in Appendix [A](#).

Chapter	Topic	Course week
1	End-to-End Machine Learning	1
2	Data Preparation & Feature Engineering	2
3	Classical ML & Rigorous Evaluation	3
4	Ensemble Methods & Hyperparameter Tuning	4
5	Unsupervised Learning & Dim. Reduction	5
6	Deep Learning Fundamentals	6
7	Computer Vision with CNNs	7
8	Sequence Models & Time Series	9
9	Transformers, LLMs & RAG	10
10	MLOps: Serving, Tracking & CI/CD	11–12
11	Monitoring, Drift & System Design	13
A	Capstone Project	14–15

Prerequisites and companion material

We assume the Data Mining unit, working Python, and comfort with linear algebra and probability at the level of a first graduate course. The primary textbook companion is [Géron \(2022\)](#); [Bishop and Bishop \(2024\)](#) and [Huyen \(2022\)](#) support the deep-learning and MLOps parts respectively. Each chapter closes with a lab section that mirrors the Jupyter notebook used in the practical session.

Javier Serrano & Emmanuel Lee Bundi
Nairobi, 2026

Part I

Foundations and Classical Machine Learning

Part II

**Deep Learning and Generative
AI**

CHAPTER 6

Deep Learning Fundamentals

Course week 6

Learning Objectives

After working through this chapter you will be able to:

- Describe the anatomy of a multi-layer perceptron, units, dense layers, weights, biases and activations, and evaluate a forward pass by hand for a small network.
- Derive the training objective for binary classification from the Bernoulli likelihood, and explain why the gradient of that objective with respect to the output logit reduces to $\hat{p} - y$.
- State and prove the backpropagation recursion, and explain why it computes every partial derivative at roughly the cost of two forward passes.
- Diagnose vanishing and exploding gradients, dead units, overfitting and unstable optimisation from training curves, and select the appropriate remedy in each case.
- Justify initialisation schemes, normalisation layers, dropout, early stopping and input standardisation from first principles rather than from folklore.
- Implement, train, regularise and evaluate a multi-layer perceptron in TensorFlow/Keras, including entity embeddings for high-cardinality categorical features and out-of-vocabulary handling.
- Decide, with evidence, whether a neural network or a gradient-boosted tree ensemble is the appropriate model for a given tabular credit-risk problem, and document that decision to supervisory standards.

6.1 Introduction: from engineered features to learned representations

Chapters 2 through 5 were, in a precise sense, chapters about *features*. The gradient-boosted ensemble that currently holds the champion position in our running credit-scoring problem does not owe its performance to the boosting algorithm alone; it owes it to the velocity ratios, rolling balances, agent-density counts and recency windows that we constructed by hand in Chapter 2. Strip those away and feed the raw transaction fields directly to the same ensemble, and performance collapses. The model was never the whole story.

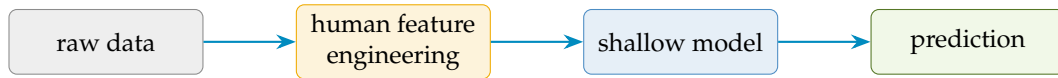
Deep learning inverts the division of labour. Instead of a human con-

structuring features and a shallow model consuming them, we stack many simple parametric transformations and let a single optimisation procedure learn what the intermediate representations should be. Each layer applies an affine map followed by a fixed non-linear function,

$$\mathbf{h}^{(l)} = \phi\left(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right), \quad (6.1)$$

and the composition of many such maps is capable of representing functions that no single affine map can. Critically, nothing tells layer 2 what to represent. The only signal reaching it is the gradient of the loss, propagated backwards through layer 3 and beyond. Representations that reduce the loss survive; those that do not are pushed out of existence by the same gradient descent that fits the output weights.

Chapters 2–5: the analyst builds the features



Chapter 6 onwards: the network learns them

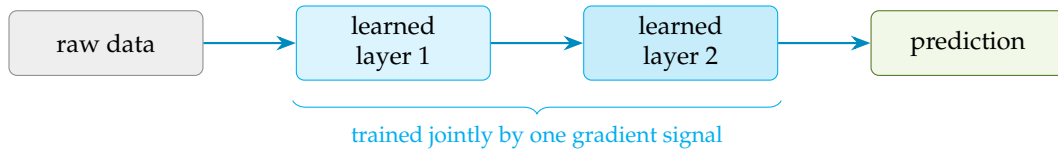


Figure 6.1: Two divisions of labour between analyst and algorithm. The upper pipeline is the one used in Chapters 2–5; the lower is the subject of Part II of this book.

This chapter is deliberately conservative about what that inversion buys you. On medium-sized, purely tabular problems, which is exactly what a thin-file credit book is, carefully tuned gradient boosting frequently remains the stronger model (Grinsztajn et al., 2022b). The honest reason to build a neural network for our lender is not that deep learning is fashionable; it is that neural networks handle two things that trees handle badly: categorical variables with tens of thousands of levels, and inputs of heterogeneous type that must be fused into a single representation. Both appear in the case study below.

Case Study: The Challenger Model, Round Five

Our Nairobi digital lender now serves roughly 3.4 million mobile-money wallets. The labelled loan book carries a default rate of approximately 4%, and the cost matrix established in Chapter 3 still governs every decision: a missed default costs about KES 10,000 in written-off principal, while a wrongly rejected creditworthy borrower costs about KES 800 in forgone margin and acquisition spend.

The credit risk committee has asked a specific question. Each loan application is associated with the mobile-money *agent* through which the borrower transacts most often, one of roughly 120,000 agents nationwide, ranging from high-volume urban super-agents to rural kiosks that process a handful of transactions a week. The champion Light-GBM model treats agent identity crudely: one-hot encoding is infeasible at that cardinality, and target encoding leaked badly in an earlier iteration. Yet the risk team is convinced that agent identity carries real signal about local economic conditions and about the integrity of the cash-in/cash-out chain.

The question for this chapter is therefore not “is deep learning better?” but “can a network that *learns* a representation of agent identity beat the champion on the frozen splits, at acceptable operational cost?” We will build that challenger, benchmark it honestly, and write up the comparison in the form a supervisor would expect to read.

6.1.1 When a network is the right instrument

Before building anything it is worth stating the decision rule plainly. Gradient boosting tends to win when the data are purely tabular, the sample is of moderate size ($n \lesssim 10^5$), the features are heterogeneous and already informative, and the team needs interpretable attributions quickly. Neural networks tend to win when at least one of the following holds:

1. **High-cardinality categorical inputs.** Agent identifiers, ward codes, device models and merchant codes have thousands to hundreds of thousands of levels. Embedding layers (Section 6.13) learn a low-dimensional dense representation of each level, trained by the same gradient as the rest of the model.
2. **Multi-modal inputs.** If the feature set must fuse tabular transaction aggregates with SMS text, geospatial trajectories or images of identity documents, a network provides a single differentiable architecture in which all modalities can be encoded and concatenated.
3. **Shared representations.** If the lender operates several products, a 30-day nano-loan, an agent float facility, a merchant advance, a shared

trunk with product-specific heads transfers signal between them.

4. **Incremental updating.** Networks can be warm-started from yesterday's weights and updated on new data, whereas boosted ensembles are typically refit from scratch.

Table 6.1: Decision criteria for the champion–challenger comparison in this chapter. The entries are qualitative judgements about *typical* tabular credit problems, not universal laws; the empirical comparison on your own frozen splits is what settles the matter.

Criterion	Gradient boosting	Neural network
Medium-sized purely tabular data	strong	weaker
High-cardinality identifiers	limited	native (embeddings)
Multi-modal fusion	poor	natural
Training cost and tuning effort	low	higher
Incremental update without full refit	no	yes (warm start)
Attribution for adverse-action notices	easier	harder, but feasible

The challenger discipline

A challenger model earns deployment only by beating the champion on the *same* frozen splits, under the *same* cost matrix, with the *same* evaluation protocol. Any comparison in which the challenger is tuned on data the champion never saw is not evidence; it is a marketing exercise. Chapter 3's protocol is not suspended because the model class changed.

6.2 The artificial neuron and the multi-layer perceptron

Definition 6.1 (Artificial neuron). Let $\mathbf{x} \in \mathbb{R}^d$ be an input vector, $\mathbf{w} \in \mathbb{R}^d$ a weight vector, $b \in \mathbb{R}$ a bias, and $\phi : \mathbb{R} \rightarrow \mathbb{R}$ a function called the *activation*. An artificial neuron is the map

$$\mathbf{x} \mapsto a = \phi(z), \quad z = \mathbf{w}^\top \mathbf{x} + b. \quad (6.2)$$

The scalar z is called the *pre-activation* or *logit*, and a the *activation* of the unit.

The three components have distinct interpretations. The weight vector $\mathbf{w}$ encodes which input directions the unit is sensitive to; the bias b shifts the threshold at which the unit responds; and ϕ determines the shape of the response. In our credit application, a unit might learn $\mathbf{w}$ that places

large positive weight on the ninety-day transaction count and large negative weight on the coefficient of variation of the wallet balance, so that its activation is high for borrowers with regular, predictable cash flow.

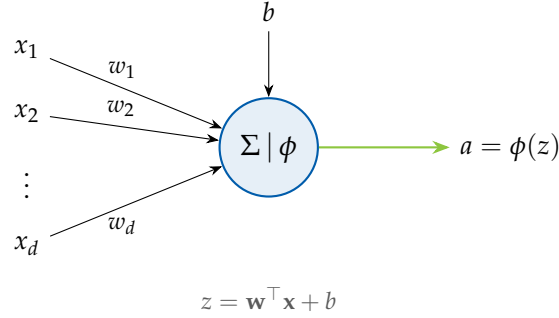


Figure 6.2: The artificial neuron of Definition 6.1. The weighted sum is a linear operation; all of the model’s expressive power beyond linear regression comes from ϕ .

A single unit with a step activation is the perceptron of Rosenblatt (1958). It has two limitations that between them motivate everything that follows. First, its decision boundary is a hyperplane, so it cannot represent the exclusive-or function on two binary inputs. Second, the step function has zero derivative almost everywhere, so it cannot be trained by any gradient-based method. Replacing the step by a differentiable ϕ and stacking units into layers removes both limitations at once.

Definition 6.2 (Dense layer and multi-layer perceptron). A *dense* (or fully connected) layer with n_{in} inputs and n_{out} units is the map $\mathbb{R}^{n_{\text{in}}} \rightarrow \mathbb{R}^{n_{\text{out}}}$ given by (6.1), parameterised by $\mathbf{W}^{(l)} \in \mathbb{R}^{n_{\text{out}} \times n_{\text{in}}}$ and $\mathbf{b}^{(l)} \in \mathbb{R}^{n_{\text{out}}}$, with ϕ applied elementwise. A *multi-layer perceptron* (MLP) with L layers is the composition

$$f_{\theta}(\mathbf{x}) = \phi^{(L)} \left(\mathbf{W}^{(L)} \phi^{(L-1)} \left(\dots \phi^{(1)} \left(\mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)} \right) \dots \right) + \mathbf{b}^{(L)} \right), \quad (6.3)$$

with $\theta = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^L$. Layers $1, \dots, L-1$ are the *hidden* layers and layer L is the *output* layer.

The reason the non-linearity cannot be omitted is worth stating formally, because students routinely assume that “more layers” is itself the source of expressive power.

Proposition 6.1 (Composition of affine maps is affine). Let $T_1(\mathbf{x}) = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$ and $T_2(\mathbf{u}) = \mathbf{W}^{(2)}\mathbf{u} + \mathbf{b}^{(2)}$ be affine maps. Then $T_2 \circ T_1$ is affine, and there exist $\tilde{\mathbf{W}}$ and $\tilde{\mathbf{b}}$ such that $(T_2 \circ T_1)(\mathbf{x}) = \tilde{\mathbf{W}}\mathbf{x} + \tilde{\mathbf{b}}$. Consequently an MLP with $\phi = \text{id}$ in every layer has exactly the representational capacity of a single affine map, regardless of L .

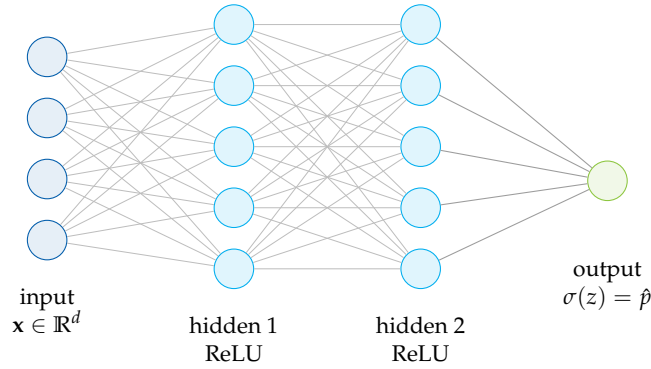


Figure 6.3: An MLP with two hidden layers, the architecture used for the challenger model in this chapter. Every unit in a dense layer is connected to every unit of the preceding layer.

Proof. Direct computation gives

$$T_2(T_1(\mathbf{x})) = \mathbf{W}^{(2)} \left(\mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)} \right) + \mathbf{b}^{(2)} = \underbrace{\mathbf{W}^{(2)} \mathbf{W}^{(1)}}_{\tilde{\mathbf{W}}} \mathbf{x} + \underbrace{\mathbf{W}^{(2)} \mathbf{b}^{(1)} + \mathbf{b}^{(2)}}_{\tilde{\mathbf{b}}},$$

which is affine. Induction on L extends the result to any finite composition: if $T_{L-1} \circ \dots \circ T_1$ is affine with matrix $\mathbf{A}$ and offset $\mathbf{c}$, then composing with T_L yields matrix $\mathbf{W}^{(L)} \mathbf{A}$ and offset $\mathbf{W}^{(L)} \mathbf{c} + \mathbf{b}^{(L)}$. $\square$

Depth without non-linearity is therefore an expensive way to fit a linear model. With non-linearity, the situation changes completely.

Theorem 6.1 (Universal approximation, informally). *Let ϕ be a continuous, non-polynomial function. For any continuous function g on a compact set $K \subset \mathbb{R}^d$ and any $\varepsilon > 0$, there exist a finite width m , weights $\{\mathbf{w}_j\}_{j=1}^m$, biases $\{b_j\}_{j=1}^m$ and output coefficients $\{c_j\}_{j=1}^m$ such that*

$$\sup_{\mathbf{x} \in K} \left| g(\mathbf{x}) - \sum_{j=1}^m c_j \phi(\mathbf{w}_j^\top \mathbf{x} + b_j) \right| < \varepsilon.$$

Proofs are given by [Cybenko \(1989\)](#) for sigmoidal ϕ and by [Hornik \(1991\)](#) in greater generality; we state it without proof because the proof technique, density arguments in $C(K)$, is not otherwise used in this course. Two caveats matter far more than the theorem itself. First, the theorem is an existence result: it says nothing about whether gradient descent will *find* the approximating weights. Second, the required width m may be astronomically large, whereas adding depth often achieves the same accuracy with exponentially fewer parameters. In practice, depth is a statement about parameter efficiency, not about what is representable in principle.

Remark 6.1 (Parameter counting). A dense layer with n_{in} inputs and n_{out} units holds $(n_{\text{in}} + 1) \times n_{\text{out}}$ parameters, the +1 accounting for the bias. This arithmetic should be performed *before* training, not discovered afterwards from an out-of-memory error.

Example 6.1 (Parameter count for the challenger). With 40 standardised numeric features and hidden layers of width 64 and 32, the network of Figure 6.3 holds

$$\underbrace{41 \times 64}_{2624} + \underbrace{65 \times 32}_{2080} + \underbrace{33 \times 1}_{33} = 4737$$

parameters. Against roughly 120,000 labelled loans this is a comfortable ratio. Once we add an embedding for 120,000 agents at 16 dimensions, however, the embedding table alone contributes 1,920,016 parameters, four hundred times the rest of the network. That asymmetry is why embeddings need their own regularisation discussion in Section 6.13.

6.3 Activation functions

The choice of ϕ determines how gradients flow, and gradient flow determines whether the network trains at all. We treat the two families separately because they fail in different ways.

Definition 6.3 (Common activations). For $z \in \mathbb{R}$ and $\mathbf{z} \in \mathbb{R}^K$ define

$$\text{logistic sigmoid: } \sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1), \quad (6.4)$$

$$\text{hyperbolic tangent: } \tanh(z) = 2\sigma(2z) - 1 \in (-1, 1), \quad (6.5)$$

$$\text{rectified linear unit: } \text{ReLU}(z) = \max(0, z), \quad (6.6)$$

$$\text{leaky ReLU: } \text{LReLU}_\alpha(z) = \max(\alpha z, z), \quad 0 < \alpha \ll 1, \quad (6.7)$$

$$\text{exponential linear unit: } \text{ELU}_\alpha(z) = \begin{cases} z & z > 0 \\ \alpha(e^z - 1) & z \leq 0, \end{cases} \quad (6.8)$$

$$\text{softmax: } \text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}. \quad (6.9)$$

Proposition 6.2 (The sigmoid derivative is bounded by one quarter). For all $z \in \mathbb{R}$, $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ and $\sigma'(z) \leq \frac{1}{4}$, with equality if and only if $z = 0$.

Proof. Write $\sigma(z) = (1 + e^{-z})^{-1}$. Differentiating,

$$\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = \sigma(z)(1 - \sigma(z)),$$

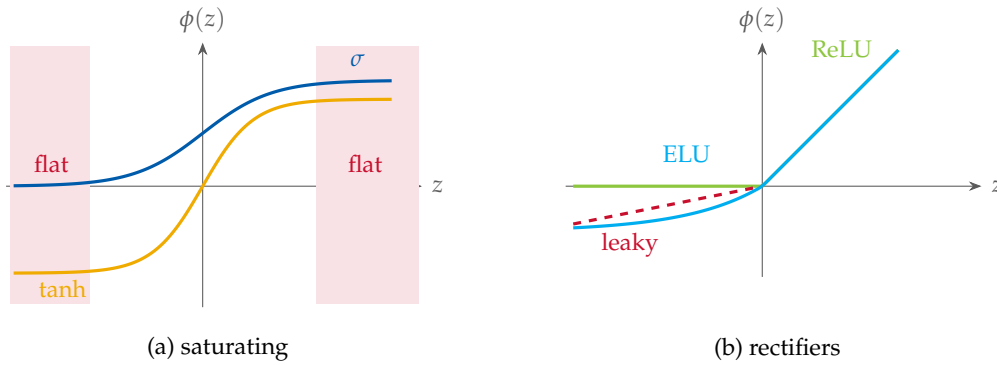


Figure 6.4: Activation functions. The shaded regions in panel (a) are where the derivative is numerically negligible; a unit whose pre-activation lands there transmits almost no gradient to earlier layers.

using $1 - \sigma(z) = e^{-z} / (1 + e^{-z})$. Now set $s = \sigma(z) \in (0, 1)$. The function $s \mapsto s(1 - s)$ is a downward parabola with maximum at $s = \frac{1}{2}$ and maximal value $\frac{1}{4}$. Since σ is a bijection from $\mathbb{R}$ to $(0, 1)$ with $\sigma(0) = \frac{1}{2}$, the bound is attained exactly at $z = 0$. $\square$

Proposition 6.2 is the seed of the vanishing-gradient problem analysed in Section 6.8: every sigmoid layer multiplies the backward signal by a factor of at most $\frac{1}{4}$, before the weight matrices have had their say. The rectifier family avoids this by having derivative exactly 1 on the positive half-line, at the price of derivative exactly 0 on the negative half-line, which creates a failure mode of its own.

Dead rectified units

A ReLU unit whose pre-activation is negative for every training example has gradient zero with respect to all of its incoming weights, forever. It is *dead*: no update can revive it, because the update itself is proportional to a gradient that is identically zero. Large learning rates and poor initialisation are the usual causes, since a single oversized update can push a unit's bias so far negative that it never fires again. Symptoms are a training loss that plateaus at a suspiciously round value and a layer whose activations are almost entirely zero. Remedies, in order of preference: reduce the learning rate, use He initialisation (Section 6.9), and if the problem persists switch to leaky ReLU or ELU, which keep a small negative slope and therefore a non-zero gradient everywhere.

For the output layer the choice is dictated by the loss, not by preference. The next proposition explains why softmax is implemented the way it is in every serious library.

Proposition 6.3 (Shift invariance of softmax). *For any $\mathbf{z} \in \mathbb{R}^K$ and any $c \in \mathbb{R}$,*

$$\text{softmax}(\mathbf{z} + c\mathbf{1}) = \text{softmax}(\mathbf{z}),$$

where $\mathbf{1}$ is the all-ones vector.

Proof. For each component k ,

$$\text{softmax}(\mathbf{z} + c\mathbf{1})_k = \frac{e^{z_k + c}}{\sum_j e^{z_j + c}} = \frac{e^c e^{z_k}}{e^c \sum_j e^{z_j}} = \frac{e^{z_k}}{\sum_j e^{z_j}} = \text{softmax}(\mathbf{z})_k. \quad \square$$

Choosing $c = -\max_k z_k$ makes every exponent non-positive and hence every exponential at most 1, which removes the risk of overflow without changing the result. This is what TensorFlow does internally, and it is the reason you should pass *logits* rather than probabilities to the library's loss functions.

6.4 The training objective

Training is the numerical minimisation of an explicit scalar objective. Making that objective explicit is the difference between engineering and hope.

Definition 6.4 (Empirical risk). Given a training set $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, a model f_θ and a per-example loss $\mathcal{L}$, the *empirical risk* is

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(y_i, f_\theta(\mathbf{x}_i)). \quad (6.10)$$

Training means finding θ that makes J small, subject to whatever regularisation is imposed; it does *not* mean finding a global minimiser, which for a non-convex J is neither achievable nor necessary.

6.4.1 Binary cross-entropy from the Bernoulli likelihood

For our default-prediction problem $y_i \in \{0, 1\}$ and the network outputs $\hat{p}_i = \sigma(z_i) \in (0, 1)$, interpreted as $\Pr(y_i = 1 \mid \mathbf{x}_i)$. The loss is not chosen arbitrarily; it follows from the probabilistic model.

Proposition 6.4 (Cross-entropy is the negative Bernoulli log-likelihood). *Suppose $y_i \mid \mathbf{x}_i \sim \text{Bernoulli}(\hat{p}_i)$ independently across i . Then maximising the likelihood of the observed labels is equivalent to minimising*

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n \left[-y_i \log \hat{p}_i - (1 - y_i) \log(1 - \hat{p}_i) \right]. \quad (6.11)$$

Proof. The Bernoulli probability mass function can be written in the single expression $\Pr(y_i) = \hat{p}_i^{y_i}(1 - \hat{p}_i)^{1-y_i}$, which returns $\hat{p}_i$ when $y_i = 1$ and $1 - \hat{p}_i$ when $y_i = 0$. By independence the likelihood of the sample is the product $\prod_{i=1}^n \hat{p}_i^{y_i}(1 - \hat{p}_i)^{1-y_i}$. Taking logarithms turns the product into a sum,

$$\ell(\boldsymbol{\theta}) = \sum_{i=1}^n [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)],$$

and since $\log$ is strictly increasing, maximising the likelihood is equivalent to maximising ℓ . Multiplying by $-1/n$ reverses the optimisation direction and rescales by a positive constant, neither of which changes the location of the optimum, and yields exactly (6.11). $\square$

The loss punishes confident errors without mercy: as $\hat{p}_i \rightarrow 0$ with $y_i = 1$, the contribution $-\log \hat{p}_i \rightarrow \infty$. This is a feature. A model that assigns probability 0.001 to a default that then occurs has made a qualitatively worse statement about the world than one that assigned 0.3, and the loss should say so.

The next result is the single most useful piece of algebra in this chapter.

Proposition 6.5 (Cross-entropy gradient at the logit). *Let $\hat{p} = \sigma(z)$ and $\mathcal{L}(y, \hat{p}) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p})$. Then*

$$\frac{\partial \mathcal{L}}{\partial z} = \hat{p} - y. \quad (6.12)$$

Proof. By the chain rule, $\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial \hat{p}} \cdot \frac{d\hat{p}}{dz}$. The first factor is

$$\frac{\partial \mathcal{L}}{\partial \hat{p}} = -\frac{y}{\hat{p}} + \frac{1-y}{1-\hat{p}} = \frac{-y(1-\hat{p}) + (1-y)\hat{p}}{\hat{p}(1-\hat{p})} = \frac{\hat{p} - y}{\hat{p}(1-\hat{p})},$$

and by Proposition 6.2 the second is $\hat{p}(1 - \hat{p})$. Multiplying, the factor $\hat{p}(1 - \hat{p})$ cancels exactly and leaves $\hat{p} - y$. $\square$

Two consequences deserve emphasis. Mathematically, the cancellation means the output layer never suffers from sigmoid saturation: even when $\hat{p}$ is close to 0 or 1, the error signal is the plain residual $\hat{p} - y$ rather than something scaled by a vanishing σ' . Numerically, the cancellation is only realised if the library computes the composition in one step, which is why `sigmoid_cross_entropy_with_logits` exists and why passing an already-squashed probability into a generic cross-entropy is a bug waiting to produce NaN.

Table 6.2: Output layer and loss for the prediction tasks arising in the running case study. The pairing is not a matter of style: each loss is the negative log-likelihood of the distribution implied by its output layer.

Task	Output layer	Loss
Default within 30 days (binary)	1 unit, sigmoid	binary cross-entropy
Loan purpose, K categories	K units, softmax	categorical cross-entropy
Recovered amount in KES	1 unit, linear	squared error, or Huber if outliers
Days until repayment	1 unit, softplus	Poisson deviance

6.4.2 Choosing the output layer and loss together

Class imbalance carries over unchanged from Chapter 3. With a 4% default rate, a model that approves every applicant achieves 96% accuracy, so accuracy must not appear anywhere in the training or model-selection loop. Two mechanisms are available. Weighting multiplies each example's loss by a class-dependent constant w_{y_i} , which is equivalent to resampling in expectation but avoids duplicating rows; in Keras this is the `class_weight` argument. Focal loss (Lin et al., 2017) instead down-weights examples the model already classifies confidently, replacing $-\log \hat{p}$ by $-(1 - \hat{p})^\gamma \log \hat{p}$ for some $\gamma > 0$, and is worth trying when the imbalance is more extreme than roughly 1:100.

Weighting changes calibration

Class weighting improves the ranking metrics that matter operationally, but it deliberately distorts the predicted probabilities: a model trained with the positive class up-weighted by a factor w systematically over-predicts default risk. If the output feeds a pricing engine or a provisioning calculation, you must recalibrate before use, see Section 6.15. Ranking and calibration are different properties and must be checked separately.

6.5 Gradient descent

Definition 6.5 (Gradient descent and its mini-batch variant). Gradient descent generates iterates

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta_t), \quad (6.13)$$

where $\eta > 0$ is the *learning rate*. If the gradient in (6.13) is computed on all n examples the method is *batch* gradient descent; if on a single randomly

drawn example it is *stochastic* gradient descent (SGD); and if on a random subset $\mathcal{B}$ of size B it is *mini-batch* SGD, with

$$\hat{\mathbf{g}}_{\mathcal{B}} = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_{\theta} \mathcal{L}(y_i, f_{\theta}(\mathbf{x}_i)). \quad (6.14)$$

One *epoch* is one pass through the training set, that is $\lceil n/B \rceil$ mini-batch updates.

Mini-batching is not merely a memory-management compromise; it has a precise statistical justification.

Proposition 6.6 (The mini-batch gradient is unbiased, with variance $O(1/B)$). *Let $\mathcal{B}$ be a uniformly random subset of size B drawn with replacement from the training set, and let $\mathbf{g}_i = \nabla_{\theta} \mathcal{L}(y_i, f_{\theta}(\mathbf{x}_i))$. Then*

$$\mathbb{E}[\hat{\mathbf{g}}_{\mathcal{B}}] = \nabla_{\theta} J(\theta), \quad \text{Var}[\hat{\mathbf{g}}_{\mathcal{B}}] = \frac{1}{B} \text{Var}[\mathbf{g}_i],$$

where the variance is taken componentwise.

Proof. Each index in $\mathcal{B}$ is drawn uniformly from $\{1, \dots, n\}$, so $\mathbb{E}[\mathbf{g}_i] = \frac{1}{n} \sum_{j=1}^n \mathbf{g}_j = \nabla_{\theta} J(\theta)$ by (6.10) and linearity of differentiation. Linearity of expectation then gives

$$\mathbb{E}[\hat{\mathbf{g}}_{\mathcal{B}}] = \frac{1}{B} \sum_{i \in \mathcal{B}} \mathbb{E}[\mathbf{g}_i] = \nabla_{\theta} J(\theta).$$

For the variance, the draws are independent and identically distributed, so the variance of the average of B i.i.d. terms is $1/B$ times the variance of one term. $\square$

The practical reading is that quadrupling the batch size halves the gradient noise but quadruples the computation per step. Mid-sized batches (B between 128 and 512 for problems of our scale) are the usual compromise, and the residual noise is not purely a cost: it perturbs the iterates away from sharp minima and acts as a weak regulariser (Keskar et al., 2017).

6.5.1 Why the learning rate dominates everything else

Proposition 6.7 (Convergence of gradient descent on a quadratic). *Let $J(\theta) = \frac{1}{2} \theta^{\top} \mathbf{H} \theta$ with $\mathbf{H}$ symmetric positive definite with eigenvalues $0 < \lambda_{\min} = \lambda_1 \leq \dots \leq \lambda_d = \lambda_{\max}$. Gradient descent (6.13) converges to $\theta = \mathbf{0}$ from every starting point if and only if $0 < \eta < 2/\lambda_{\max}$. Under the optimal choice $\eta^* = 2/(\lambda_{\min} + \lambda_{\max})$ the error contracts by the factor*

$$\frac{\kappa - 1}{\kappa + 1}, \quad \kappa = \frac{\lambda_{\max}}{\lambda_{\min}},$$

per iteration, where κ is the condition number of $\mathbf{H}$.

Proof. Since $\nabla J(\theta) = \mathbf{H}\theta$, the iteration is $\theta_{t+1} = (\mathbf{I} - \eta\mathbf{H})\theta_t$. Diagonalise $\mathbf{H} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$ with $\mathbf{Q}$ orthogonal, and write $\mathbf{u}_t = \mathbf{Q}^\top\theta_t$. In these coordinates the iteration decouples completely into d scalar recursions $u_{t+1,j} = (1 - \eta\lambda_j)u_{t,j}$, whose solution is $u_{t,j} = (1 - \eta\lambda_j)^t u_{0,j}$. This tends to zero for every j and every starting point if and only if $|1 - \eta\lambda_j| < 1$ for all j , that is $0 < \eta < 2/\lambda_j$ for all j , which is the single condition $0 < \eta < 2/\lambda_{\max}$.

The asymptotic rate is governed by the slowest mode, $\rho(\eta) = \max_j |1 - \eta\lambda_j| = \max\{|1 - \eta\lambda_{\min}|, |1 - \eta\lambda_{\max}|\}$, because the maximum over the eigenvalue range of the convex function $\lambda \mapsto |1 - \eta\lambda|$ is attained at an endpoint. Minimising ρ over η balances the two endpoint terms: setting $1 - \eta\lambda_{\min} = \eta\lambda_{\max} - 1$ gives $\eta^* = 2/(\lambda_{\min} + \lambda_{\max})$, and substituting back yields

$$\rho(\eta^*) = 1 - \frac{2\lambda_{\min}}{\lambda_{\min} + \lambda_{\max}} = \frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} = \frac{\kappa - 1}{\kappa + 1}. \quad \square$$

Proposition 6.7 is worth more than its modest generality suggests, because near a minimum any smooth loss is approximately quadratic with $\mathbf{H}$ the Hessian. It delivers three lessons at once. Too large an η does not merely slow convergence; it makes the iteration *diverge*, which is why an exploding loss is almost always a learning-rate problem. The attainable rate depends on the condition number κ , not on the raw scale of the loss, which is precisely why input standardisation matters (Section 6.12), since it is the input covariance that shapes $\mathbf{H}$ for the first layer. And there is a single best step size for a given curvature, so tuning η first, on a logarithmic grid, is not superstition but the direct consequence of the analysis.

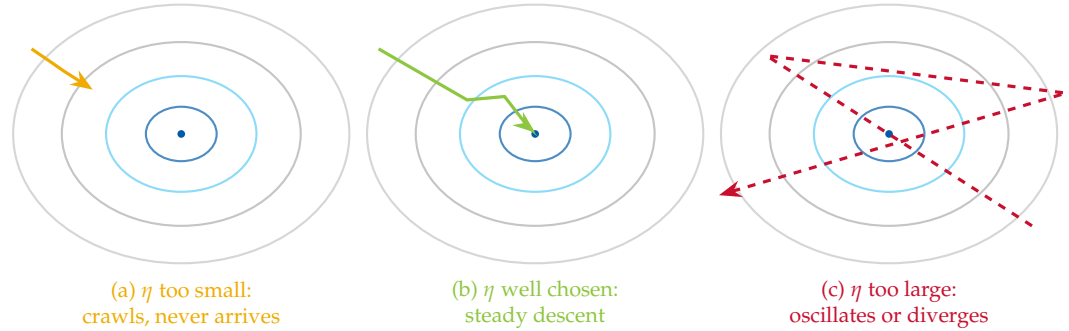


Figure 6.5: The three regimes of Proposition 6.7. Panel (c) corresponds to $\eta > 2/\lambda_{\max}$, where the iteration amplifies the error along the sharpest curvature direction at every step.

6.6 Backpropagation

A network of the size in Example 6.1 has several thousand parameters, and realistic architectures have millions. Computing $\partial J / \partial \theta_k$ separately for

each k by finite differences would require one forward pass per parameter, which is hopeless. Backpropagation computes all of them in a single backward sweep.

Definition 6.6 (Computational graph and the error signal). A *computational graph* is a directed acyclic graph whose nodes are intermediate quantities and whose edges record functional dependence. For an MLP with pre-activations $\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}$ and activations $\mathbf{h}^{(l)} = \phi(\mathbf{z}^{(l)})$, define the *error signal* of layer l as

$$\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}} \in \mathbb{R}^{n_l}. \quad (6.15)$$

Theorem 6.2 (The backpropagation recursion). For an MLP as in Definition 6.2 with elementwise activation ϕ , the error signals satisfy the backward recursion

$$\delta^{(l)} = \left(\mathbf{W}^{(l+1)\top} \delta^{(l+1)} \right) \odot \phi'(\mathbf{z}^{(l)}), \quad l = L-1, \dots, 1, \quad (6.16)$$

where $\odot$ denotes the elementwise (Hadamard) product, and the parameter gradients are

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} \mathbf{h}^{(l-1)\top}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}. \quad (6.17)$$

For a sigmoid output unit with cross-entropy loss, the recursion is initialised by $\delta^{(L)} = \hat{p} - y$.

Proof. Fix a layer $l < L$ and a unit index j . The scalar $z_j^{(l)}$ influences the loss only through the activations of layer l , and specifically only through $h_j^{(l)} = \phi(z_j^{(l)})$, which in turn influences the loss only through the pre-activations of layer $l+1$. The multivariate chain rule therefore gives

$$\delta_j^{(l)} = \frac{\partial \mathcal{L}}{\partial z_j^{(l)}} = \sum_k \frac{\partial \mathcal{L}}{\partial z_k^{(l+1)}} \frac{\partial z_k^{(l+1)}}{\partial h_j^{(l)}} \frac{\partial h_j^{(l)}}{\partial z_j^{(l)}}.$$

The first factor is $\delta_k^{(l+1)}$ by definition. For the second, note that $z_k^{(l+1)} = \sum_m W_{km}^{(l+1)} h_m^{(l)} + b_k^{(l+1)}$, so $\partial z_k^{(l+1)} / \partial h_j^{(l)} = W_{kj}^{(l+1)}$. The third is $\phi'(z_j^{(l)})$ because ϕ acts elementwise. Substituting,

$$\delta_j^{(l)} = \left(\sum_k W_{kj}^{(l+1)} \delta_k^{(l+1)} \right) \phi'(z_j^{(l)}),$$

and recognising the sum as the j -th component of $\mathbf{W}^{(l+1)\top} \delta^{(l+1)}$ gives (6.16).

For the parameter gradients, $z_j^{(l)} = \sum_m W_{jm}^{(l)} h_m^{(l-1)} + b_j^{(l)}$ depends on $W_{jm}^{(l)}$ only through that single term, so

$$\frac{\partial \mathcal{L}}{\partial W_{jm}^{(l)}} = \frac{\partial \mathcal{L}}{\partial z_j^{(l)}} \frac{\partial z_j^{(l)}}{\partial W_{jm}^{(l)}} = \delta_j^{(l)} h_m^{(l-1)},$$

which in matrix form is the outer product in (6.17); similarly $\partial z_j^{(l)} / \partial b_j^{(l)} = 1$ gives the bias gradient. The initialisation $\delta^{(L)} = \hat{p} - y$ is Proposition 6.5. $\square$

Remark 6.2 (Computational cost). Each step of (6.16) requires one matrix–vector product with $\mathbf{W}^{(l+1)\top}$, which costs the same number of multiply–accumulate operations as the forward product with $\mathbf{W}^{(l+1)}$. The outer products in (6.17) add a comparable term. The whole backward sweep therefore costs roughly twice a forward pass, *independently of the number of parameters*. This is the fact that makes deep learning computationally possible at all; it is a consequence of reverse-mode automatic differentiation being efficient for functions $\mathbb{R}^P \rightarrow \mathbb{R}$ with P large (Baydin et al., 2018).

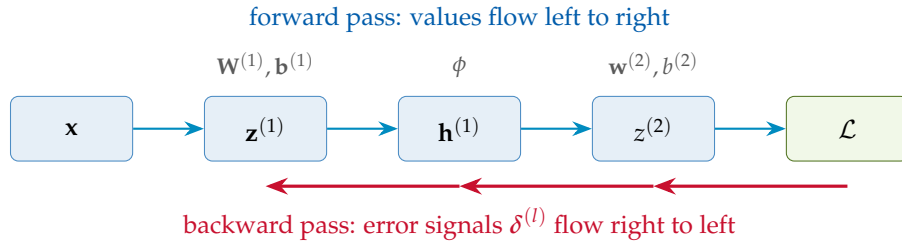


Figure 6.6: The computational graph underlying Theorem 6.2. The forward pass caches every $\mathbf{z}^{(l)}$ and $\mathbf{h}^{(l)}$; the backward pass consumes those caches, which is why training requires more memory than inference.

Example 6.2 (A forward pass by hand). Take two standardised features, one hidden layer of two ReLU units, and a sigmoid output. Let

$$\mathbf{x} = \begin{pmatrix} 0.8 \\ 0.3 \end{pmatrix}, \quad \mathbf{W}^{(1)} = \begin{pmatrix} 1 & -2 \\ 0.5 & 1 \end{pmatrix}, \quad \mathbf{b}^{(1)} = \begin{pmatrix} 0 \\ -0.4 \end{pmatrix}, \quad \mathbf{w}^{(2)} = \begin{pmatrix} 2 \\ -1 \end{pmatrix}, \quad b^{(2)} = -0.1.$$

Note that $\mathbf{W}^{(1)}$ here maps $\mathbf{x}$ to the pre-activations through $\mathbf{z}^{(1)} = \mathbf{W}^{(1)\top} \mathbf{x} + \mathbf{b}^{(1)}$, matching the row-vector convention used by Keras. Then

$$\mathbf{z}^{(1)} = \begin{pmatrix} 1(0.8) + 0.5(0.3) \\ -2(0.8) + 1(0.3) - 0.4 \end{pmatrix} = \begin{pmatrix} 0.95 \\ -1.70 \end{pmatrix}, \quad \mathbf{h}^{(1)} = \text{ReLU}(\mathbf{z}^{(1)}) = \begin{pmatrix} 0.95 \\ 0 \end{pmatrix}.$$

The second unit is inactive for this borrower and will pass no gradient back through its incoming weights on this example. The output logit is $z^{(2)} =$

$2(0.95) - 1(0) - 0.1 = 1.80$, so $\hat{p} = \sigma(1.80) \approx 0.858$. If this borrower did *not* default, $y = 0$, the output error signal is $\delta^{(2)} = \hat{p} - y \approx 0.858$: a large positive signal instructing the network to reduce $z^{(2)}$, which it does chiefly by shrinking the weight attached to the active first unit.

6.7 Optimisers beyond plain gradient descent

6.7.1 Momentum

Definition 6.7 (Momentum). With $\mathbf{g}_t = \nabla_{\theta} J(\theta_t)$, the momentum method maintains a velocity vector

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \mathbf{g}_t, \quad \theta_{t+1} = \theta_t - \eta \mathbf{v}_t, \quad (6.18)$$

with $\mathbf{v}_0 = \mathbf{0}$ and $\beta \in [0, 1)$, typically 0.9.

Proposition 6.8 (Effective step size under momentum). *If the gradient is constant, $\mathbf{g}_t = \mathbf{g}$ for all t , then $\mathbf{v}_t \rightarrow \mathbf{g}/(1 - \beta)$ as $t \rightarrow \infty$, so the asymptotic step length is $\eta/(1 - \beta)$ times the gradient.*

Proof. Unrolling (6.18) with $\mathbf{v}_0 = \mathbf{0}$ gives $\mathbf{v}_t = \sum_{k=0}^{t-1} \beta^k \mathbf{g} = \mathbf{g} \frac{1 - \beta^t}{1 - \beta}$, a finite geometric sum. Since $0 \leq \beta < 1$ we have $\beta^t \rightarrow 0$, hence $\mathbf{v}_t \rightarrow \mathbf{g}/(1 - \beta)$. $\square$

With $\beta = 0.9$ the effective step is ten times larger than plain gradient descent would take along a direction of consistent gradient, while components that alternate in sign, the zig-zag across a narrow valley, partially cancel in the running sum. This is exactly the pathology that Proposition 6.7 attributes to a large condition number. Note the practical corollary: increasing β without reducing η effectively increases the step size and can destabilise a previously stable configuration.

6.7.2 Adam

Definition 6.8 (Adam). Adam (Kingma and Ba, 2015) maintains exponential moving averages of the gradient and of its elementwise square,

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad (6.19)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2, \quad (6.20)$$

with $\mathbf{m}_0 = \mathbf{v}_0 = \mathbf{0}$, and applies the bias-corrected update

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}, \quad \theta_{t+1} = \theta_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}. \quad (6.21)$$

Standard values are $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ and $\eta = 10^{-3}$.

Proposition 6.9 (Why the bias correction is necessary). *Suppose the gradients $\mathbf{g}_1, \mathbf{g}_2, \dots$ are drawn from a stationary distribution with mean $\bar{\mathbf{g}}$. Then $\mathbb{E}[\mathbf{m}_t] = (1 - \beta_1^t) \bar{\mathbf{g}}$, so the uncorrected average underestimates $\bar{\mathbf{g}}$, severely so for small t ; the corrected estimate $\hat{\mathbf{m}}_t$ is unbiased.*

Proof. Unrolling the recursion from $\mathbf{m}_0 = \mathbf{0}$,

$$\mathbf{m}_t = (1 - \beta_1) \sum_{k=1}^t \beta_1^{t-k} \mathbf{g}_k.$$

Taking expectations and using stationarity, $\mathbb{E}[\mathbf{m}_t] = (1 - \beta_1) \bar{\mathbf{g}} \sum_{k=1}^t \beta_1^{t-k}$. The sum is geometric with t terms and ratio β_1 , equal to $(1 - \beta_1^t)/(1 - \beta_1)$, so $\mathbb{E}[\mathbf{m}_t] = (1 - \beta_1^t) \bar{\mathbf{g}}$. Dividing by $1 - \beta_1^t$ gives $\mathbb{E}[\hat{\mathbf{m}}_t] = \bar{\mathbf{g}}$. $\square$

At $t = 1$ with $\beta_1 = 0.9$ the uncorrected estimate is only 10% of the true mean, and with $\beta_2 = 0.999$ the second-moment estimate is 0.1% of its target; without correction the ratio in (6.21) would be wildly mis-scaled during exactly the early steps that determine which basin the optimiser enters.

The division by $\sqrt{\hat{\mathbf{v}}_t}$ gives each parameter its own effective step size, inversely proportional to the recent magnitude of its gradient. This matters greatly for embeddings: the row of the embedding table belonging to a rural agent seen twelve times in the training set receives a gradient on very few batches, and Adam compensates by taking larger steps when it does. Loshchilov and Hutter (2019) show that ℓ_2 regularisation implemented by adding $\lambda \theta$ to the gradient interacts badly with this rescaling, and propose AdamW, which decouples weight decay from the adaptive term; AdamW is the default for the Transformer models of Chapter 9.

6.7.3 Learning-rate schedules

A constant η is a compromise between the large steps that are useful early and the small steps needed to settle. Two schedules cover most needs. Reduce-on-plateau multiplies η by a factor (typically 0.5) whenever a monitored validation metric fails to improve for a fixed number of epochs; it is self-tuning and hard to misuse. Cosine decay anneals η smoothly from $\eta_{\max}$ to near zero over the planned training horizon,

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \frac{\pi t}{T} \right),$$

usually preceded by a short linear warm-up that avoids destabilising the adaptive moments at t small. For the tabular networks in this chapter, reduce-on-plateau paired with early stopping is sufficient; the cosine schedule becomes important in Chapter ??.

6.8 Pathologies of deep training

Theorem 6.2 makes the central difficulty visible. The error signal reaching layer 1 is obtained by *multiplying* $L - 1$ factors together, each factor a weight matrix transposed and a diagonal matrix of activation derivatives. Repeated multiplication of quantities that are systematically smaller or larger than one is a recipe for exponential decay or exponential growth.

Proposition 6.10 (Exponential bounds on the propagated error). *Let $\|\cdot\|$ denote the Euclidean norm on vectors and the induced spectral norm on matrices, and suppose $|\phi'(z)| \leq \gamma$ for all z and $\|\mathbf{W}^{(l)}\| \leq w$ for every layer. Then for the MLP of Definition 6.2,*

$$\|\delta^{(1)}\| \leq (\gamma w)^{L-1} \|\delta^{(L)}\|.$$

In particular, for sigmoid activations $\gamma = \frac{1}{4}$ by Proposition 6.2, so the bound is $(w/4)^{L-1} \|\delta^{(L)}\|$, which decays geometrically whenever $w < 4$.

Proof. From (6.16), and using $\|\mathbf{a} \odot \mathbf{b}\| \leq (\max_j |b_j|) \|\mathbf{a}\|$,

$$\|\delta^{(l)}\| \leq \gamma \|\mathbf{W}^{(l+1)\top} \delta^{(l+1)}\| \leq \gamma \|\mathbf{W}^{(l+1)}\| \|\delta^{(l+1)}\| \leq \gamma w \|\delta^{(l+1)}\|,$$

where the middle inequality is the definition of the induced norm together with $\|\mathbf{A}^\top\| = \|\mathbf{A}\|$. Applying this $L - 1$ times from $l = L - 1$ down to $l = 1$ gives the claim. $\square$

The bound is only an upper bound, so it certifies vanishing rather than guaranteeing explosion, but the mechanism runs symmetrically in both directions: if typical factors exceed one, the error signal grows geometrically and the loss overflows to NaN within a few steps. Three practical signatures follow.

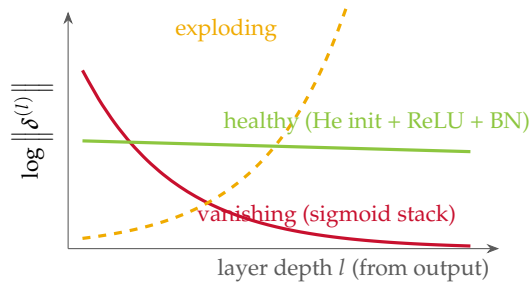


Figure 6.7: The three gradient-flow regimes of Proposition 6.10. The diagnostic is available directly in TensorBoard as per-layer gradient histograms; a network whose early layers show gradient norms orders of magnitude below the late layers is not learning its early representations at all.

6.9 Initialisation

Initialisation is not a detail. It decides where on Figure 6.7 training begins.

Two naive choices fail immediately. Initialising all weights to zero makes every unit in a layer compute the identical function and receive the identical gradient, so they remain identical forever, the network has the capacity of a single unit per layer. Initialising with large random values drives pre-activations into the saturated regions of Figure 6.4(a), where gradients vanish. What is needed is a scale that keeps the variance of the signal roughly constant as it passes through layers.

Proposition 6.11 (He initialisation for rectified units). *Consider a dense layer $z = \sum_{j=1}^{n_{in}} w_j x_j$ in which the weights w_j are i.i.d. with mean zero and variance ς^2 , independent of the inputs x_j , and the inputs are i.i.d. with mean zero and variance τ^2 . Then $\text{Var}(z) = n_{in} \varsigma^2 \tau^2$. If in addition the inputs are the ReLU activations of a preceding layer whose pre-activations were symmetrically distributed about zero with variance τ_{pre}^2 , then $\mathbb{E}[x_j^2] = \frac{1}{2} \tau_{pre}^2$, and the choice*

$$\varsigma^2 = \frac{2}{n_{in}}$$

preserves the second moment of the pre-activations from layer to layer.

Proof. Since the w_j have mean zero and are independent of the x_j , each term has $\mathbb{E}[w_j x_j] = \mathbb{E}[w_j] \mathbb{E}[x_j] = 0$, so $\mathbb{E}[z] = 0$ and $\text{Var}(z) = \mathbb{E}[z^2]$. Expanding the square and using independence to kill the cross terms,

$$\mathbb{E}[z^2] = \sum_{j=1}^{n_{in}} \mathbb{E}[w_j^2] \mathbb{E}[x_j^2] = n_{in} \varsigma^2 \mathbb{E}[x_1^2].$$

When $x_j = \max(0, u_j)$ with u_j symmetric about zero, we have $\mathbb{E}[x_j^2] = \mathbb{E}[u_j^2 \mathbf{1}\{u_j > 0\}] = \frac{1}{2} \mathbb{E}[u_j^2] = \frac{1}{2} \tau_{pre}^2$, because symmetry makes the positive and negative half-lines contribute equally to $\mathbb{E}[u_j^2]$. Substituting, $\mathbb{E}[z^2] = \frac{1}{2} n_{in} \varsigma^2 \tau_{pre}^2$, and setting this equal to τ_{pre}^2 gives $\varsigma^2 = 2/n_{in}$. $\square$

The factor of two is precisely the compensation for the half of the input distribution that ReLU sets to zero (He et al., 2015). For symmetric saturating activations no such correction is needed, and Glorot initialisation (Glorot and Bengio, 2010) instead balances the forward and backward variance requirements by taking $\varsigma^2 = 2/(n_{in} + n_{out})$.

For exploding gradients, initialisation alone is not enough; the standard remedy is *gradient clipping*, which rescales the gradient vector whenever its norm exceeds a threshold c :

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1, \frac{c}{\|\mathbf{g}\|}\right).$$

Table 6.3: Initialiser selection. In Keras these are passed as the `kernel_initializer` argument; biases are conventionally initialised to zero, which is safe because the weights already break symmetry.

Hidden activation	Initialiser	Keras name
ReLU, leaky ReLU, ELU	He (variance $2/n_{\text{in}}$)	<code>he_normal</code>
tanh, sigmoid, softmax	Glorot	<code>glorot_uniform</code>
SELU (self-normalising)	LeCun	<code>lecun_normal</code>

This preserves the direction of the update while bounding its length, and is available in Keras as the `clipnorm` argument to any optimiser. For architectures deeper than a handful of layers, skip connections (Chapter 7) provide a path along which gradients travel without being multiplied by weight matrices at all, and are the structural rather than numerical solution to the same problem.

6.10 Normalisation layers

Definition 6.9 (Batch normalisation). Let $z_1, \dots, z_B$ be the pre-activations of one unit across a mini-batch of size B , with batch mean μ_B and batch variance σ_B^2 . Batch normalisation (Ioffe and Szegedy, 2015) replaces z_i by

$$\tilde{z}_i = \gamma \hat{z}_i + \beta, \quad \hat{z}_i = \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad (6.22)$$

where γ and β are learned parameters, one pair per unit. At inference time μ_B and σ_B^2 are replaced by moving averages accumulated during training.

Three effects follow. Normalising keeps pre-activations in the well-conditioned region of the activation function, which by Proposition 6.7 improves the conditioning of the effective optimisation problem and permits a larger learning rate. The learned γ and β mean nothing is lost: the layer can undo the normalisation exactly if that is what minimises the loss. And because μ_B and σ_B are computed from a random mini-batch, each example is normalised slightly differently on each epoch, which injects noise and regularises weakly.

The train/inference asymmetry is a deployment bug factory

Batch normalisation behaves differently in training and inference mode. During training it uses batch statistics; at inference it must use the accumulated moving averages, because a production request may contain a single row and the “batch statistics” of one row are degenerate. Keras handles this automatically through the training flag, and `model.predict` sets it correctly. Hand-rolled serving code that calls the layer directly, or that re-implements the forward pass in another language for latency reasons, is the classic way to ship a model that scored beautifully offline and produces nonsense online. This failure mode recurs in Chapter 10; the general principle is that the exported artefact must contain the whole pipeline, including its inference-mode behaviour, not just the weights.

For small batches, or for the sequence models of Chapter 8, *layer normalisation* (Ba et al., 2016) is the steadier choice: it normalises across the features of a single example rather than across the batch, so its behaviour is identical in training and inference and independent of batch size.

6.11 Regularisation

With several thousand parameters and a hundred thousand rows, our challenger is not obviously over-parameterised; add the embedding table of Example 6.1 and it certainly is. Three tools, used in combination, control the resulting variance.

6.11.1 Dropout

Definition 6.10 (Dropout). During training, dropout (Srivastava et al., 2014) multiplies each unit’s activation by an independent Bernoulli mask, $\tilde{h}_j = m_j h_j$ with $m_j \sim \text{Bernoulli}(1 - p)$, and rescales by $1/(1 - p)$ so that $\mathbb{E}[\tilde{h}_j] = h_j$. At inference the mask is removed and no rescaling is applied. The parameter p is the *drop rate*, typically between 0.2 and 0.5.

The intuition connects directly to Chapter 4: bagging reduces variance by averaging models that were fitted to perturbed data, and dropout does something similar by averaging sub-networks that share their weights. For the simplest case this intuition can be made exact.

Proposition 6.12 (Dropout on a linear model is a scaled ridge penalty). Consider the linear model $\hat{y} = \mathbf{w}^\top (\mathbf{m} \odot \mathbf{x})$ with independent masks $m_j \sim \text{Bernoulli}(q)$, $q = 1 - p$, and no rescaling. Then the expected

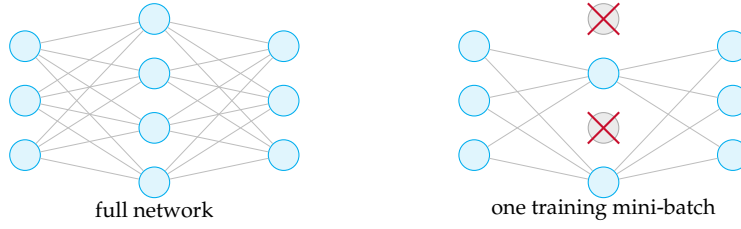


Figure 6.8: Dropout thins the network differently on every mini-batch. Over the course of training, an exponentially large family of sub-networks is fitted, and inference approximates their average.

squared-error loss on a single example satisfies

$$\mathbb{E}_{\mathbf{m}} \left[\left(y - \mathbf{w}^\top (\mathbf{m} \odot \mathbf{x}) \right)^2 \right] = \left(y - q \mathbf{w}^\top \mathbf{x} \right)^2 + q(1-q) \sum_{j=1}^d w_j^2 x_j^2.$$

That is, dropout is equivalent to fitting the deterministically shrunk model $q\mathbf{w}$ together with an ℓ_2 penalty whose per-coordinate weight is proportional to x_j^2 .

Proof. Write $S = \mathbf{w}^\top (\mathbf{m} \odot \mathbf{x}) = \sum_j w_j m_j x_j$. Since the m_j are independent with $\mathbb{E}[m_j] = q$ and $\text{Var}(m_j) = q(1-q)$, we have $\mathbb{E}[S] = q \mathbf{w}^\top \mathbf{x}$ and, by independence across j , $\text{Var}(S) = \sum_j w_j^2 x_j^2 \text{Var}(m_j) = q(1-q) \sum_j w_j^2 x_j^2$. Now use the bias–variance decomposition of the expected squared error about the fixed target y :

$$\mathbb{E} [(y - S)^2] = (y - \mathbb{E}[S])^2 + \text{Var}(S),$$

which is obtained by adding and subtracting $\mathbb{E}[S]$ inside the square and noting that the cross term vanishes. Substituting the two moments computed above gives the stated identity. $\square$

The proposition also explains why dropout is applied to standardised inputs in practice: if the x_j^2 differ by orders of magnitude, so does the implied penalty, and the regularisation becomes an arbitrary function of measurement units.

6.11.2 Early stopping and weight decay

Early stopping monitors a validation metric each epoch, halts when it has failed to improve for a fixed *patience*, and restores the parameters from the best epoch. It is the cheapest and most reliable regulariser available, and it converts the number of epochs from a hyperparameter you must guess into a quantity the data determines. Two design points matter. The monitored metric should be the one that matters operationally, for us, validation PR-AUC, not validation loss, and `restore_best_weights` must be enabled, or you keep the weights from the worst epoch of the patience window.

Weight decay adds $\lambda \|\theta\|_2^2$ to the objective, shrinking parameters toward zero exactly as ridge regression does in Chapter 3. For networks trained with Adam, use the decoupled form (AdamW) rather than adding the penalty to the loss (Loshchilov and Hutter, 2019).

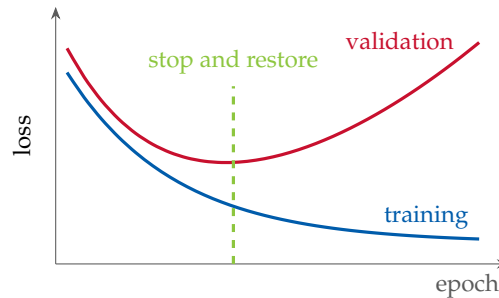


Figure 6.9: Early stopping. The gap between the curves measures overfitting; the minimum of the validation curve, not the end of the training curve, defines the model you ship.

6.12 Input scaling and conditioning

Tree ensembles are invariant to monotone rescaling of any single feature, because a split threshold simply moves with the transformation. Gradient descent enjoys no such invariance, and Proposition 6.7 says exactly why: the achievable convergence rate is governed by the condition number of the curvature matrix, and for the first layer that matrix inherits the scale structure of the input covariance. A feature measured in KES with values in the millions, sitting beside a ratio in $[0, 1]$, produces an extreme condition number and therefore an extremely slow or unstable optimisation.

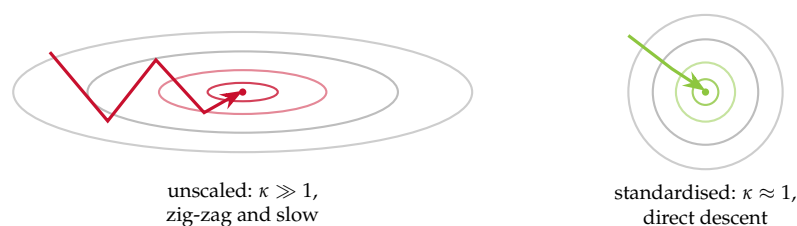


Figure 6.10: The geometric content of Proposition 6.7. Standardisation does not change the model class; it changes the geometry of the surface the optimiser must traverse.

The operational rules follow directly from Chapter 2 and are unchanged by the switch to networks.

1. **Standardise, fitting on the training fold only.** Compute μ and σ on the training split and apply them to validation and test. Fitting the

scaler on the full dataset leaks distributional information about the evaluation data into training.

2. **Impute, and record the imputation.** Networks cannot ingest NaN; a single missing value poisons the entire forward pass through the arithmetic. Impute, and add a binary missingness indicator so that the fact of absence remains available to the model.
3. **Shuffle within the training window each epoch,** so that mini-batches are representative, while leaving the temporal ordering of the train/validation/test split itself untouched.
4. **Fix the seeds and record the versions.** One call to `set_random_seed` in `tf.keras.utils` covers Python, NumPy and TensorFlow at once. Reproducibility is not a nicety when a model decision must be defended to a supervisor eighteen months later.

Scaling before splitting

The most common leakage in this chapter's lab is fitting `StandardScaler` on the whole dataset before splitting. The resulting validation score is optimistic and the optimism is invisible: nothing crashes, no warning is emitted, and the number simply cannot be reproduced in production. The order is always split, then fit the scaler on the training fold, then transform every fold with that same fitted scaler, and the fitted scaler travels with the model into the exported artefact.

6.13 Entity embeddings

We now come to the feature that motivated the challenger in the first place: agent identity, with roughly 120,000 levels.

Definition 6.11 (Embedding layer). Let $\mathcal{V} = \{1, \dots, V\}$ be a finite vocabulary and let $\mathbf{E} \in \mathbb{R}^{V \times k}$ be a learned matrix. The *embedding* of level v is the row $\mathbf{E}_{v,:} \in \mathbb{R}^k$, and $k \ll V$ is the *embedding dimension*. The matrix $\mathbf{E}$ is a parameter of the model and is trained by the same gradient descent as every other weight.

Proposition 6.13 (An embedding lookup is a one-hot matrix product). Let $\mathbf{o}_v \in \mathbb{R}^V$ be the one-hot indicator of level v . Then $\mathbf{E}^\top \mathbf{o}_v = \mathbf{E}_{v,:}^\top$. Consequently an embedding layer is exactly a dense layer applied to a one-hot encoding, with the bias omitted and the matrix product implemented as a table lookup.

Proof. The j -th component of $\mathbf{E}^\top \mathbf{o}_v$ is $\sum_{u=1}^V E_{uj}(\mathbf{o}_v)_u$. All terms with $u \neq v$ vanish because $(\mathbf{o}_v)_u = 0$ there, leaving E_{vj} , which is the j -th component of row v . $\square$

Proposition 6.13 tells us that nothing new is being learned in principle, but everything changes in practice. Materialising a 120,000-column one-hot matrix is infeasible in memory, whereas a lookup into a $120,000 \times 16$ table costs a single indexing operation. More importantly, the dense representation *shares statistical strength*: two agents that appear in similar borrower populations receive similar embedding vectors, so an agent seen forty times in training borrows information from agents that behave like it. One-hot encoding, by contrast, treats every level as orthogonal to every other and learns nothing transferable.

two of the sixteen embedding dimensions

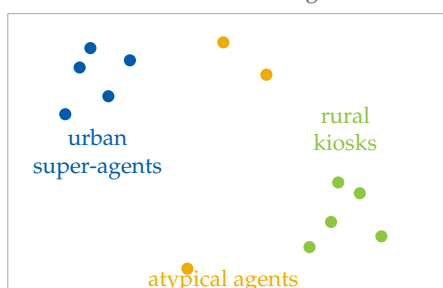


Figure 6.11: A two-dimensional projection of a learned agent embedding. Structure of this kind is a by-product of fitting the credit model, not an input to it; inspecting it is a useful sanity check and a source of hypotheses for the risk team. **Illustrative, regenerate from the lab notebook.**

Three design decisions accompany every embedding.

Dimension. A common heuristic is $k \approx \min(50, \lceil 1.6 V^{0.56} \rceil)$, or more simply k between 8 and 32 for vocabularies in the tens of thousands. The dimension should be treated as a hyperparameter, but it is a low-priority one relative to the learning rate.

Regularisation. Example 6.1 showed the embedding table dominating the parameter count. Rare levels are the danger: a level appearing three times can have its sixteen coordinates fitted almost freely to those three labels. Remedies are to apply weight decay to the embedding matrix, to add dropout after the concatenation, and, most effective, to collapse levels below a minimum support threshold into a shared “rare agent” index before training.

Out-of-vocabulary handling. A production request will eventually arrive carrying an agent identifier that did not exist when the model was trained. This is not an edge case; with agent onboarding running continuously it is a weekly certainty. Reserve index 0 for unseen levels, map any unrecognised identifier to it at inference time, and ensure that index 0 is trained, for example by mapping the rare-level bucket to it, so that its embedding is a sensible average rather than a random initialisation.

Silent out-of-vocabulary failure

An embedding layer indexed with a value $\geq V$ raises an error in eager mode but may produce undefined behaviour inside a compiled serving graph. Worse, a lookup table that silently maps unknown identifiers to index 0 *without* index 0 having been trained will return the initialisation vector, producing confident nonsense. Test the unseen-agent path explicitly, with an assertion in the serving code and a unit test in the repository. Drift in the agent vocabulary is one of the monitored quantities in Chapter 11.

6.14 Implementation in TensorFlow and Keras

TensorFlow provides the tensor algebra, reverse-mode automatic differentiation, hardware placement and serialisation; Keras (`tf.keras`) provides the layer, loss, optimiser, metric and callback abstractions on top. Models can be built three ways: Sequential for a plain stack, the functional API for arbitrary directed acyclic graphs, and subclassing for full Python control. The challenger needs the functional API, because it has two input branches.

6.14.1 The training loop, written out

Before using `model.fit`, it is worth writing the loop it replaces. The following trains a logistic regression, an MLP with no hidden layer, using nothing but the tape.

```
import tensorflow as tf

tf.keras.utils.set_random_seed(42)

W = tf.Variable(tf.random.normal([n_features, 1], stddev=0.1))
b = tf.Variable(tf.zeros([1]))
opt = tf.keras.optimizers.SGD(learning_rate=0.1)

for step in range(500):
    with tf.GradientTape() as tape:          # record the forward
        operations                          # operations
        logits = X_train @ W + b           # note: logits, not
        probabilities                       # probabilities
        loss = tf.reduce_mean(
            tf.nn.sigmoid_cross_entropy_with_logits(
                labels=y_train, logits=logits))
        grads = tape.gradient(loss, [W, b]) # one backward sweep
        opt.apply_gradients(zip(grads, [W, b])) # theta <- theta - eta * g

    if step % 100 == 0:
        print(f"step {step:4d}  loss {loss.numpy():.4f}")
```

Listing 6.1: The training loop of Definition 6.5 written explicitly with `tf.GradientTape`. Every element of Section 6.5 and Theorem 6.2 appears here: the forward pass, the recorded graph, the backward sweep, the update.

Two details in Listing 6.1 are load-bearing. The tape records operations only inside its context, so any computation you want differentiated must happen within the `with` block. And the loss consumes *logits*: the fused `sigmoid_cross_entropy_with_logits` realises the cancellation of Proposition 6.5 in a numerically stable way, whereas computing σ first and then taking a logarithm invites $\log 0$.

6.14.2 The baseline network

```
from tensorflow.keras import layers

model = tf.keras.Sequential([
    layers.Input(shape=(n_features,)),           # already standardised
    layers.Dense(64, activation="relu", kernel_initializer="he_normal"),
    layers.BatchNormalization(),
    layers.Dropout(0.3),
    layers.Dense(32, activation="relu", kernel_initializer="he_normal"),
    layers.Dropout(0.3),
    layers.Dense(1, activation="sigmoid"),       # P(default | x)
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss="binary_crossentropy",
    metrics=[tf.keras.metrics.AUC(name="roc_auc"),
             tf.keras.metrics.AUC(curve="PR", name="pr_auc"),
             tf.keras.metrics.Recall(name="recall")],
)

model.summary()                                # read the parameter count before training
```

Listing 6.2: The baseline MLP. Every choice here has been justified earlier in the chapter: He initialisation (Proposition 6.11), batch normalisation (Definition 6.9), dropout (Definition 6.10), and a sigmoid output paired with binary cross-entropy (Proposition 6.4).

```
callbacks = [
    tf.keras.callbacks.EarlyStopping(monitor="val_pr_auc", mode="max",
                                     patience=10, restore_best_weights=True),
    tf.keras.callbacks.ReduceLROnPlateau(monitor="val_loss",
                                          factor=0.5, patience=4),
    tf.keras.callbacks.TensorBoard(log_dir="logs/mlp_v1",
                                   histogram_freq=1),    # gradient
    histograms
]
```

```

history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=200,
    batch_size=256,
    class_weight={0: 1.0, 1: pos_weight},    # imbalance, ~4% default rate
    callbacks=callbacks,
    verbose=2,
)

```

Listing 6.3: Training with callbacks. The epoch count is a ceiling, not a decision: early stopping decides when to halt and which weights to keep.

Setting `histogram_freq=1` makes TensorBoard log per-layer weight and gradient distributions, which is how the three regimes of Figure 6.7 are diagnosed in practice rather than in theory.

6.14.3 The embedding challenger

```

num_in  = layers.Input(shape=(n_features,), name="numeric")
agent_in = layers.Input(shape=(), dtype="int64", name="agent_id")

emb = layers.Embedding(input_dim=n_agents + 1,      # +1 for the OOV bucket
                       output_dim=16,
                       embeddings_regularizer=tf.keras.regularizers.l2(1e-6),
                       name="agent_embedding")(agent_in)
emb = layers.Flatten()(emb)

x = layers.Concatenate()([num_in, emb])
x = layers.Dense(64, activation="relu", kernel_initializer="he_normal")(x)
x = layers.Dropout(0.3)(x)
x = layers.Dense(32, activation="relu", kernel_initializer="he_normal")(x)
out = layers.Dense(1, activation="sigmoid")(x)

model = tf.keras.Model(inputs=[num_in, agent_in], outputs=out)

```

Listing 6.4: The functional API, with a numeric branch and an agent-identity branch fused by concatenation. Index 0 of the embedding table is reserved for unseen agents.

6.14.4 Hyperparameter search and debugging

The search space, in descending order of importance, is: the learning rate on a logarithmic grid over $[10^{-4}, 10^{-2}]$; the width, in $\{32, 64, 128, 256\}$, and the depth, in $\{1, 2, 3\}$; the dropout rate, in $\{0.0, 0.2, 0.3, 0.5\}$; the batch size, in $\{128, 256, 512\}$; and the embedding dimension, in $\{8, 16, 32\}$. Random or Bayesian search dominates grid search for the reason established in Chapter 4, with a fixed budget, random sampling explores many more distinct

values of the one hyperparameter that actually matters. Keras Tuner with the Hyperband strategy, or Optuna wrapping a Keras objective, are the standard tools; both should be combined with early stopping so that unpromising trials die cheaply.

Table 6.4: Debugging triage. Each symptom maps to a mechanism established earlier in this chapter, which is why the fixes are principled rather than superstitious.

Symptom	Mechanism	First remedy
Loss becomes NaN	$\eta > 2/\lambda_{\max}$, or $\log 0$	lower η ; use fused logit losses; <code>clipnorm</code>
Loss frozen near 0.693	dead ReLUs; $0.693 = \ln 2$ is the loss of $\hat{p} \equiv \frac{1}{2}$	He initialisation; lower η ; leaky ReLU
Training PR-AUC $\gg$ validation	overfitting	dropout, weight decay, early stopping
Validation metric jumps between epochs	mini-batch gradient variance $\propto 1/B$	larger batch; batch normalisation
Convergence painfully slow	large condition number κ	standardise inputs; add normalisation layers
Correct offline, wrong in the API	train/inference mode mismatch	export the whole pipeline; test single-row inference

6.15 Evaluation, calibration and governance

A trained network is not yet a credit model. Three further checks stand between the two.

6.15.1 Ranking versus calibration

Chapter 3 established PR-AUC and cost-weighted decision thresholds as the operative metrics under a 4% base rate. Those carry over unchanged. What is new is that networks trained with class weighting or heavy regularisation are frequently *miscalibrated*: they rank borrowers correctly while assigning probabilities that do not match observed frequencies (Guo et al., 2017). If the score feeds only an accept/reject threshold, miscalibration is tolerable; if it feeds risk-based pricing, expected-loss provisioning or a capital calculation, it is not.

Temperature scaling is the standard remedy: fit a single scalar $T > 0$ on the validation set by minimising the loss of $\sigma(z/T)$, then apply it at inference. Its most useful property is that it costs you nothing in ranking.

Proposition 6.14 (Temperature scaling preserves the ranking). *For any $T > 0$, the map $z \mapsto \sigma(z/T)$ is strictly increasing. Consequently, for any two examples, $\sigma(z_1/T) < \sigma(z_2/T)$ if and only if $z_1 < z_2$; the induced ordering of the sample*

is unchanged, and every ranking metric that depends on the scores only through their order, ROC-AUC, PR-AUC, Kendall's τ , is invariant to T .

Proof. The map $z \mapsto z/T$ is strictly increasing because $T > 0$, and σ is strictly increasing because $\sigma'(z) = \sigma(z)(1 - \sigma(z)) > 0$ for all z by Proposition 6.2 and $\sigma(z) \in (0, 1)$. The composition of strictly increasing maps is strictly increasing, hence order-preserving and injective. Ranking metrics are functions of the induced order alone, so they take the same value before and after scaling. $\square$

Calibration is therefore a free improvement in the only dimension where the network was deficient, and it should be a standard post-processing step, fitted on the validation fold and frozen alongside the weights.

6.15.2 Explanation

A rejected applicant is entitled to a reason, and “the network decided” is not one. Two families of methods apply. Gradient-based attributions, notably integrated gradients (Sundararajan et al., 2017), attribute the prediction to input features by integrating the gradient along a path from a baseline input to the actual input; they are cheap and natural for differentiable models. Sampling-based attributions such as SHAP (Lundberg and Lee, 2017) apply equally to the network and to the champion ensemble, which makes them the better choice here for the specific reason that the two models’ attributions can then be compared on the same footing.

6.15.3 Fairness and segment-level performance

Aggregate PR-AUC hides compensating errors. A network can improve overall performance while degrading recall in rural counties, simply because urban borrowers dominate the training distribution and the loss is an unweighted average over examples. The evaluation protocol must therefore be run *per segment*, county, gender where lawfully processed, device tier, agent type, and reported as a table, not a single number.

Kenya Context

Two instruments govern this model in Kenya, and both bear directly on the technical choices made in this chapter.

The **Data Protection Act, 2019** confers a right in respect of automated decision-making that produces legal or similarly significant effects. A credit decline is such an effect. In practice this means: the lawful basis for processing must be documented; the attribution method of Section 6.15 must be capable of producing a specific, intelligible reason for an individual decision, not merely a global feature ranking; and data minimisation applies to the embedding inputs as much as to anything else, an agent identifier is personal data about the agent.

The **Central Bank of Kenya Digital Credit Providers Regulations, 2022** require licensed providers to demonstrate that pricing and credit decisions are non-discriminatory and that models are subject to governance and periodic review. For a challenger model this translates into concrete artefacts: the champion–challenger comparison on identical frozen splits; the per-segment performance and calibration tables; the documented decision on whether to promote the challenger; and a monitoring plan for the agent vocabulary and score distribution once deployed.

Neither instrument prohibits neural networks. Both make undocumented ones indefensible. The practical consequence is that the model card is part of the deliverable, not an afterthought: data window, split definition, preprocessing, architecture, hyperparameter search, metrics overall and by segment, calibration method, attribution method, known failure modes, and the retraining trigger.

Table 6.5: Champion–challenger summary of the form the credit committee should receive. **All figures are PLACEHOLDERS** and must be regenerated from `lab06_deep_learning.ipynb` on the frozen splits before any distribution of this chapter.

Metric (test fold)	LightGBM	MLP	MLP + embeddings
PR-AUC	0.412	0.389	0.428
ROC-AUC	0.842	0.831	0.851
Recall at 10% alert budget	0.518	0.494	0.539
Expected cost per 1,000 applications (KES)	96,400	101,200	92,700
Brier score after calibration	0.031	0.033	0.030
Worst-county PR-AUC	0.331	0.298	0.344

What a challenger must demonstrate

Promotion of the challenger requires four things simultaneously: a better cost-weighted outcome on the test fold; no material degradation in any monitored segment; calibration adequate for the downstream use; and an operational story, training time, serving latency, retraining cadence, out-of-vocabulary handling, that the engineering team has actually tested. A win on aggregate PR-AUC alone is not a case for deployment.

Lab 6: An MLP Challenger in TensorFlow (1.5 h)

The notebook `lab06_deep_learning.ipynb` implements the challenger described throughout this chapter, on the frozen splits established in Chapter 3.

1. **Setup (10 min).** Load the credit dataset with the frozen temporal splits. Build a `scikit-learn` preprocessing pipeline that imputes, adds missingness indicators and standardises, fitted on the training fold only.
2. **The loop by hand (15 min).** Reproduce Listing 6.1. Plot the loss trajectory. Verify empirically that the gradient returned by the tape matches Proposition 6.5 on a single example.
3. **Baseline MLP (20 min).** Build and train Listings 6.2 and 6.3. Record validation PR-AUC and the parameter count.
4. **Embeddings (20 min).** Extend to Listing 6.4. Collapse agents below a minimum support into the shared bucket, reserve index 0 for unseen agents, and verify the out-of-vocabulary path with an explicit test.
5. **Deliberate failure (15 min).** Break the model in three controlled ways, remove input standardisation, set $\eta = 1$, and replace all hidden activations by sigmoid in a five-layer stack, and match each resulting training curve to a row of Table 6.4.
6. **Verdict (10 min).** Compare all three models on the test fold using PR-AUC, expected cost and per-county calibration. Write the promotion recommendation, with reasons, in the final notebook cell.

Deliverable: the completed notebook, the `logs/` directory for TensorBoard, and a one-page model card, committed to your repository.

Exercises

1. **Affine collapse.** A colleague proposes a network with three dense layers of widths 128, 64 and 32, all with linear activations, followed by a sigmoid output. Using Proposition 6.1, state the model class this network actually represents, and compute how many parameters it wastes relative to the equivalent minimal model on $d = 40$ inputs.
2. **Sigmoid saturation.** Using Proposition 6.2, find the values of z for which $\sigma'(z) < 0.01$. Then, using Proposition 6.10, compute how many sigmoid layers with $\|\mathbf{W}\| = 1$ suffice to reduce the propagated error signal by a factor of 10^6 .
3. **Forward and backward by hand.** For the network of Example 6.2, take a second borrower with $\mathbf{x} = (-0.5, 1.2)^\top$ and true label $y = 1$. Compute $\mathbf{z}^{(1)}$, $\mathbf{h}^{(1)}$, $\hat{p}$ and $\delta^{(2)}$. Then apply Theorem 6.2 to obtain $\partial\mathcal{L}/\partial\mathbf{w}^{(2)}$, $\delta^{(1)}$ and $\partial\mathcal{L}/\partial\mathbf{W}^{(1)}$. State whether either hidden unit fails to receive an update from this example, and contrast your answer with Example 6.2.
4. **The cancellation, and what breaks without it.** Derive $\partial\mathcal{L}/\partial z$ for a sigmoid output paired with *squared error* rather than cross-entropy. Show that the result contains a factor $\sigma'(z)$, and explain in two or three sentences why this makes squared error a poor choice for classification with saturating outputs.
5. **Batch size and gradient noise.** Using Proposition 6.6, determine the factor by which the standard deviation of the mini-batch gradient changes when the batch size grows from 64 to 512. If a single step costs time proportional to B , is the trade favourable? State the assumption under which your answer holds.
6. **Learning rate and curvature.** A quadratic loss has Hessian eigenvalues ranging from $\lambda_{\min} = 0.02$ to $\lambda_{\max} = 8$. Using Proposition 6.7, compute the largest stable learning rate, the optimal learning rate, the contraction factor at that optimum, and the number of iterations required to reduce the error by a factor of 100.
7. **He initialisation.** A dense layer has $n_{\text{in}} = 256$ and ReLU activations. Using Proposition 6.11, give the standard deviation of the He normal initialiser. Then repeat the derivation for leaky ReLU with slope α , and show that the appropriate variance is $2 / ((1 + \alpha^2)n_{\text{in}})$.
8. **Dropout as ridge.** Using Proposition 6.12, state the implied per-coordinate penalty weight when $p = 0.5$. Explain what happens to the implied penalty if one feature is left unstandardised with values in the millions, and relate your answer to Section 6.12.

9. **Embedding budget.** Your lender has 120,000 agents, of which 71% appear fewer than five times in the training set. Compute the embedding table size at $k = 16$ with and without collapsing low-support agents into a single bucket. Then argue, in a short paragraph, which of the two variants you would submit to the risk committee, and what evidence would settle the question empirically.
10. **Calibration and ranking.** Using Proposition 6.14, decide which of the following quantities change when temperature scaling with $T = 1.8$ is applied to a trained model: ROC-AUC, PR-AUC, the Brier score, the expected cost at a threshold of $\hat{p} = 0.15$, and the expected cost at the *cost-optimal* threshold re-derived after scaling. Justify each answer in one sentence.

Key Terms

Activation	The fixed non-linear function applied elementwise to a layer's pre-activations; the source of all expressive power beyond affine maps.
Backpropagation	Reverse-mode automatic differentiation applied to a network's computational graph, computing all parameter gradients in one backward sweep.
Batch normalisation	Normalisation of pre-activations across a mini-batch, with learned rescaling; behaves differently in training and inference mode.
Dead unit	A rectified unit whose pre-activation is negative on every training example, and which therefore receives zero gradient permanently.
Dropout	Stochastic removal of units during training, equivalent to fitting an implicit ensemble of thinned sub-networks.
Early stopping	Halting training when a monitored validation metric stops improving, and restoring the best weights.
Embedding	A learned dense vector representation of a categorical level; equivalent to a dense layer on a one-hot encoding, implemented as a lookup.
Epoch	One complete pass through the training set, comprising $\lceil n/B \rceil$ mini-batch updates.
Error signal $\delta^{(l)}$	The derivative of the loss with respect to layer l 's pre-activations; the quantity propagated backwards.
Logit	The pre-activation of an output unit, before the sigmoid or softmax is applied; the numerically stable interface to loss functions.
Mini-batch	A random subset of training examples used to form an unbiased gradient estimate with variance inversely proportional to its size.
Multi-layer perceptron	A composition of dense layers with elementwise non-linear activations.
Out-of-vocabulary	A categorical level unseen during training; must be mapped to a reserved, trained index at inference.
Temperature scaling	Post-hoc calibration by dividing logits by a single learned scalar; preserves ranking exactly.
Vanishing gradient	Geometric decay of the error signal with depth, caused by repeated multiplication by factors smaller than one.

Further Reading

[Géron \(2022, Ch. 10–11\)](#) is the closest companion to this chapter and follows the same TensorFlow/Keras stack; Chapter 10 covers the MLP and the Keras APIs, Chapter 11 covers initialisation, normalisation, optimisers and regularisation in the same order used here. [Bishop and Bishop \(2024, Ch. 6–8\)](#) gives the probabilistic framing of the loss functions and a careful treatment of backpropagation and automatic differentiation. [Prince \(2024, Ch. 3–7\)](#) is the most accessible modern account of shallow and deep networks, loss surfaces and gradient-based fitting, and its figures pair well with Section [6.5](#).

On specific topics: [Glorot and Bengio \(2010\)](#) and [He et al. \(2015\)](#) for initialisation; [Ioffe and Szegedy \(2015\)](#) and [Ba et al. \(2016\)](#) for normalisation; [Srivastava et al. \(2014\)](#) for dropout; [Kingma and Ba \(2015\)](#) and [Loshchilov and Hutter \(2019\)](#) for adaptive optimisers; [Baydin et al. \(2018\)](#) for automatic differentiation as a field; [Guo and Berkhahn \(2016\)](#) for entity embeddings on tabular data; and [Grinsztajn et al. \(2022b\)](#) for the empirical case that gradient boosting remains competitive on tabular problems. For the governance material, [Huyen \(2022, Ch. 6–8\)](#) treats model evaluation and deployment concerns that Chapter [10](#) develops further.

Part III

**MLOps and Production
Systems**

Appendices

Capstone Project

Solutions to Exercises

This appendix collects worked solutions, one section per chapter. Solutions are deliberately fuller than a marking scheme requires: they model the style of argument, framing before formalism, business units before metrics, honesty caveats stated explicitly, that the assignments and the capstone are graded against.

Chapter 1: Introduction to Machine Learning

Solution to Exercise 1 (Framing). (a) Streaming fraud detection with a 300-alerts/day operations team.

Mitchell specification.

- *T*: score each transaction at initiation, before settlement, and emit an alert for those most likely to be fraudulent.
- *E*: historical transaction logs with fraud labels arriving with a delay of days to weeks, from chargebacks, customer complaints, and investigator dispositions; labels are incomplete (undetected fraud is labelled “legitimate”).
- *P*: fraud value captured (or recall) *at a fixed budget of 300 alerts per day* — operationally, precision@*k* with $k = 300/\text{day}$, weighted by transaction value.

Framing block. **Objective:** minimize net fraud losses subject to the investigation-capacity constraint and a false-positive customer-friction budget. **ML task:** binary classification (or anomaly scoring) on a *stream*; the drift speed of adversarial behaviour argues for online or frequently retrained batch learning (Table 1.4), evaluated prequentially. **Metric:** value-weighted recall at the 300-alert budget; secondary, time-to-detect for new fraud patterns. **Baseline:** the incumbent rules engine — its capture rate at the same alert volume is the number to beat. **Constraints:** per-transaction

scoring latency in the tens of milliseconds; adversarial drift; delayed and censored labels; rollback capability for bad online updates.

Two honesty points earn full marks: the label censoring (undetected fraud contaminates the negatives, so measured recall is optimistic), and the feedback loop — blocking transactions changes which frauds are ever observed, the streaming analogue of the selective-labels problem of [Section 1.4](#).

(b) Thirty-day readmission from clinical pathway data.

Mitchell specification.

- *T*: at discharge, estimate the probability that the patient is readmitted within 30 days.
- *E*: historical admission episodes — diagnoses, procedures, length of stay, vitals trajectories, prior utilization — joined to the observed readmission outcome.
- *P*: recall (or expected cost captured) at the capacity of the post-discharge intervention programme (follow-up calls, home visits); PR-AUC as the threshold-free summary, given a low base rate.

Framing block. **Objective:** allocate a fixed post-discharge intervention capacity to the patients whose readmission it is most likely to prevent. **ML task:** binary classification at the discharge-decision point; batch retraining is adequate (clinical drift is slow relative to fraud). **Metric:** recall at the intervention budget; per-segment calibration, since resource allocation across wards and demographics will be scrutinized. **Baseline:** the incumbent clinical risk score (e.g. a LACE-style index) or current clinician triage. **Constraints:** explainability to clinicians; data-protection compliance for health records; fairness across demographic groups; label subtleties — death within 30 days is a competing risk that must be handled explicitly, not silently coded as “no readmission”.

The deepest framing observation: the deployment intent is *interventional* (we act on high-risk patients), so post-deployment outcome data is shaped by the model’s own decisions — future retraining must account for the intervention, exactly the hidden feedback loop of [Section 1.4](#).

Solution to Exercise 2 (ML or not).

(a) Blocking transactions above a regulatory limit: rule. The requirement is a rule — crisp, complete, auditable, and legally fixed. ML could only approximate it, adding error where zero error is mandated and destroying auditability. Right-hand column of [Table 1.1](#) on every criterion.

(b) SIM-swap account takeover: ML, wrapped in rules. The signal is high-dimensional and behavioural (device changes, dormancy-then-drain patterns, atypical beneficiaries), and the adversary adapts, so any static rulebook rots within months — the left-hand column on every criterion.

The mature design keeps hard rules as guardrails (e.g. mandatory step-up authentication within N hours of a SIM change) with an ML risk score ranking the grey zone; the rules also generate the labelled outcomes that train the model, the “rules first, ML second” pattern of [Section 1.2.3](#).

(c) Routing support tickets by language: ML, but off the shelf. Hand-written rules (character ranges, keyword lists) are brittle across code-switching and short texts, so learning is justified; but language identification is a solved commodity problem, and a lightweight pretrained classifier meets the accuracy, latency, and cost bar. The build-or-not lesson: “use ML” does not imply “train your own” — the total cost-of-ownership argument of [Table 1.1](#) favours the smallest maintained artifact that clears the requirement.

Solution to Exercise 3 (ERM: mean and median as optimal constants).

Squared error. For a constant predictor c , the empirical risk is

$$J(c) = \frac{1}{n} \sum_{i=1}^n (y_i - c)^2, \quad J'(c) = -\frac{2}{n} \sum_{i=1}^n (y_i - c),$$

and setting $J'(c) = 0$ gives $c^* = \frac{1}{n} \sum_i y_i = \bar{y}$, the sample mean; $J''(c) = 2 > 0$ confirms the minimum.

Absolute error. $G(c) = \frac{1}{n} \sum_i |y_i - c|$ is convex and piecewise linear, so we argue by subgradient. For c not equal to any y_i ,

$$G'(c) = \frac{1}{n} \left(\#\{i : y_i < c\} - \#\{i : y_i > c\} \right),$$

which is negative while fewer than half the points lie below c and positive once more than half do. The minimum is therefore attained where the two counts balance: at the sample median (for even n , anywhere on the interval between the two middle order statistics — the minimizer need not be unique).

Connection to [Section 1.6.2](#). Minimizing RMSE trains the model toward the conditional *mean*, which is pulled by extreme outcomes; minimizing MAE trains it toward the conditional *median*, which is robust to them. Choosing between [Eq. \(1.4\)](#) is therefore choosing *which summary of the target distribution the model learns* — a business decision (how do errors actually cost?) dressed as a metric choice.

Solution to Exercise 4 (Cost-sensitive thresholds).

Two-way derivation. For an applicant with calibrated default probability $\hat{p}$, compare conditional expected costs:

$$\mathbb{E}[C \mid \text{approve}] = \hat{p} c_{\text{FN}}, \quad \mathbb{E}[C \mid \text{decline}] = (1 - \hat{p}) c_{\text{FP}}.$$

Decline exactly when $\hat{p} c_{\text{FN}} > (1 - \hat{p}) c_{\text{FP}}$; rearranging,

$$\hat{p} > \frac{c_{\text{FP}}}{c_{\text{FP}} + c_{\text{FN}}} = t^*,$$

which is Eq. (1.6). The derivation *uses* calibration: $\hat{p}$ must be interpretable as a probability for the expectations to be meaningful, which is why calibration is a precondition, not a nicety.

Three-way extension. Add the action *refer*, costing a review fee c_R and (idealizing) resulting in the correct decision, so it incurs no error cost. The three conditional expected costs are $\hat{p} c_{FN}$ (approve), $(1 - \hat{p}) c_{FP}$ (decline), and c_R (refer). Comparing pairwise:

- approve beats refer iff $\hat{p} c_{FN} < c_R$, i.e. $\hat{p} < t_L := c_R / c_{FN}$;
- decline beats refer iff $(1 - \hat{p}) c_{FP} < c_R$, i.e. $\hat{p} > t_U := 1 - c_R / c_{FP}$.

The optimal policy is therefore: *approve* if $\hat{p} < t_L$, *refer* if $t_L \leq \hat{p} \leq t_U$, *decline* if $\hat{p} > t_U$. The referral band is non-empty iff $t_L < t_U$, i.e.

$$c_R < \frac{c_{FN} c_{FP}}{c_{FN} + c_{FP}},$$

the harmonic combination of the two error costs; cheap review widens the band, expensive review collapses it to the single threshold t^* — which, one checks, always lies inside $[t_L, t_U]$ when the band exists. This is exactly the “ $\hat{p}$ near t^* routes to a human” edge of Fig. 1.9, now with the band derived rather than asserted. (A fuller model would let the human err too; the band then narrows in proportion to reviewer accuracy, a good extension discussion.)

Solution to Exercise 5 (Leakage audit).

(a) Number of repayment reminders sent: target leakage. Reminders exist only after origination and are triggered by delinquency — the feature is a downstream *consequence* of the label. At application time it is identically empty, so a model trained with it learns to read the outcome off its own paperwork and collapses in production (Definition 1.3).

(b) Median mobile-money balance over the 90 days before application: safe — conditional on discipline. The window $[t_0 - 90d, t_0)$ respects the as-of contract of Table 1.6. The residual risks to state for full marks: the feature must be computed from event data as it stood at t_0 (late-arriving corrections or restated ledgers re-introduce temporal leakage), and the training-time computation must share code with the serving-time computation or training/serving skew replaces leakage as the failure mode.

(c) Standard deviation of balance over the loan term: temporal leakage. The loan term lies strictly *after* the origination decision the model is meant to inform. The feature is computable in the historical warehouse (which is why it slips into training sets) but can never be computed at prediction time. Textbook violation of the as-of rule.

(d) Merchant-ID one-hot with random row-wise splits: group leakage. Repeat merchants appear on both sides of a random split, so the identifier

lets the model memorize *who* defaulted rather than learn *which behaviours* predict default; validation scores inflate and the model is useless for the new applicants it exists to score (whose IDs it has never seen). Remedies: split by merchant group *and* time; drop the raw identifier in favour of behavioural aggregates with proper as-of semantics; if entity-level history is genuinely predictive, encode it as backdated features (e.g. prior loans, prior repayment record as of t_0), which are legitimate.

Solution to Exercise 6 (Bias–variance: the vanishing cross-terms).

Setup as in Section 1.9.1: $y = g(x) + \varepsilon$ with $\mathbb{E}[\varepsilon] = 0$, $\text{Var}(\varepsilon) = \sigma^2$, ε independent of the training sample S ; $\hat{f}$ is trained on S ; $\bar{f}(x) = \mathbb{E}_S[\hat{f}(x)]$. Fix x and expand (all expectations over S and ε jointly):

First cross-term. Expanding $\mathbb{E}[(g + \varepsilon - \hat{f})^2] = \mathbb{E}[(g - \hat{f})^2] + 2\mathbb{E}[\varepsilon(g - \hat{f})] + \mathbb{E}[\varepsilon^2]$, the middle term factorizes because ε is independent of S and $\hat{f}$ is a function of S (and $g(x)$ is a constant for fixed x):

$$\mathbb{E}[\varepsilon(g - \hat{f})] = \mathbb{E}[\varepsilon] \mathbb{E}_S[g - \hat{f}] = 0 \cdot \mathbb{E}_S[g - \hat{f}] = 0.$$

Independence is used to factorize; the zero mean of ε kills the product. This leaves $\mathbb{E}[(g - \hat{f})^2] + \sigma^2$.

Second cross-term. Insert and subtract $\bar{f}$: $(g - \hat{f}) = (g - \bar{f}) + (\bar{f} - \hat{f})$, so

$$\mathbb{E}_S[(g - \hat{f})^2] = (g - \bar{f})^2 + 2(g - \bar{f}) \mathbb{E}_S[\bar{f} - \hat{f}] + \mathbb{E}_S[(\bar{f} - \hat{f})^2].$$

Here $(g - \bar{f})$ is a deterministic quantity (both terms are fixed numbers once x is fixed), so it exits the expectation; and by the *definition* of $\bar{f}$, $\mathbb{E}_S[\bar{f} - \hat{f}] = \bar{f} - \mathbb{E}_S[\hat{f}] = 0$. The cross-term vanishes, leaving $\text{Bias}^2 + \text{Variance} + \sigma^2$, which is Eq. (1.7). Summary of where each assumption bites: independence of ε and S kills the first cross-term; the definition of $\bar{f}$ as the S -average kills the second.

Solution to Exercise 7 (Splits, empirically).

A model answer on synthetic data with controlled drift (students' numbers on real data will differ in size, not in sign):

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(8401)
n, d = 30_000, 10
t = np.sort(rng.uniform(0, 1, n))          # "application date" in [0,1]
X = rng.normal(size=(n, d))
w = np.array([2.0, -1.5, 1.0, 0.5, 0, 0, 0, 0, 0, 0])
```

```

drift = 2.5 * t                                     # concept drifts over time
logits = X @ w + drift * X[:, 3] - 3.2
y = rng.binomial(1, 1 / (1 + np.exp(-logits)))

model = Pipeline([("s", StandardScaler()),
                  ("m", LogisticRegression(max_iter=2_000,
                                           class_weight="balanced"))])

# (a) Random shuffled split -- time ignored
Xa_tr, Xa_te, ya_tr, ya_te = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=8401)
pa = model.fit(Xa_tr, ya_tr).predict_proba(Xa_te)[: , 1]

# (b) Temporal split -- train on the past, test on the future
cut = int(0.75 * n)                                # rows already time-ordered
pb = (model.fit(X[:cut], y[:cut])
      .predict_proba(X[cut:]))[: , 1]

print(f"random split PR-AUC: {average_precision_score(ya_te, pa):.3f}")
print(f"temporal split PR-AUC: {average_precision_score(y[cut:], pb):.3f}")

```

Listing B.1: Random vs. temporal split under drift: the random split reports an optimistic number that no deployment can honour.

Direction and explanation. The random split scores higher. Under shuffling, training and test rows are interleaved in time, so the training set contains examples from the *same regime* as every test example — the model has effectively seen the future, and the drift term never has to be extrapolated. The temporal split forces genuine forward prediction into a drifted regime, which is exactly what deployment does (Fig. 1.6). **The honest number is the temporal one;** the gap between the two is not noise but a first, free estimate of how much drift will cost in production — worth reporting in the decision memo of Section 1.10.

Solution to Exercise 8 (Fairness).

Let F_A, F_B be the score CDFs of the calibrated model $\hat{p}$ in the two regions, with base rates $\pi_A = \mathbb{E}[\hat{p} \mid A] = 0.03$ and $\pi_B = \mathbb{E}[\hat{p} \mid B] = 0.09$ (calibration plus the tower property force the group mean score to equal the group base rate). Approval is $\hat{p} \leq t$ for a single global t .

Impossibility across thresholds. Demographic parity at *every* threshold would mean $F_A(t) = F_B(t)$ for all t , i.e. identical score distributions — hence identical means, contradicting $0.03 \neq 0.09$. So there is no global-threshold policy family satisfying parity uniformly.

The single-threshold case. At one fixed t , parity requires the accident $F_A(t) = F_B(t)$ at that exact point. But any calibrated model that actually uses the risk information distinguishing the regions places more probability mass at higher scores for region B (in the natural case, F_B is stochastically dominated by — i.e. sits below — F_A), giving $F_B(t) < F_A(t)$: region B's approval rate is strictly lower at every useful threshold. Even in contrived

cases where the CDFs cross at some t , the coincidence is knife-edge — it fails for neighbouring thresholds and evaporates at the first retrain — so it cannot ground a policy. The structural conclusion: with unequal base rates, a calibrated score plus a single global threshold *cannot* deliver demographic parity; achieving parity would require group-specific thresholds, which raises its own legal and ethical questions, or abandoning calibration — the conflict promised in [Section 1.11](#).

Proposal to the regulator. A defensible position for the running case: retain per-group *calibration* (an uncalibrated score corrupts every downstream decision through [Eq. \(1.6\)](#)) and adopt *equal opportunity* — equal true-positive rates among repayers — as the operative criterion, i.e. good borrowers in both regions face the same approval odds; then justify any residual approval-rate gap by the base-rate difference while committing to close it upstream (product design, data enrichment for the thin-file region). **Post-deployment monitoring:** per-region approval rate, TPR/FPR among matured labels, calibration curves, and their drift over time — the fairness axis of the Week-13 monitors, reported segment-by-segment as [Section 1.11](#) prescribes.

[Solution to Exercise 9 \(Hybrid architecture\).](#)

A full-marks answer reproduces the topology of [Fig. 1.9](#) with the component classification made explicit:

- **Classical ML (the decision core).** The gradient-boosted or logistic scoring model with its leakage-safe feature pipeline ([Listing 1.5](#)), served behind the versioned API of [Listing 1.7](#). It alone emits $\hat{p}$; it is trained, calibrated, validated, and monitored under the discipline of Chapters 1–4 and 11–13.
- **LLM components (the language edges).** (i) A RAG service over lending policy, KYC regulation, and product documents, answering underwriter questions with citations; (ii) an extraction model converting unstructured inputs (statements, free-text loan purpose, chat) into *typed features* that enter the classical pipeline — validated like any other upstream data source; (iii) an agent orchestrating the workflow: gather data, call the scoring API, draft the reason-coded communication.
- **Human review.** Applications with $\hat{p} \in [t_L, t_U]$ — the referral band derived in [Solution 4](#) — route to a credit officer, with the agent’s brief attached.

Where calibration must be preserved end-to-end — the marking crux: the probability $\hat{p}$ must flow *numerically unaltered* from the scorer through the API to the thresholding policy and into any human-facing brief. The

agent may *explain* the score; it must never *re-estimate*, round, re-rank, or paraphrase it into a different number, and no LLM output may feed back into the decision path — the moment a stochastic text generator sits between the calibrated score and the threshold of [Eq. \(1.6\)](#), the optimality of the policy, the audit trail, and the fairness monitoring of Solution 8 are all void. Diagrammatically: every arrow into the approve/refer/decline node originates at the classical scorer; LLM arrows terminate at humans or at the scorer's *inputs*, never at its output.

Chapter 2: Data Preparation and Feature Engineering

Solution to Exercise 2.1. Consider the electronic medical records of a network of Kenyan outpatient clinics.

MCAR. A batch of laboratory results is lost when the connection between the lab analyser and the records system fails for an afternoon. Which patients' rows are affected is determined by the timing of the outage, not by the patients or their values: $P(M \mid X) = P(M)$ ([Definition 2.1](#)). Mean imputation of the missing lab values yields an *unbiased* estimate of the population mean (the missing rows are a random sample), although it understates the variance (Exercise 2.3).

MAR. Blood-pressure readings are missing far more often at rural dispensaries that lack a working monitor than at urban clinics. Conditional on the *observed* facility-type column, missingness carries no further information about the reading itself ([Definition 2.2](#)). Unconditional mean imputation is *biased* whenever blood pressure differs systematically by facility type (urban and rural patient populations differ): the imputed mean is dominated by urban readings. Imputation that conditions on facility type — a group-wise mean, or the model-based imputers of the chapter — removes the bias.

MNAR. HIV-test results are missing because some patients decline testing, and the propensity to decline is associated with the patient's own (unobserved) status and risk perception: $P(M \mid X)$ depends on X_{mis} ([Definition 2.3](#)). Mean imputation is *biased*, and no imputation scheme built from the observed data alone removes the bias; per [Remark 2.1](#), the honest treatment records the refusal itself as an indicator feature and interprets downstream estimates with the selection mechanism in mind.

In summary, mean imputation is unbiased for the mean only under MCAR; under MAR it requires conditioning on the drivers of missingness; under MNAR it is biased regardless.

Solution to Exercise 2.2. Let $R_i \in \{0, 1\}$ indicate that row i is a complete case, and let X_i be its (complete-data) value. MCAR states $P(R \mid X) = P(R)$,

i.e. $R \perp X$. The density of X among complete cases is, by Bayes' rule,

$$f(x | R = 1) = \frac{P(R = 1 | X = x) f(x)}{P(R = 1)} = \frac{P(R = 1) f(x)}{P(R = 1)} = f(x),$$

where the middle equality uses independence. The complete cases are therefore distributed identically to the full population — a random subsample. Consequently, with $n_{cc} = \sum_i R_i$ the (random) number of complete cases,

$$\mathbb{E} \left[\frac{1}{n_{cc}} \sum_{i: R_i=1} X_i \mid n_{cc} > 0 \right] = \mathbb{E}[X],$$

by taking the expectation first conditionally on the pattern $(R_1, \dots, R_n)$ — under which each retained X_i has mean $\mathbb{E}[X]$ by independence — and then over the pattern. The complete-case mean is unbiased; the only cost of MCAR missingness is the loss of efficiency from the reduced sample size. $\square$

Solution to Exercise 2.3. With point-estimate imputation (`sample_posterior=False`), every imputed entry is set to the conditional-mean prediction $\hat{\mathbb{E}}[X_{\text{mis}} | X_{\text{obs}}]$ of the final regression sweep of Eq. (2.2). Decompose the true variance by the law of total variance:

$$\text{Var}(X) = \underbrace{\mathbb{E}[\text{Var}(X | X_{\text{obs}})]}_{\text{residual spread}} + \underbrace{\text{Var}(\mathbb{E}[X | X_{\text{obs}}])}_{\text{explained spread}}.$$

The imputed values realise only the second term: they sit exactly on the conditional-mean surface, with zero residual spread. The completed column's sample variance therefore converges to $\text{Var}(\mathbb{E}[X | X_{\text{obs}}])$, understating $\text{Var}(X)$ by the entire residual component $\mathbb{E}[\text{Var}(X | X_{\text{obs}})]$, and the understatement grows with the missing fraction.

Downstream, any procedure that treats the completed data as if observed — ordinary least squares, a t -test, a confidence interval for a feature mean — inherits standard errors that are too small: intervals are too narrow and p -values anti-conservative, because the analysis pretends the imputed entries carry the same information as real measurements. The remedies are those of proper multiple imputation: draw fills from the posterior (`sample_posterior=True`), create M completed datasets, and combine estimates with Rubin's rules, which add the between-imputation variance the point-estimate shortcut discards.

Solution to Exercise 2.4. Two mechanisms, both consistent with the chapter, explain a *drop* in AUC after trimming $|z| > 3$ rows.

(a) *The outliers were the signal.* In credit risk the extreme tail of behaviour is disproportionately informative about default: erratic whale deposits, balance-inflation spikes, abnormal withdrawal bursts (see the pitfall in

Section 2.3). Deleting $|z| > 3$ rows removes a non-random, minority-class-enriched slice of the training data — defaulters exhibit extreme behaviour more often than repayers — so the model loses precisely the discriminative region of the feature space, and its ranking quality on an untrimmed evaluation set falls.

(b) *The analyst created a training–serving mismatch.* Rows were removed from the *training* data, but the validation/test data (and, eventually, production traffic) still contain extreme rows. The model is now evaluated on a region of the input space it was never fitted on — a self-inflicted violation of the distribution-overlap requirement of **Section 2.6**. A subsidiary aggravating factor: on heavy-tailed transaction data the Gaussian $|z| > 3$ rule flags a large fraction of perfectly legitimate rows (the z-score’s μ and σ are themselves inflated by the tail), so the deletion is both extensive and systematically targeted at high-activity customers.

Solution to Exercise 2.5. From Eq. (2.7), the maximum maps to

$$x'_{\max} = \frac{x_{\max} - x_{\min}}{x_{\max} - x_{\min}} = 1$$

identically — the scaled value of the largest point is 1 whatever its magnitude, which is exactly why a single extreme outlier cannot be “detected” in the scaled data: it is normalised into the boundary.

Let m be the median. Its scaled value is $m' = (m - x_{\min}) / (x_{\max} - x_{\min})$. Replace the maximum by $10 x_{\max}$ (all other order statistics unchanged); the new scaled median is

$$m'' = \frac{m - x_{\min}}{10 x_{\max} - x_{\min}}, \quad \frac{m''}{m'} = \frac{x_{\max} - x_{\min}}{10 x_{\max} - x_{\min}}.$$

For $x_{\min} = 0$ this ratio is exactly $1/10$: the median’s scaled value collapses by a factor of ten, and with it the entire bulk of the distribution is crushed toward 0 while the outlier alone occupies the top of the unit interval. This is the fragility that motivates the robust scaler of **Eq. (2.8)**.

Solution to Exercise 2.6. Within level c , model each target as conditionally Bernoulli, $y_i \mid \theta_c \sim \text{Bernoulli}(\theta_c)$, and place the conjugate prior

$$\theta_c \sim \text{Beta}(\alpha, \beta), \quad \alpha = m \bar{y}, \quad \beta = m(1 - \bar{y}),$$

so that the prior mean is $\alpha / (\alpha + \beta) = \bar{y}$ (the global rate) and the prior strength — the number of pseudo-observations — is $\alpha + \beta = m$. Observing n_c rows of level c with $s_c = \sum_i y_i = n_c \bar{y}_c$ successes, conjugacy gives the posterior $\theta_c \mid \text{data} \sim \text{Beta}(\alpha + s_c, \beta + n_c - s_c)$, whose mean is

$$\mathbb{E}[\theta_c \mid \text{data}] = \frac{\alpha + s_c}{\alpha + \beta + n_c} = \frac{m \bar{y} + n_c \bar{y}_c}{m + n_c} = \text{TE}(c),$$

which is [Eq. \(2.11\)](#) exactly. The limits confirm the shrinkage behaviour: as $n_c \rightarrow 0$, $\text{TE}(c) \rightarrow \bar{y}$ (an unseen or rare level is pulled entirely to the prior), and as $n_c \rightarrow \infty$, $\text{TE}(c) \rightarrow \bar{y}_c$ (abundant evidence overwhelms the prior). The smoothing parameter m is thus interpretable as the number of global-prior pseudo-counts each level must “outweigh” before its own history dominates — and setting it too small is precisely what lets a level seen once memorise its own label ([Section 2.5.3](#)). $\square$

Solution to Exercise 2.7. *Mechanism.* Individually, a start date and an end date are weak predictors of churn. Jointly they encode the label almost perfectly at extraction time: an employee who has churned has a populated `contract_end_date` lying before the snapshot date, whereas a current employee has a null, sentinel, or future-dated value. Any model able to combine the two features (or a feature and the implicit snapshot date) reconstructs “has this contract already terminated?” — which is the churn label, or a near-deterministic proxy for it. The end date is a *post-outcome field*, written by the HR system as a consequence of the very event being predicted: cause 6 of [Section 2.7.2](#), hiding behind two innocuous-looking columns that pass a univariate correlation screen.

A screen for pairwise leakage. Extend the univariate screen of [Section 2.7](#) to constructed pairs: (i) for every pair of same-typed columns (two dates, two amounts), automatically generate their difference (and, for amounts, their ratio) and rank these derived features by $|\rho(\cdot, y)|$ or mutual information with the target; (ii) as a model-based variant, fit a depth-2 decision tree on each feature pair alone and flag any pair whose standalone AUC approaches the full model’s. Either variant catches start/end-date leakage immediately (the difference is tenure-to-termination; the tiny tree learns “end date < snapshot”). The screen complements, but does not replace, the lineage question — *which system writes this field, and when?* — that identifies post-outcome fields even when their statistical footprint is subtle.

Solution to Exercise 2.8. Assume the hash of [Eq. \(2.10\)](#) assigns each of the K levels independently and uniformly to one of d buckets. Two given levels collide with probability $1/d$, and there are $\binom{K}{2}$ pairs, so by linearity of expectation the expected number of colliding pairs is

$$\mathbb{E}[\text{\#collisions}] = \binom{K}{2} \frac{1}{d} = \frac{K(K-1)}{2d}.$$

For $K = 10^4$ and $d = 256$:

$$\frac{10^4 \times (10^4 - 1)}{2 \times 256} = \frac{99\,990\,000}{512} \approx 1.95 \times 10^5$$

colliding pairs — with an average bucket load of $K/d \approx 39$ levels, so *every* bucket is shared. This is not necessarily fatal (tree models tolerate the induced noise, and the load is spread evenly), but it quantifies the trade-off:

at this cardinality, $d = 256$ buys a fixed 256-column footprint at the price of heavy aliasing, and raising d to 2^{12} – 2^{14} reduces the expected pair collisions by one to two orders of magnitude while remaining far cheaper than a 10^4 -column one-hot encoding.

Solution to Exercise 2.9. *Leakage mechanism.* Support-line contact is not exogenous to repayment trouble. Two concrete routes: (i) borrowers who are already struggling call to ask about extensions, penalties, or restructuring *after* the loan being scored has been disbursed — so if the 7-day window is computed relative to label-observation time (or to “now” in the training warehouse) rather than to the scoring timestamp t , the feature ingests post- t information in direct violation of the as-of rule (Eq. (2.1)); (ii) worse, some inbound calls are *returns of collections outreach* — the collections team calls delinquent borrowers, who call back — making the count a laundered copy of the delinquency flag, exactly the post-outcome pattern of `days_since_collections_call` in Listing 2.11 (cause 6 of Section 2.7.2).

Redesign. Keep the legitimate signal (a customer’s baseline propensity to need support may genuinely correlate with risk) by: fixing the window to end strictly before the scoring timestamp, $[t - 7d, t)$, enforced by the feature platform’s point-in-time semantics (Section 2.8); excluding call types causally downstream of the credit process (collections callbacks, repayment-difficulty queries about an *active* loan), which requires the call-category field and its write-time lineage; and validating the rebuilt feature with the correlation screen — its association with the target should fall from “implausible” to “modest” if the leak, and not the baseline behaviour, was carrying the load.

Solution to Exercise 2.10. A five-sentence rebuttal. First, native NaN routing (Chen and Guestrin, 2016) solves only the mechanical problem of fitting a tree through incomplete rows; it does not answer the diagnostic questions — *why* is the value missing, and is the missingness itself predictive? — that Lab 2’s MCAR/MAR/MNAR audit exists to settle. Second, under MNAR the explicit indicator of Remark 2.1 is a first-class, interpretable feature whose effect can be measured, reviewed for fairness, and explained to a credit officer, whereas a learned default branch direction is an implicit, per-split artefact that cannot. Third, the pipeline must remain model-agnostic: the moment we benchmark against a regulatory logistic scorecard (Eq. (2.13)) or a neural baseline (??), models without native handling need the imputation stage anyway, and comparing models across different missing-data treatments confounds the comparison. Fourth, training-serving consistency (Section 2.8) requires that whatever missing-value semantics the model learned be reproduced exactly in the online path — a fitted, serialised imputer inside the Pipeline of Listing 2.12 is a deployable contract, while “the library handles it” is not. Fifth, most of Lab 2’s effort — parsing, validity rules, entity resolution, leakage hunting — is not imputa-

tion at all, and no amount of native NaN support dedupes a customer with three SIM cards.

Chapter 3: Classical Machine Learning and Rigorous Evaluation

Solution to Exercise 3.1. A defensible committee-minute sentence: “*Holding every other feature in the model fixed, applicants with late-night transaction activity have $e^{0.62} \approx 1.86$ times the odds of default of otherwise-identical applicants without it.*” The load-bearing words are *odds* and *holding every other feature fixed*.

The important caveat: the coefficient measures a *conditional association*, not a *causal effect*. It is a partial effect within this particular model and this particular preprocessing pipeline (Chapter 2): add, remove, or re-encode correlated features and the coefficient — and hence the “effect” of late-night activity — can change substantially. It licenses no intervention claim (“discouraging late-night transactions would reduce defaults” does not follow). A secondary caveat worth a footnote in the minute: an odds ratio is not a risk ratio. At the portfolio’s 3% base rate the two are numerically close ($OR \approx RR$ when the outcome is rare), but the identification should still be avoided in a document regulators may read.

Solution to Exercise 3.2. *Derivative of the sigmoid.* With $\sigma(z) = (1 + e^{-z})^{-1}$,

$$\frac{\partial \sigma}{\partial z} = \frac{e^{-z}}{(1 + e^{-z})^2} = \underbrace{\frac{1}{1 + e^{-z}}}_{\sigma(z)} \cdot \underbrace{\frac{e^{-z}}{1 + e^{-z}}}_{1 - \sigma(z)} = \sigma(z)(1 - \sigma(z)).$$

Gradient of the log loss. Write the per-example loss with $\hat{p} = \sigma(z)$, $z = \theta^\top \mathbf{x}$:

$$\ell(\theta) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p}).$$

By the chain rule,

$$\frac{\partial \ell}{\partial z} = -\frac{y}{\hat{p}} \hat{p}(1 - \hat{p}) + \frac{1 - y}{1 - \hat{p}} \hat{p}(1 - \hat{p}) = -y(1 - \hat{p}) + (1 - y) \hat{p} = \hat{p} - y,$$

and since $\partial z / \partial \theta = \mathbf{x}$, $\nabla_{\theta} \ell = (\hat{p} - y) \mathbf{x}$. Averaging over the m training examples gives Eq. (3.4): $\nabla_{\theta} J = \frac{1}{m} \sum_i (\hat{p}_i - y_i) \mathbf{x}_i$. The remarkable cancellation of the $\hat{p}(1 - \hat{p})$ factor is a signature of pairing the sigmoid with its *matched* loss (cross-entropy); the same cancellation reappears at the output layer of every neural classifier in ??.

Solution to Exercise 3.3. From Eq. (3.11), $F_{\beta} = (1 + \beta^2)PR / (\beta^2 P + R)$.

Limit $\beta \rightarrow 0$: the expression tends to $PR/R = P$ directly.

Limit $\beta \rightarrow \infty$: divide numerator and denominator by β^2 :

$$F_\beta = \frac{(1 + \beta^{-2}) PR}{P + \beta^{-2} R} \xrightarrow{\beta \rightarrow \infty} \frac{PR}{P} = R.$$

So β interpolates between a pure-precision and a pure-recall score, which is exactly why it should be set from the cost structure.

Numerical part. With $P = 0.40$, $R = 0.75$, $\beta = 2$:

$$F_2 = \frac{(1 + 4) \cdot 0.40 \cdot 0.75}{4 \cdot 0.40 + 0.75} = \frac{1.50}{2.35} \approx 0.638,$$

noticeably above $F_1 = 2 \cdot 0.3/1.15 \approx 0.522$ because F_2 rewards this classifier's recall-heavy profile.

Solution to Exercise 3.4. Fix an instance with calibrated probability p of being positive. The two actions have expected costs

$$\mathbb{E}[\text{cost} \mid \text{flag}] = p c_{TP} + (1 - p) c_{FP}, \quad \mathbb{E}[\text{cost} \mid \text{pass}] = p c_{FN} + (1 - p) c_{TN}.$$

Flagging is optimal iff $p c_{TP} + (1 - p) c_{FP} \leq p c_{FN} + (1 - p) c_{TN}$, i.e.

$$(1 - p) (c_{FP} - c_{TN}) \leq p (c_{FN} - c_{TP}).$$

Under the “reasonableness” conditions $c_{FP} > c_{TN}$ and $c_{FN} > c_{TP}$ (each error costs more than the corresponding correct decision — otherwise one action dominates and no threshold exists), both bracketed terms are positive and we may divide:

$$t^* = \frac{c_{FP} - c_{TN}}{(c_{FP} - c_{TN}) + (c_{FN} - c_{TP})}.$$

Setting $c_{TP} = c_{TN} = 0$ recovers Eq. (3.13). Only cost *differences* within each true class matter — adding a constant to a whole row of the cost matrix leaves t^* unchanged. Concretely: if every flag triggers a manual review costing KES 200 regardless of outcome ($c_{TP} = c_{FP, \text{new}} - c_{FP, \text{old}} = 200$ added to the flag column), then with the case-study numbers $t^* = (1000)/(1000 + 9800) \approx 0.093$: review costs push the threshold up, and the scorer should flag slightly less eagerly.

Solution to Exercise 3.5. Let the positive scores be $s_1^+, \dots, s_{m_+}^+$ and the negative scores $s_1^-, \dots, s_{m_-}^-$, and assume first that all scores are distinct. Sort all $m_+ + m_-$ instances by decreasing score and trace the empirical ROC curve from $(0, 0)$: each positive encountered moves the curve *up* by $1/m_+$, each negative moves it *right* by $1/m_-$.

Consider the horizontal step contributed by a particular negative j . Its height above the FPR axis at that moment is

$$(\text{number of positives already passed})/m_+ = \#\{i : s_i^+ > s_j^-\}/m_+.$$

The area of that step is therefore $\#\{i : s_i^+ > s_j^-\} / (m_+ m_-)$. Summing over all negatives,

$$\text{AUC} = \frac{1}{m_+ m_-} \sum_{j=1}^{m_-} \sum_{i=1}^{m_+} \mathbb{1}[s_i^+ > s_j^-] = \frac{U}{m_+ m_-},$$

where $U = \sum_{i,j} \mathbb{1}[s_i^+ > s_j^-]$ is precisely the Mann–Whitney U statistic of the positive-class scores. Dividing by the number of pairs turns the count into the empirical probability $P(s^+ > s^-)$ for a uniformly random (positive, negative) pair, which is [Proposition 3.2](#).

Ties. If k_+ positives and k_- negatives share a score, the ROC convention traces a straight diagonal segment through the tied block, contributing area equal to what the pairs would contribute at $\frac{1}{2}$ each; correspondingly U is defined with $\frac{1}{2}$ per tied pair. This yields the tie-corrected statement $\text{AUC} = P(s^+ > s^-) + \frac{1}{2}P(s^+ = s^-)$ of [Eq. \(3.15\)](#).

Solution to Exercise 3.6. Suppose scores are independent of labels. Fix any threshold t and let $A = \{\mathbf{x} : s(\mathbf{x}) \geq t\}$ be the alert set. By independence, the label distribution *inside* A equals the marginal label distribution, so

$$\mathbb{E}[\text{Precision}(t)] = P(y = 1 \mid \mathbf{x} \in A) = \pi,$$

the prevalence — at *every* threshold, hence at every achievable recall level. The PR “curve” of a label-independent scorer is therefore the horizontal line at height π , and its area (average precision) is

$$\text{AP} = \int_0^1 \pi d(\text{recall}) = \pi.$$

On the case-study portfolio $\pi \approx 0.03$, so chance PR–AUC ≈ 0.03 . Reporting “PR–AUC = 0.31” without the baseline invites two opposite misreadings: against an implicit 0.5 anchor (borrowed from ROC intuition) it looks poor, whereas against the true chance level it is a tenfold lift. The convention of [Section 3.4.3](#) — always state PR–AUC *together with* the prevalence — exists to force the correct reading.

Solution to Exercise 3.7. From [Eq. \(3.17\)](#), $\text{se} = \sqrt{r(1-r)/n_+} \leq 0.02$ requires

$$n_+ \geq \frac{r(1-r)}{\text{se}^2} = \frac{0.7 \times 0.3}{0.02^2} = \frac{0.21}{0.0004} = 525 \text{ positives.}$$

At 3% prevalence that means $525/0.03 = 17,500$ validation rows — for the validation split *alone*, before training data is even counted.

The implication is sobering and is the entire motivation of [Section 3.6](#): on a small credit portfolio (say 10,000 labelled loans in total, ~ 300 defaulters), *no single split can estimate recall to within two points*. The honest options

are exactly the chapter’s toolkit: aggregate information across stratified folds (Definition 3.7), attach bootstrap intervals to whatever held-out estimate is reported (Section 3.6.4), and refuse to interpret differences smaller than the resulting uncertainty band.

Solution to Exercise 3.8. With $n_{01} = 63$ (only model B correct) and $n_{10} = 41$ (only model A correct), Eq. (3.19) gives

$$\chi^2 = \frac{(|63 - 41| - 1)^2}{63 + 41} = \frac{21^2}{104} = \frac{441}{104} \approx 4.24.$$

Since $4.24 > \chi_{1,0.95}^2 = 3.84$, we reject at the 5% level the null hypothesis that the two models have equal error rates: model B’s advantage on the discordant cases is unlikely to be chance ($p \approx 0.04$).

The 9,800 concordant cases are irrelevant because they carry no information about the *difference* between the models: whether both are right or both are wrong on a case, that case shifts both models’ accuracies by the same amount. McNemar’s test conditions on the set of disagreements and asks only whether the 63:41 split is compatible with a fair coin — which is also why the test remains valid on heavily imbalanced test sets where accuracy itself is uninformative (Section 3.3.1).

Solution to Exercise 3.9. Applying Eq. (3.20) coordinate-wise with $\lambda = 0.4$:

$$(1, 0, 0) + 0.4 [(0, 1, 0) - (1, 0, 0)] = (1, 0, 0) + 0.4 (-1, 1, 0) = (0.6, 0.4, 0).$$

The result is invalid as a data point because a one-hot block encodes a *categorical* fact — the applicant’s region — and $(0.6, 0.4, 0)$ asserts membership of 60% in one region and 40% in another. No real applicant generates such a row; it violates the encoder’s sum-to-one, single-one structure, and any coefficient interpretation (Example 3.1) built on “the region-A indicator” silently breaks.

SMOTE-NC’s repair. SMOTE-NC interpolates only the *numeric* coordinates via Eq. (3.20), and sets each *categorical* feature of the synthetic point to the majority value of that feature among the k minority-class nearest neighbours (with the categorical features also entering the distance computation through a fixed penalty term). The synthetic row is then a legal row: valid one-hot blocks, interpolated numerics.

Residual risk. The synthetic point marries numeric values interpolated between two (possibly different-region) borrowers with a categorical value voted from the neighbourhood, so it can realise *combinations* of features that never co-occur in reality — e.g. transaction magnitudes typical of Nairobi CBD attached to a rural-county label. The class-conditional joint distribution is thereby distorted, and, as Section 3.7.2 already warned for plain

SMOTE, the altered class prior means the resulting scores are not calibrated: recalibrate (Section 3.5) before applying Proposition 3.1.

Solution to Exercise 3.10. Let $w = c_{FN}/c_{FP}$ and suppose training re-weights each positive example by w (equivalently, imagine each positive duplicated w times). For a scorer that is calibrated *on the re-weighted distribution*, the probability it reports is the re-weighted posterior. If p denotes the true (original-distribution) posterior $P(y = 1 \mid \mathbf{x})$, re-weighting the positive class by w transforms it to

$$p_w = \frac{w p}{w p + (1 - p)},$$

since the class-conditional densities are untouched and only the effective prior odds are multiplied by w . Thresholding the weighted model at 0.5 flags iff

$$p_w \geq \frac{1}{2} \iff w p \geq 1 - p \iff p \geq \frac{1}{1 + w} = \frac{c_{FP}}{c_{FP} + c_{FN}} = t^*,$$

which is exactly the unweighted model thresholded at t^* of Eq. (3.13). This is Elkan (2001)'s theorem: for Bayes-optimal (calibrated) scorers, cost-proportionate re-weighting and threshold moving induce the *same* decision rule.

Why try threshold moving first, then? Because the equivalence is exact only in the idealised limit, and every practical consideration favours the decision-level route. (i) It is *free and reversible*: no retraining, and the cost matrix can be revised — or differ across products — while one model serves them all, each business line simply reading a different operating point off the same score. (ii) It *preserves calibration*: re-weighting deliberately distorts the class prior, so the weighted model's outputs are no longer probabilities on production traffic and must be recalibrated before anyone prices risk with them. (iii) With finite samples and misspecified models the two routes genuinely diverge, and re-weighting can degrade the ranking it was meant to help. (iv) It is *auditable*: “we score with model M and decline above 7.4% calibrated risk, derived from this cost matrix” is a one-line policy a regulator can inspect. Escalate to weighting or resampling only when the evidence says the *ranking itself* — not the decision rule — is the weak link (Section 3.7.1).

Chapter 4: Decision Trees, Ensembles and Hyperparameter Tuning

Solution to Exercise 4.1. *Independent voters.* With $p = 0.55$, the number of correct voters is $K \sim \text{Binomial}(M, 0.55)$ and the majority is correct when

$K > M/2$. The normal approximation $K \approx \mathcal{N}(Mp, Mp(1-p))$ is justified for these M by the usual rule $Mp(1-p) \gg 1$ (it equals 2.7, 25.0 and 247.7 respectively; for $M = 11$ the approximation is rough and the exact sum is preferable, but it errs by under two points here). Writing $z = (M/2 - Mp) / \sqrt{Mp(1-p)}$ and $P_{\text{maj}} \approx 1 - \Phi(z) = \Phi(-z)$:

M	$-z$	P_{maj} (normal)	P_{maj} (exact)
11	0.332	0.630	0.633
101	1.005	0.843	0.846
1001	3.164	0.9992	0.9992

The committee converges on certainty, exactly as [Proposition 4.1](#) promises.

Correlated blocks. If the M voters consist of ten blocks of $M/10$ perfectly correlated clones, every clone in a block casts the same vote, so the committee behaves as *ten* independent voters, whatever M is. For $M = 1001$ (≈ 100 clones per block) the relevant computation is the $M = 10$ (even; take $M = 11$ as the odd comparator) row, not the $M = 1001$ row: $P_{\text{maj}} \approx 0.63$, and no increase of M at fixed block structure improves it. The *effective committee size* is the number of independent error patterns, not the number of members. This is [Eq. \(4.5\)](#) in disguise — within-block correlation $\rho = 1$ contributes pure floor — and it is precisely the situation of a forest whose trees share their root split: trees that agree on the first partition of the data are partial clones, adding `n_estimators` buys little, and the productive intervention is decorrelation (`max_features`), not more trees.

Solution to Exercise 4.2. Let $\bar{f} = \sum_m c_m f_m$ with $\sum_m c_m = 1$, $c_m \geq 0$, $\text{Var}[f_m] = \sigma^2$ and pairwise correlation ρ . Then

$$\begin{aligned} \text{Var}[\bar{f}] &= \sum_m c_m^2 \sigma^2 + \sum_{m \neq m'} c_m c_{m'} \rho \sigma^2 \\ &= \sigma^2 \left[(1 - \rho) \sum_m c_m^2 + \rho \left(\sum_m c_m \right)^2 \right] = \sigma^2 \left[(1 - \rho) \sum_m c_m^2 + \rho \right], \end{aligned}$$

using $\sum_{m \neq m'} c_m c_{m'} = (\sum_m c_m)^2 - \sum_m c_m^2 = 1 - \sum_m c_m^2$. Since $\rho \leq 1$, minimising the variance means minimising $\sum_m c_m^2$ subject to $\sum_m c_m = 1$. By Cauchy–Schwarz (or a Lagrange multiplier), $\sum_m c_m^2 \geq \frac{1}{M} (\sum_m c_m)^2 = \frac{1}{M}$, with equality iff $c_m = 1/M$ for all m : *uniform weights are optimal*, and they recover [Proposition 4.2](#).

The optimality is conditional on the exchangeability assumptions. If the members have *unequal* variances — equivalently, unequal skill, as in [Listing 4.2](#) where a tuned LightGBM is simply a better estimator than the logistic baseline — the same Lagrangian calculation with member variances σ_m^2 (and $\rho = 0$ for clarity) gives optimal weights $c_m \propto 1/\sigma_m^2$: precision weighting. Soft voting with unequal weights is the practical, integer-granularity version of this, and stacking ([Section 4.8](#)) is its data-driven completion — the meta-learner *estimates* the weights instead of assuming exchangeability.

Solution to Exercise 4.3. *The limit without logarithms.* From $1 - x \leq e^{-x}$ with $x = \frac{1}{n}$,

$$\left(1 - \frac{1}{n}\right)^n \leq e^{-1}.$$

For the matching lower bound, apply $1 - x \geq e^{-x/(1-x)}$ (valid for $0 \leq x < 1$, and itself a rearrangement of $1 + u \leq e^u$ with $u = x/(1-x)$) to get $\left(1 - \frac{1}{n}\right)^n \geq e^{-n \cdot \frac{1/n}{1-1/n}} = e^{-n/(n-1)} \rightarrow e^{-1}$. The two bounds pinch the sequence to e^{-1} . For $n = 20$, $\left(\frac{19}{20}\right)^{20} = 0.3585$, already within 2.5% of $e^{-1} \approx 0.3679$ — the asymptotics arrive early, which is why “37% out of bag” is a safe rule of thumb even for modest samples.

Why an OOB-chosen threshold misbehaves. (i) *Statistical:* each OOB prediction $\bar{f}_{-i}^{\text{oob}}(x_i)$ in Eq. (4.6) averages only the $\approx 0.37M$ trees that missed row i , whereas the deployed model averages all M . A mean of $0.37M$ trees is a noisier, differently calibrated score than a mean of M trees, so the probability distribution on which the OOB threshold was optimised is not the distribution the production model emits; with the asymmetric costs of the case study (KES 10,000 vs KES 800), even a small calibration shift moves the cost-optimal cut-off materially. (ii) *Temporal:* the OOB estimate randomises over rows, so its implicit population is the historical mixture of the training window. The threshold’s economics depend on the *future* default base rate and score distribution, which drift; the temporal validation window of Chapter 3 is the only estimate that rehearses the deployment condition. In short, OOB may rank models acceptably, but the decision threshold should be set — and re-set, per Chapter 11 — on out-of-time data.

Solution to Exercise 4.4. After stage m , the new weight mass on the rows that f_m got wrong is

$$W_{\text{wrong}}^{(m+1)} = \sum_{i: f_m(x_i) \neq y_i} w_i^{(m)} e^{+\alpha_m} = e^{\alpha_m} \varepsilon_m W^{(m)}, \quad W_{\text{corr}}^{(m+1)} = e^{-\alpha_m} (1 - \varepsilon_m) W^{(m)},$$

where $W^{(m)} = \sum_i w_i^{(m)}$. Substituting $e^{\alpha_m} = \sqrt{(1 - \varepsilon_m)/\varepsilon_m}$ from Proposition 4.4,

$$W_{\text{wrong}}^{(m+1)} = \sqrt{\frac{1 - \varepsilon_m}{\varepsilon_m}} \varepsilon_m W^{(m)} = \sqrt{\varepsilon_m (1 - \varepsilon_m)} W^{(m)} = W_{\text{corr}}^{(m+1)}.$$

The two masses are equal, so the weighted error of f_m on the updated weights is exactly $\frac{1}{2}$: each learner is rendered useless against the distribution it leaves behind. The implication is that AdaBoost can never profitably select the same hypothesis twice in a row — the re-weighting is an *orthogonalisation* step that forces every stage to contribute something the previous stage did not, which is the sequential counterpart of the decorrelation that bagging obtains by randomisation. (Hypotheses can recur later, once the weight distribution has moved on.)

Solution to Exercise 4.5. (a) *Binary log-loss.* With $y \in \{0, 1\}$, logits F , and $\sigma(F) = (1 + e^{-F})^{-1}$,

$$L(y, F) = -[y \ln \sigma(F) + (1 - y) \ln(1 - \sigma(F))].$$

Using $\sigma'(F) = \sigma(F)(1 - \sigma(F))$,

$$\frac{\partial L}{\partial F} = -\left[y \frac{\sigma'}{\sigma} - (1 - y) \frac{\sigma'}{1 - \sigma}\right] = -[y(1 - \sigma) - (1 - y)\sigma] = \sigma(F) - y,$$

so the pseudo-residual [Eq. \(4.7\)](#) is $r = y - \sigma(F)$: the error in predicted probability, as claimed in [Section 4.6.3](#).

(b) *Pinball loss.* At quantile level $\tau \in (0, 1)$,

$$L_\tau(y, F) = \begin{cases} \tau(y - F), & y \geq F, \\ (1 - \tau)(F - y), & y < F, \end{cases} \quad r = -\frac{\partial L_\tau}{\partial F} = \begin{cases} \tau, & y > F, \\ \tau - 1, & y < F, \end{cases}$$

(the loss is not differentiable at $y = F$; any subgradient value in $[\tau - 1, \tau]$ serves). The boosted model then nudges its prediction up by a constant amount whenever it under-predicts and down whenever it over-predicts, with asymmetric magnitudes τ and $1 - \tau$; the fixed point of that asymmetry is the τ -quantile.

Use in collections. A model of the $\tau = 0.9$ quantile of repayment delay answers, per borrower, “by what day will this loan have been repaid with 90% confidence?” That is the planning quantity a collections operation needs: it sizes the follow-up queue conservatively, sets the day on which an unpaid loan becomes genuinely anomalous (rather than merely late-ish) and therefore worth an agent’s call, and prices expected recovery timelines into cash-flow forecasts. The conditional *mean* delay would understate all three, because delay distributions are heavy-tailed and the cost of under-provisioning is asymmetric — the same cost-sensitivity logic as [Chapter 3](#), applied to a regression target.

Solution to Exercise 4.6. Under squared error $L = \frac{1}{2}(y - F)^2$ we have $g_i = \partial_F L = F(x_i) - y_i = -r_i$ (minus the ordinary residual) and $h_i = \partial_F^2 L = 1$. Hence for leaf j , $G_j = -\sum_{i \in I_j} r_i$ and $H_j = |I_j|$, and [Eq. \(4.11\)](#) gives

$$w_j^* = -\frac{G_j}{H_j + \lambda} = \frac{\sum_{i \in I_j} r_i}{|I_j| + \lambda} \xrightarrow{\lambda=0} \frac{1}{|I_j|} \sum_{i \in I_j} r_i,$$

the mean residual in the leaf — so vanilla gradient boosting ([Section 4.6.3](#)) is recovered as the unregularised special case. For $\lambda > 0$,

$$w_j^* = \frac{H_j}{H_j + \lambda} \cdot \underbrace{\frac{1}{H_j} \sum_{i \in I_j} r_i}_{\text{mean residual}},$$

a shrinkage of the mean residual by the factor $H_j/(H_j + \lambda) \in (0, 1)$. The factor is close to 1 for well-populated leaves and small for thin ones: a leaf holding $H_j = 500$ loans with $\lambda = 5$ keeps 99% of its estimate, while a leaf holding 5 loans keeps only 50%. The ridge penalty is thus an automatic scepticism dial — conclusions drawn from few borrowers are pulled toward “no adjustment” — which is exactly the behaviour one wants in a 4.1%-default portfolio where thin leaves are dominated by noise, and it complements the hard floor that `min_child_samples` imposes in [Listing 4.4](#).

Solution to Exercise 4.7. *Counterexample.* Hold every feature fixed except the delinquency feature `days_past_due` (x , in days), and suppose the unconstrained model’s fitted risk is piecewise constant:

$$F(x) = \begin{cases} 0.10, & 0 \leq x < 10, \\ 0.30, & 10 \leq x < 20, \\ 0.20, & 20 \leq x < 30, \\ 0.50, & x \geq 30. \end{cases}$$

Any reasonable notion of the *average* response is increasing — the least-squares linear fit of F on x has positive slope, and the overall trend runs from 0.10 to 0.50 — yet the model is not monotone: it assigns *lower* risk at $x = 25$ than at $x = 15$.

Governance failure. Consider two applicants identical in every respect except that applicant A has 15 days of arrears and applicant B has 25. Suppose the approval threshold sits at 0.25: A is declined, B is approved. The adverse-action reason issued to A necessarily cites delinquency — it is the only feature in play — yet B, with *strictly worse* delinquency, was approved. The stated reason is therefore falsified by the lender’s own decisions: A’s appeal, a regulator’s file review, or a journalist’s paired test will surface the inconsistency, and under the automated-decision provisions of the Kenya DPA (2019) the lender must stand behind an explanation its model contradicts. The average trend is irrelevant to this failure because adverse-action reasons are given to *individuals*, who live at specific points of the curve, not on its regression line. The monotonic constraint of [Definition 4.5](#) removes the failure mode by construction, at the cost of the (noise-driven) dip — which is precisely the trade described in [Section 4.7.3](#).

Solution to Exercise 4.8. By [Proposition 4.5](#), an equal-resolution grid with $B = 729$ and $d = 6$ uses $r = 729^{1/6} = 3$ distinct values per axis, so each of the two important hyperparameters is probed at just *three* values, each combination repeated across the $3^4 = 81$ settings of the four irrelevant axes. Random search probes 729 distinct values of each important axis.

Probability the grid never enters the best decile of both important axes simultaneously. Model each axis as $[0, 1]$ with the “best decile” an interval of length 0.1 whose location is unknown, and take the grid’s three values per axis

to be evenly spaced with a uniformly random offset, $\{u, u + \frac{1}{3}, u + \frac{2}{3}\}$ with $u \sim \text{U}[0, \frac{1}{3})$ (assumptions stated: independent axes, uniform target location, one-third spacing). Since the target interval $0.1 < \frac{1}{3}$, at most one grid value can fall in it, and the probability that one does is $3 \times 0.1 = 0.3$ per axis. Because the grid is a Cartesian product, hitting the decile marginally on both important axes guarantees a trial at the intersection; the two axes are independent, so

$$P(\text{grid hits both}) = 0.3 \times 0.3 = 0.09, \quad P(\text{grid never does}) = 0.91.$$

For random search, a single trial lands in both deciles with probability $0.1 \times 0.1 = 0.01$, so

$$P(\text{random never does}) = (1 - 0.01)^{729} = 0.99^{729} \approx e^{-7.29} \approx 7 \times 10^{-4}.$$

The grid misses the jointly-best region nine times out of ten; random search finds it essentially always. This is Fig. 4.11 with numbers attached, and it is the practical content of low effective dimensionality (Bergstra and Bengio, 2012).

Solution to Exercise 4.9. With $N = 81$, $b = 25$ and $\eta = 3$, the rungs are:

Rung	1	2	3	4	5
Configurations alive	81	27	9	3	1
Budget each (rounds)	25	75	225	675	2025
Cost of rung	2025	2025	2025	2025	2025

Each rung costs exactly $81 \times 25 = 2025$ configuration-rounds — the $1/\eta$ survival rate and the $\times \eta$ budget growth cancel — so the total is $5 \times 2025 = 10,125$ configuration-rounds. Running all 81 configurations to the full 2,025 rounds would cost $81 \times 2025 = 164,025$: successive halving explores the same 81 candidates for 6.2% of the cost, a $16.2\times$ saving, at the price of trusting rung-1 rankings.

Where the trust fails. The schedule assumes that performance after 25 boosting rounds predicts performance after 2,025. The clearest counterexample in our own chapter is the learning rate (Eq. (4.8)): a configuration with $\nu = 0.02$ is deliberately slow — at 25 rounds it has barely moved from the base rate and looks terrible next to a $\nu = 0.3$ configuration — yet it is exactly the kind of run that attains the *lowest* final validation loss. Aggressive halving systematically executes the low- ν candidates at rung 1 and crowns a fast-starting, earlier-overfitting configuration. Mitigations: hedge across brackets (Hyperband, Li et al., 2017), delay pruning (`n_warmup_steps` in Listing 4.6), or normalise the comparison by learning-curve extrapolation rather than raw score.

Solution to Exercise 4.10. Each measured score is $\hat{\mathcal{V}}_t = v + \varepsilon_t$ with $\varepsilon_t = sZ_t$ and $Z_t \sim \mathcal{N}(0, 1)$ i.i.d. The search reports

$$\max_t \hat{\mathcal{V}}_t = v + s \max_t Z_t \implies \mathbb{E}[\max_t \hat{\mathcal{V}}_t] = v + s \mathbb{E}[\max_{t \leq T} Z_t],$$

since v is common and the maximum commutes with the shift. All T configurations are *equally good* by construction, so the entire excess over v is optimism manufactured by selection. Using the standard extreme-value approximation $\mathbb{E}[\max_{t \leq T} Z_t] \approx \sqrt{2 \ln T}$: for $T = 100$, $\sqrt{2 \ln 100} = \sqrt{9.21} \approx 3.03$, so with fold-level noise $s = 0.004$ in PR-AUC the expected reported best is

$$v + 0.004 \times 3.03 \approx v + 0.012.$$

An HPO study of one hundred trials will thus claim roughly $+0.012$ PR-AUC that is not there, even when tuning does nothing at all. Two readings matter for Lab 4. First, the magnitude is the same order as the optimism gap students typically measure between `study.best_value` and the sealed out-of-time score — part of that gap is drift, but a predictable slice of it is pure winner’s curse, and Table 4.7’s tuned-model row would read ≈ 0.28 rather than 0.294 if the inner-loop best were taken at face value. Second, the formula shows the levers: optimism grows only as $\sqrt{\ln T}$ (slowly — more trials are not catastrophic) but linearly in s , so the cheapest honesty gain is reducing fold noise: more folds, repeated CV, or larger validation windows. And, of course, reporting from data the search never touched (Fig. 4.13) removes the curse entirely — which is the point of the chapter’s final pitfall box.

Chapter 5: Unsupervised Learning and Dimensionality Reduction

Solution to Exercise 5.1. Let $J^*(k)$ denote the optimal inertia with k clusters. Take an optimal solution for k with centroids $\mu_1, \dots, \mu_k$ and assignment c . Build a feasible $(k + 1)$ -cluster solution by choosing any point x_r that is not already alone in its cluster, adding the centroid $\mu_{k+1} = x_r$, and reassigning x_r to it. Every other point keeps its assignment and hence its contribution to Eq. (5.1); the contribution of x_r falls from $\|x_r - \mu_{c(r)}\|^2 \geq 0$ to 0. Therefore $J^*(k + 1) \leq J^*(k)$. At $k = m$ we may place one centroid on each point, giving $J^*(m) = 0$.

Consequently “minimise inertia over k ” always returns $k = m$, which is the degenerate clustering in which every wallet is its own segment: the criterion is vacuous because model complexity is unpenalised. The elbow heuristic is not vacuous because it does not minimise J ; it examines the *rate* at which J falls and looks for the point past which additional clusters

buy little. That is an informal appeal to a complexity penalty, which BIC (Eq. (5.9)) makes formal — and this is precisely why BIC has a genuine interior minimum where inertia does not.

Solution to Exercise 5.2. Partition $C_1 = \{0, 1, 2\}$, $C_2 = \{10, 11, 12\}$. By symmetry it suffices to do C_1 .

- $x = 0$: $a = \frac{1+2}{2} = 1.5$; $b = \frac{10+11+12}{3} = 11$; $s = (11 - 1.5)/11 = 0.8636$.
- $x = 1$: $a = \frac{1+1}{2} = 1$; $b = \frac{9+10+11}{3} = 10$; $s = (10 - 1)/10 = 0.9000$.
- $x = 2$: $a = \frac{2+1}{2} = 1.5$; $b = \frac{8+9+10}{3} = 9$; $s = (9 - 1.5)/9 = 0.8333$.

C_2 mirrors these exactly, so $\bar{s} = \frac{1}{3}(0.8636 + 0.9000 + 0.8333) = 0.8657$.

Partition $C'_1 = \{0, 1\}$, $C'_2 = \{2, 10, 11, 12\}$.

- $x = 0$: $a = 1$; $b = \frac{2+10+11+12}{4} = 8.75$; $s = (8.75 - 1)/8.75 = 0.8857$.
- $x = 1$: $a = 1$; $b = \frac{1+9+10+11}{4} = 7.75$; $s = 0.8710$.
- $x = 2$: $a = \frac{8+9+10}{3} = 9$; $b = \frac{2+1}{2} = 1.5$; $s = (1.5 - 9)/9 = -0.8333$.
- $x = 10$: $a = \frac{8+1+2}{3} = 3.6\bar{6}$; $b = \frac{10+9}{2} = 9.5$; $s = (9.5 - 3.6\bar{6})/9.5 = 0.6140$.
- $x = 11$: $a = \frac{9+1+1}{3} = 3.6\bar{6}$; $b = 10.5$; $s = 0.6508$.
- $x = 12$: $a = \frac{10+2+1}{3} = 4.3\bar{3}$; $b = 11.5$; $s = 0.6232$.

$\bar{s} = \frac{1}{6}(0.8857 + 0.8710 - 0.8333 + 0.6140 + 0.6508 + 0.6232) = 0.4686$.

Comment. The natural partition scores far higher (0.866 versus 0.469), so the silhouette correctly prefers it. More instructive is the single strongly negative value, $s(2) = -0.833$: the point $x = 2$ is on average much closer to C'_1 than to the cluster it was assigned to. This is exactly what the silhouette *diagram* is for. An analyst looking only at the mean would see a mediocre 0.469; an analyst looking at the per-point profile sees immediately *which* point is misassigned and can fix it. Note also that $\bar{s}$ for the bad partition is still positive, because four of six points are content — averages hide misassignment, profiles do not.

Solution to Exercise 5.3. Take the six points, with axis 1 in Kenyan shillings and axis 2 as a count of transactions:

	P_1	P_2	P_3	P_4	P_5	P_6
x_1 (KES)	1000	1100	1200	9000	9100	9200
x_2 (count)	2	40	78	2	40	78

Raw data. Squared distances are dominated by x_1 , whose spread is ~ 8000 against ~ 76 for x_2 . K -means with $k = 2$ therefore splits on value: $\{P_1, P_2, P_3\}$ versus $\{P_4, P_5, P_6\}$. (Check: the within-cluster sum of squares for this split is $2 \cdot (20000 + 2 \cdot 1352.7) \approx 4.5 \times 10^4$, whereas splitting on count gives clusters spanning 8000 in x_1 and costs $\sim 1.3 \times 10^8$.)

Standardised data. After centring and dividing by the sample standard deviations ($\sigma_1 \approx 4271$, $\sigma_2 \approx 34.9$), both axes have unit spread and the configuration becomes a symmetric 2×3 grid at $x_1 \approx \pm 0.94$ and $x_2 \approx \{-1.09, 0, 1.09\}$. The two candidate splits now cost the same by symmetry along x_1 , but the three-level x_2 axis has the larger total spread (± 1.09 against ± 0.94), so K -means prefers to cut along x_2 , returning e.g. $\{P_1, P_4\}$ (low count) against $\{P_2, P_3, P_5, P_6\}$. Perturb the numbers slightly to break the tie in whichever direction you like; the point is that the two answers are different and both are “correct” for their respective metrics.

Which to defend. The standardised one — but the honest defence is not that standardisation is a rule. It is that the raw-data partition answers a question nobody asked: “are these transactions large or small in KES?” is answerable with a single threshold and needs no clustering at all. If the business genuinely wants value tiers, use a threshold and say so. Clustering earns its keep only when several dimensions jointly matter, and that requires them to be commensurable. Where a monetary feature is heavy-tailed, prefer a quantile or log transform to plain standardisation, since a handful of whale wallets will otherwise dominate the standardised axis too.

Solution to Exercise 5.4. Construction. Work in $\mathbb{R}^2$ with $\varepsilon = 1.05$ and $\text{minPts} = 8$. Build two clusters, each a dense blob plus one outlying “antenna” point:

- Cluster A : ten points inside a ball of radius 0.05 centred at $(-0.6, 0)$, plus a single point $p_A = (0, 0)$.
- Cluster B : ten points inside a ball of radius 0.05 centred at $(2.6, 0)$, plus a single point $p_B = (2, 0)$.
- The candidate border point $q = (1, 0)$.

Every blob point is core: it sees the other nine blob points (mutual distance ≤ 0.1) and its antenna (distance $0.6 \pm 0.05 \leq \varepsilon$), so $|N_\varepsilon| = 11 \geq 8$. *Each antenna is core:* p_A sees its blob (ten points at ≈ 0.6), itself, and q (distance exactly $1 \leq \varepsilon$), so $|N_\varepsilon(p_A)| = 12 \geq 8$. *q is not core:* its ε -ball contains only q , p_A and p_B , since both blobs sit at distance $\approx 1.6 > \varepsilon$; thus $|N_\varepsilon(q)| = 3 < 8$. Finally A and B are distinct components, because $d(p_A, p_B) = 2 > \varepsilon$ and q — the only object between them — is not core, so it cannot propagate reachability.

Therefore q is a border point directly density-reachable from p_A and from p_B , which belong to different clusters. DBSCAN assigns q to whichever cluster is expanded first: process p_A before p_B and q joins A ; reverse the visitation order and q joins B . Its label is genuinely order-dependent.

Why this does not undermine reproducibility. By Proposition 5.3, the core-point set and the partition of core points into clusters are order-invariant. The ambiguity therefore never changes the number of clusters, their shape, or the membership of any core point — only the label of a point sitting in a low-density bridge, whose ε -ball is by construction almost empty. Operationally: the segments you name and price are determined by their cores, and a point whose label flips is exactly one about which you should not be making a confident decision. If bit-for-bit determinism is required for audit, fix the input ordering and record it; or use HDBSCAN, which assigns such points by membership strength rather than by visitation order.

Solution to Exercise 5.5. With $k = 2$, $\pi_1 = \pi_2 = \frac{1}{2}$, and shared variance σ^2 , the Gaussian normalising constants cancel in Eq. (5.5):

$$\gamma_{i1} = \frac{\exp\left(-\frac{(x_i - \mu_1)^2}{2\sigma^2}\right)}{\exp\left(-\frac{(x_i - \mu_1)^2}{2\sigma^2}\right) + \exp\left(-\frac{(x_i - \mu_2)^2}{2\sigma^2}\right)} = \frac{1}{1 + \exp\left(\frac{(x_i - \mu_1)^2 - (x_i - \mu_2)^2}{2\sigma^2}\right)}.$$

Expanding the difference of squares, $(x - \mu_1)^2 - (x - \mu_2)^2 = -2x(\mu_1 - \mu_2) + \mu_1^2 - \mu_2^2 = -2(\mu_1 - \mu_2)\left(x - \frac{\mu_1 + \mu_2}{2}\right)$. Hence

$$\gamma_{i1} = \sigma_{\text{logistic}}(w(x_i - x_0)), \quad w = \frac{\mu_1 - \mu_2}{\sigma^2}, \quad x_0 = \frac{\mu_1 + \mu_2}{2},$$

where $\sigma_{\text{logistic}}(z) = (1 + e^{-z})^{-1}$. So the responsibility is exactly a logistic function of x with **slope parameter** $w = (\mu_1 - \mu_2)/\sigma^2$ and **midpoint** at the arithmetic mean of the two component means, where $\gamma_{i1} = \frac{1}{2}$.

As $\sigma^2 \rightarrow 0^+$ with $\mu_1 \neq \mu_2$ fixed, $|w| \rightarrow \infty$: the logistic curve steepens into a step at x_0 , so $\gamma_{i1} \rightarrow \mathbb{1}\{x_i \text{ nearer } \mu_1\}$ for every $x_i \neq x_0$. This is precisely Proposition 5.5 in one dimension: the midpoint x_0 is the perpendicular bisector (here, a single point), and the soft assignment collapses to K -means' hard nearest-centroid rule. The general lesson is that σ^2 acts as a temperature: large σ^2 gives soft, hedged memberships; small σ^2 gives confident hard ones. Reporting a segment membership of 0.51 versus 0.49 as though it were a fact is a failure to read this parameter.

Solution to Exercise 5.6. Divergence. Take $m \geq 2$ distinct points and the family

$$\theta_\epsilon : \quad \pi_1 = \pi_2 = \frac{1}{2}, \quad \mu_1 = x_1, \quad \sigma_1^2 = \epsilon, \quad \mu_2 = \bar{x}, \quad \sigma_2^2 = s^2 > 0 \text{ fixed.}$$

The first component's density at x_1 is $\mathcal{N}(x_1 \mid x_1, \epsilon) = (2\pi\epsilon)^{-1/2}$, so the contribution of observation 1 to the log-likelihood is at least $\log(\frac{1}{2}(2\pi\epsilon)^{-1/2}) \rightarrow +\infty$ as $\epsilon \rightarrow 0^+$. Every other observation $i \geq 2$ retains the second component's contribution, bounded below by $\log(\frac{1}{2}\mathcal{N}(x_i \mid \bar{x}, s^2)) > -\infty$, which does not depend on ϵ . Hence $\ell(\theta_\epsilon) \rightarrow +\infty$: the likelihood is unbounded above and no global maximiser exists. Geometrically, the fitted model has spiked onto a single data point and explains nothing.

Effect of reg_covar. Adding $\lambda > 0$ to the diagonal replaces σ_j^2 by $\sigma_j^2 + \lambda$, so the achievable density at a point is bounded by $(2\pi\lambda)^{-1/2}$ and the per-observation log-likelihood by $\log(\pi_{\max}(2\pi\lambda)^{-1/2}) < \infty$. Summing over m observations bounds ℓ above, restoring existence of a maximum on the (now compact-in-effect) parameter region, and the degenerate spikes are removed from the search space entirely.

Price paid. Three costs. (i) The estimator is biased: every fitted covariance is inflated by λ , so genuinely tight clusters are reported as looser than they are, and density-based anomaly scores are correspondingly flattened. (ii) The reported likelihood, and therefore BIC, is no longer that of the model you claim to be fitting, so BIC comparisons are only valid at a *fixed* λ . (iii) λ is a scale-dependent hyperparameter: the same value means something different on standardised and unstandardised features, which is one more reason to fix the scaling before the mixture. In practice keep λ at the smallest value that prevents collapse (the scikit-learn default 10^{-6} on standardised data is usually right) and treat a fit that needs a large λ as evidence that k is too big.

Solution to Exercise 5.7. Free parameters split into weights, means and covariances. Weights contribute $k - 1$ (they sum to one); means contribute kn . Covariances:

covariance_type	Covariance parameters	Total p
full	$k \frac{n(n+1)}{2}$	$(k-1) + kn + k \frac{n(n+1)}{2}$
tied	$\frac{n(n+1)}{2}$	$(k-1) + kn + \frac{n(n+1)}{2}$
diag	kn	$(k-1) + 2kn$
spherical	k	$(k-1) + kn + k$

For $n = 32, k = 5$: $k - 1 = 4$ and $kn = 160$.

- full: $\frac{32 \cdot 33}{2} = 528$ per component, so $p = 4 + 160 + 5(528) = \mathbf{2804}$.
- tied: $p = 4 + 160 + 528 = 692$.
- diag: $p = 4 + 2(160) = \mathbf{324}$.
- spherical: $p = 4 + 160 + 5 = 169$.

With $m = 150,000$, $\ln m = \ln(1.5 \times 10^5) \approx 11.9184$. For full to beat diag on BIC we need

$$p_{\text{full}} \ln m - 2 \ln \hat{L}_{\text{full}} < p_{\text{diag}} \ln m - 2 \ln \hat{L}_{\text{diag}},$$

i.e.

$$\begin{aligned} \ln \hat{L}_{\text{full}} - \ln \hat{L}_{\text{diag}} &> \frac{1}{2}(2804 - 324)(11.9184) \\ &= \frac{1}{2}(2480)(11.9184) \approx \mathbf{14,778}. \end{aligned}$$

Averaged over the sample this is only $14,778/150,000 \approx 0.0985$ nats per observation — about a 10% relative improvement in average likelihood. So with 150,000 wallets, full covariances are a *cheap* upgrade: modest correlation structure within segments will pay for 2480 extra parameters comfortably. Repeat the arithmetic at $m = 2000$ and the required gap becomes ≈ 0.5 nats per observation, at which point diag usually wins. The general moral is that the “right” covariance type is a function of sample size, not of taste.

Solution to Exercise 5.8. Independent coordinates. A short simulation (numpy default generator, seed 0; $m = 1000$ points and one query point drawn from the same distribution) gives ratios of the order

n	2	10	100	1000
i.i.d. coordinates	121	3.04	0.44	0.12
5-factor latent model	179	10.4	8.88	7.55

(means over 20 replications; the $n = 2$ column is enormous and wildly variable because $D_{\min}$ can be arbitrarily small in low dimension, so read the *trend* across columns, not the individual digits). In the i.i.d. row the contrast between nearest and furthest neighbour collapses roughly like $n^{-1/2}$, exactly as Proposition 5.6 predicts from $\text{Var}[D_n^2]/(\mathbb{E}[D_n^2])^2 = \mathcal{O}(1/n)$.

Latent factor model. Generate $u_i \in \mathbb{R}^5$ i.i.d. standard normal and set $x_i = Au_i + 0.05 \eta_i$ with $A \in \mathbb{R}^{n \times 5}$ fixed and η_i small isotropic noise (second row of the table). The ratio now stays around 8–10 from $n = 10$ all the way to $n = 1000$: raising the ambient dimension by two orders of magnitude does essentially nothing, because the distances are governed by the five latent coordinates and the extra measured dimensions carry almost no independent variability.

Explanation. The proof of Proposition 5.6 needs the coordinate contributions δ_t to be (near-)independent, so that $\text{Var}[D_n^2]$ grows like n while $\mathbb{E}[D_n^2]$ grows like n , forcing the relative variance to 0. Under the factor model the δ_t are strongly dependent: the effective, or intrinsic, dimension is 5, not n , and it is the intrinsic dimension that governs concentration. This is the entire justification for the chapter’s strategy. We do not escape the curse by wishing it away; we escape it because real behavioural features are redundant, and PCA finds the small number of directions along

which the data actually varies. It is also why one should measure the explained-variance curve before concluding that a 200-feature problem is high-dimensional: it may have an intrinsic dimension of 20.

Solution to Exercise 5.9. Proof. From the proof of Proposition 5.7,

$$E(W) = \frac{1}{m} \sum_i \|x_i\|^2 - \text{tr}(W^\top S W).$$

For centred data $\frac{1}{m} \sum_i \|x_i\|^2 = \text{tr}(S) = \sum_{t=1}^n \lambda_t$. At the optimum $W = W_d$ we have, by Theorem 5.1, $\text{tr}(W_d^\top S W_d) = \sum_{t \leq d} \lambda_t$. Hence

$$E(W_d) = \sum_{t=1}^n \lambda_t - \sum_{t \leq d} \lambda_t = \sum_{t > d} \lambda_t,$$

as claimed: the mean squared reconstruction error is exactly the variance you threw away.

Numerical verification. With $S = \begin{pmatrix} 4 & 2 \\ 2 & 4 \end{pmatrix}$ from Example 5.2, $\lambda_1 = 6$, $\lambda_2 = 2$, $v_1 = \frac{1}{\sqrt{2}}(1, 1)^\top$. For $d = 1$ the formula predicts $E = \lambda_2 = 2$. Take three centred points consistent with S , for instance

$$x_1 = (3, 1)^\top, \quad x_2 = (-1, -3)^\top, \quad x_3 = (-2, 2)^\top.$$

Projection: $z_i = v_1^\top x_i$ gives $z_1 = 4/\sqrt{2} = 2\sqrt{2}$, $z_2 = -4/\sqrt{2} = -2\sqrt{2}$, $z_3 = 0$. Reconstruction $\hat{x}_i = z_i v_1$: $\hat{x}_1 = (2, 2)^\top$, $\hat{x}_2 = (-2, -2)^\top$, $\hat{x}_3 = (0, 0)^\top$. Residuals: $x_1 - \hat{x}_1 = (1, -1)^\top$ with squared norm 2; $x_2 - \hat{x}_2 = (1, -1)^\top$, squared norm 2; $x_3 - \hat{x}_3 = (-2, 2)^\top$, squared norm 8. Mean = $(2 + 2 + 8)/3 = 4$.

The mean over this particular triple is 4, not 2, because three points do not reproduce S : their own sample covariance is $\begin{pmatrix} 14/3 & 2/3 \\ 2/3 & 14/3 \end{pmatrix}$, whose second eigenvalue is $14/3 - 2/3 = 4$ — and 4 is exactly what the formula predicts for *their* covariance. The identity holds sample-wise, and checking it this way is a useful reminder that λ_t are properties of the sample you actually projected, not of an assumed population.

Solution to Exercise 5.10. A defensible one-page answer covers four things.

What to measure next. (i) Whether the disparity is in the *clustering* or in the *limit rule*: compute segment membership rates by county, then, within segment, compare limits — if limits are equal within segment, the mechanism is composition, not pricing. (ii) The behavioural distance between this segment and its neighbours: if the silhouette for this cluster is weak, the segment may be an artefact and the disparity a modelling choice rather than a finding. (iii) Whether the underlying features are themselves proxies: regress county on the top principal components and see how predictable it

is; high predictability means the “behavioural” representation substantially encodes geography. (iv) Outcome data where available — if this segment’s historical default rate genuinely differs, the disparity may be justified; if it does not, the 40% limit reduction lacks a risk basis.

Instruments engaged. Both discussed in the chapter’s regulatory box. The Data Protection Act, 2019 governs the derived segment as personal data: lawful basis, purpose limitation, and the data subject’s rights of access and objection apply to an inferred behavioural profile as much as to the raw transaction log. The CBK Digital Credit Providers Regulations, 2022 require pricing and terms to be disclosed and non-exploitative and credit decisions to be explicable — an unexplainable “cluster 3” limit rule does not meet that standard, and a limit reduction concentrated on two counties invites scrutiny of whether the differentiation is risk-based or merely correlational.

Two remedies with costs. (i) *Re-fit excluding geography-loaded features* (agent-location density, merchant-mix features tied to regional paybills). Cost: some genuine predictive signal is lost, so expected credit losses rise; and the remedy is incomplete, since geography can be reconstructed from residual features. (ii) *Keep the segmentation but sever it from pricing*, using segments only for product design and communication while limits are set by the audited, monotone scorecard of [Chapter 4](#) with fairness constraints. Cost: loses the operational simplicity of segment-based pricing, and requires a second governance process for the scorecard. A third, cheaper option worth listing: cap the maximum limit differential across segments, which bounds the harm without discarding the model.

When shipping unchanged is defensible. If the disparity is *explained* by a legitimate, disclosed, risk-relevant behavioural difference that survives conditioning on geography — for example, the segment is defined by seasonal single-harvest income, the lower limit reflects a measured higher default rate for bullet-repayment products, the county concentration is a consequence of where that livelihood occurs rather than of county itself, and an applicant from either county with non-seasonal income lands in a different segment. The test is counterfactual: does an otherwise identical borrower from a different county receive the same limit? If yes, the segment is tracking livelihood, not location, and the position is arguable — provided the reasoning, the tests and the residual disparity are documented before launch, and monitored afterwards.

Chapter 6: Deep Learning Fundamentals

Solution to Exercise 6.1. By Proposition [6.1](#), the three linear hidden layers collapse to a single affine map. Writing $\mathbf{W}^{(1)} \in \mathbb{R}^{128 \times 40}$, $\mathbf{W}^{(2)} \in \mathbb{R}^{64 \times 128}$,

$\mathbf{W}^{(3)} \in \mathbb{R}^{32 \times 64}$ and $\mathbf{w}^{(4)} \in \mathbb{R}^{1 \times 32}$, the network computes

$$\hat{p} = \sigma(\tilde{\mathbf{w}}^\top \mathbf{x} + \tilde{b}), \quad \tilde{\mathbf{w}}^\top = \mathbf{w}^{(4)} \mathbf{W}^{(3)} \mathbf{W}^{(2)} \mathbf{W}^{(1)} \in \mathbb{R}^{1 \times 40}.$$

The model class is therefore **logistic regression on the original 40 features** — nothing more. Note in particular that the widths 128, 64, 32 do not enlarge the class: the composite matrix has rank at most 1 regardless, because the output layer is one-dimensional.

Counting parameters layer by layer, using Remark 6.1:

$$\underbrace{41 \times 128}_{5248} + \underbrace{129 \times 64}_{8256} + \underbrace{65 \times 32}_{2080} + \underbrace{33 \times 1}_{33} = 15,617,$$

against $40 + 1 = 41$ for the equivalent logistic regression. The network therefore carries $15,617 - 41 = 15,576$ redundant parameters, roughly 381 times the minimum, while representing exactly the same set of functions.

Two qualifications are worth making, because the exercise is not *only* a counting exercise. First, redundant parameterisation is not free at training time: it multiplies memory traffic and gradient computation for no gain in expressiveness. Second, the over-parameterised form is not perfectly equivalent *as an optimisation problem*: the loss surface of a deep linear network is non-convex in θ even though the induced function class is the convex-friendly logistic one, and the factorised parameterisation induces an implicit bias in the trajectory of gradient descent. The correct diagnosis remains that the architecture is a mistake: if a linear model is wanted, fit a linear model; if non-linearity is wanted, insert an activation function.

Solution to Exercise 6.2. Part 1. By Proposition 6.2, $\sigma'(z) = s(1-s)$ with $s = \sigma(z)$. Solving $s(1-s) = 0.01$, that is $s^2 - s + 0.01 = 0$, gives

$$s = \frac{1 \pm \sqrt{1 - 0.04}}{2} = \frac{1 \pm \sqrt{0.96}}{2} \in \{0.010102, 0.989898\}.$$

Since $s \mapsto s(1-s)$ is a downward parabola, $\sigma'(z) < 0.01$ precisely when s lies outside $[0.010102, 0.989898]$. Converting the upper root back to the logit scale,

$$z = \ln \frac{s}{1-s} = \ln \frac{0.989898}{0.010102} = \ln(97.99) = 4.5849,$$

and by the symmetry $\sigma(-z) = 1 - \sigma(z)$ the lower root gives -4.5849 . Hence

$$\sigma'(z) < 0.01 \iff |z| > 4.585 \quad (\text{to three decimal places}).$$

A pre-activation of magnitude only about 4.6 — entirely unremarkable for unstandardised inputs — already suppresses the gradient by a factor of at least 25 relative to its maximum of $1/4$.

Part 2. With $\|\mathbf{W}^{(l)}\| = 1$ and $\gamma = 1/4$, Proposition 6.10 gives the bound $(\gamma w)^{L-1} = 4^{-(L-1)}$. Requiring $4^{-(L-1)} \leq 10^{-6}$,

$$(L-1) \ln 4 \geq 6 \ln 10 \implies L-1 \geq \frac{6 \times 2.302585}{1.386294} = 9.97,$$

so $L-1 = 10$ multiplicative factors suffice, that is a network of $\mathbf{L} = 11$ layers. Ten sigmoid layers are enough to reduce the error signal reaching the first layer by six orders of magnitude *even when the weight matrices are perfectly scaled*. This is the quantitative content of the claim that deep sigmoid stacks do not train, and it is why Section 6.9 treats initialisation and Section 6.10 treats normalisation as necessities rather than refinements.

Solution to Exercise 6.3. Take the convention of Example 6.2, in which $\mathbf{z}^{(1)} = \mathbf{W}^{(1)\top} \mathbf{x} + \mathbf{b}^{(1)}$, and set $\mathbf{x} = (-0.5, 1.2)^\top$:

$$z_1^{(1)} = 1(-0.5) + 0.5(1.2) = 0.1, \quad z_2^{(1)} = -2(-0.5) + 1(1.2) - 0.4 = 1.8,$$

so $\mathbf{z}^{(1)} = (0.1, 1.8)^\top$ and, since both components are positive, $\mathbf{h}^{(1)} = \text{ReLU}(\mathbf{z}^{(1)}) = (0.1, 1.8)^\top$. The output logit is

$$z^{(2)} = 2(0.1) - 1(1.8) - 0.1 = -1.7, \quad \hat{p} = \sigma(-1.7) = \frac{1}{1 + e^{1.7}} = 0.15447.$$

With $y = 1$ the loss is $\mathcal{L} = -\ln(0.15447) = 1.868$, and by Proposition 6.5 the output error signal is

$$\delta^{(2)} = \hat{p} - y = 0.15447 - 1 = -0.84553.$$

It is negative, correctly instructing the network to *increase* $z^{(2)}$ for this borrower.

Applying (6.17) at the output layer,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}^{(2)}} = \delta^{(2)} \mathbf{h}^{(1)} = -0.84553 \begin{pmatrix} 0.1 \\ 1.8 \end{pmatrix} = \begin{pmatrix} -0.08455 \\ -1.52196 \end{pmatrix}, \quad \frac{\partial \mathcal{L}}{\partial b^{(2)}} = -0.84553.$$

The second component is eighteen times the first, purely because $h_2^{(1)} = 1.8$ is eighteen times $h_1^{(1)}$: gradient magnitude at a weight is proportional to the activation it multiplies, which is the algebraic reason unscaled activations produce unbalanced updates.

Applying the recursion (6.16) with $\text{ReLU}'(z) = 1$ for $z > 0$,

$$\delta^{(1)} = (\mathbf{w}^{(2)} \delta^{(2)}) \odot \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 2(-0.84553) \\ -1(-0.84553) \end{pmatrix} = \begin{pmatrix} -1.69107 \\ 0.84553 \end{pmatrix},$$

and, since $z_k^{(1)} = \sum_j W_{jk}^{(1)} x_j + b_k^{(1)}$ implies $\partial \mathcal{L} / \partial W_{jk}^{(1)} = x_j \delta_k^{(1)}$,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \mathbf{x} \delta^{(1)\top} = \begin{pmatrix} 0.84553 & -0.42277 \\ -2.02928 & 1.01464 \end{pmatrix}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(1)}} = \begin{pmatrix} -1.69107 \\ 0.84553 \end{pmatrix}.$$

The comparison with Example 6.2. Neither hidden unit fails to update here: both pre-activations are positive, so $\text{ReLU}'(z_k^{(1)}) = 1$ for $k = 1, 2$ and both rows of $\partial\mathcal{L}/\partial\mathbf{W}^{(1)}$ are non-zero. This is the opposite of Example 6.2, where the second unit had $z_2^{(1)} = -1.70 < 0$, its activation derivative was zero, and the Hadamard product in (6.16) annihilated its error signal, leaving its incoming weights untouched by that example.

The instructive point is that the two examples together show ReLU inactivity is a property of the *example*, not of the unit. Unit 2 was silent for the first borrower and highly active for the second, and over a mini-batch it receives gradient from whichever examples excite it. A unit becomes *dead* in the sense of the pitfall in Section 6.3 only when its pre-activation is negative for *every* example in the training set, at which point no batch can revive it.

Solution to Exercise 6.4. Take $\mathcal{L}(y, \hat{p}) = \frac{1}{2}(y - \hat{p})^2$ with $\hat{p} = \sigma(z)$. By the chain rule and Proposition 6.2,

$$\frac{\partial\mathcal{L}}{\partial z} = \frac{\partial\mathcal{L}}{\partial\hat{p}} \cdot \frac{d\hat{p}}{dz} = -(y - \hat{p}) \sigma'(z) = (\hat{p} - y) \hat{p} (1 - \hat{p}).$$

Comparing with $\partial\mathcal{L}/\partial z = \hat{p} - y$ from Proposition 6.5, the squared-error gradient carries the extra factor $\hat{p}(1 - \hat{p}) = \sigma'(z) \leq \frac{1}{4}$.

That factor is harmful in the worst possible place. Consider an example with $y = 1$ that the network confidently misclassifies, $\hat{p} = 0.01$. The residual $\hat{p} - y = -0.99$ is nearly maximal, but the gradient is $-0.99 \times 0.01 \times 0.99 = -0.0098$: the network learns *a hundred times more slowly* from the example it gets most badly wrong. Cross-entropy, by contrast, delivers the full residual -0.99 . The saturating factor makes the learning rate effectively example-dependent in exactly the wrong direction, and it re-introduces the vanishing-gradient mechanism of Proposition 6.10 at the output layer, where Proposition 6.5 had shown it could be eliminated by pairing the loss with the activation correctly. A secondary objection is that the squared error composed with a sigmoid is non-convex in z even for a single linear layer, whereas cross-entropy is convex, so the choice also costs optimisation guarantees in the simplest case.

Solution to Exercise 6.5. By Proposition 6.6, $\text{Var}[\hat{\mathbf{g}}_B] = \text{Var}[\mathbf{g}_i]/B$, so the standard deviation scales as $B^{-1/2}$. Increasing B from 64 to 512 multiplies the batch size by 8 and therefore multiplies the gradient standard deviation by

$$\sqrt{\frac{64}{512}} = \frac{1}{\sqrt{8}} = 0.354,$$

a reduction of about 64.6%.

Is the trade favourable? Under the stated cost model — time strictly proportional to B — it is not. Eight times the computation buys a factor of

$\sqrt{8} \approx 2.83$ in noise. The comparison that matters is against the alternative use of the same budget: eight successive steps at $B = 64$. Those eight steps consume the same computation, and if the parameters move little over the eight steps their averaged gradient also has standard deviation reduced by $\sqrt{8}$ — the same noise reduction, but with eight actual parameter updates instead of one. Small batches therefore dominate at equal compute in this idealised accounting, which is the theoretical reason mini-batch SGD outperforms batch gradient descent per unit of work.

The assumptions under which this holds, and why practice departs from them:

1. *The variance model.* Examples are drawn i.i.d. (with replacement); without replacement a finite-population correction applies, which matters only when B is a non-negligible fraction of n .
2. *Stationarity within the comparison.* The eight-step argument assumes $\text{Var}[\mathbf{g}_i]$ and ∇J are approximately constant over those eight updates. Near a sharp minimum or with a large learning rate this fails.
3. *Strict proportionality of time to B .* This is the assumption that most clearly breaks on real hardware. A GPU processing a batch of 64 is largely idle; a batch of 512 may cost far less than eight times as much wall-clock time, and in that regime the large batch wins outright.
4. *Fixed learning rate.* Larger batches permit a larger stable η , which can convert the noise reduction into faster progress rather than merely quieter steps.

Finally, noise is not purely a cost: [Keskar et al. \(2017\)](#) give evidence that the extra gradient noise of small batches biases the optimiser toward flatter minima that generalise better, so a pure variance-minimisation objective is the wrong target in the first place.

Solution to Exercise 6.6. Apply Proposition 6.7 with $\lambda_{\min} = 0.02$ and $\lambda_{\max} = 8$.

Largest stable learning rate. Convergence holds if and only if $0 < \eta < 2/\lambda_{\max}$, so

$$\eta_{\text{crit}} = \frac{2}{8} = 0.25,$$

and any $\eta \geq 0.25$ diverges along the sharpest curvature direction.

Optimal learning rate.

$$\eta^* = \frac{2}{\lambda_{\min} + \lambda_{\max}} = \frac{2}{8.02} = 0.24938.$$

Note how close η^* lies to η_{crit} : the optimal step is within 0.25% of the divergence boundary. This is characteristic of ill-conditioned problems and

explains why tuning η on a badly conditioned loss feels like balancing on a knife edge — and why standardising inputs, which reduces κ , is worth more than heroic learning-rate search.

Contraction factor. With $\kappa = 8/0.02 = 400$,

$$\rho = \frac{\kappa - 1}{\kappa + 1} = \frac{399}{401} = \mathbf{0.995012}.$$

Iterations for a hundredfold error reduction. Requiring $\rho^t \leq 10^{-2}$,

$$t \geq \frac{\ln 10^{-2}}{\ln \rho} = \frac{-4.60517}{-0.0049999} = 921.03 \implies t = \mathbf{922} \text{ iterations.}$$

For comparison, a perfectly conditioned problem with $\kappa = 1$ has $\rho = 0$ and converges in a single step. The 922 iterations are the entire price of the condition number, and since $\rho \approx 1 - 2/\kappa$ for large κ , the iteration count grows roughly linearly in κ : halving the condition number by rescaling one badly scaled feature halves the training time.

Solution to Exercise 6.7. Part 1. By Proposition 6.11, $\varsigma^2 = 2/n_{\text{in}} = 2/256 = 0.0078125$, so

$$\varsigma = \sqrt{0.0078125} = \frac{1}{\sqrt{128}} = \mathbf{0.08839}.$$

This is what `he_normal` uses: a zero-mean Gaussian with that standard deviation.

Part 2. Repeat the derivation with $\text{LReLU}_\alpha(u) = \max(\alpha u, u)$, that is $x = u$ for $u > 0$ and $x = \alpha u$ for $u \leq 0$. Let u be symmetric about zero with variance τ_{pre}^2 . Splitting the expectation over the two half-lines and using symmetry, which makes each half contribute $\frac{1}{2}\mathbb{E}[u^2]$,

$$\mathbb{E}[x^2] = \mathbb{E}[u^2 \mathbf{1}\{u > 0\}] + \alpha^2 \mathbb{E}[u^2 \mathbf{1}\{u \leq 0\}] = \frac{1}{2}\tau_{\text{pre}}^2 + \alpha^2 \cdot \frac{1}{2}\tau_{\text{pre}}^2 = \frac{1}{2}(1 + \alpha^2)\tau_{\text{pre}}^2.$$

Substituting into $\text{Var}(z) = n_{\text{in}}\varsigma^2\mathbb{E}[x^2]$ from the first part of Proposition 6.11 and requiring $\text{Var}(z) = \tau_{\text{pre}}^2$ for variance preservation across layers,

$$n_{\text{in}} \varsigma^2 \frac{1}{2}(1 + \alpha^2)\tau_{\text{pre}}^2 = \tau_{\text{pre}}^2 \implies \varsigma^2 = \frac{2}{(1 + \alpha^2)n_{\text{in}}},$$

as required. Two sanity checks confirm the formula. At $\alpha = 0$ the activation is ordinary ReLU and the result reduces to $2/n_{\text{in}}$. At $\alpha = 1$ the activation is the identity, no half of the distribution is discarded, and the result becomes $1/n_{\text{in}}$ — the classical variance-preserving scale for a linear layer. For the usual $\alpha = 0.01$ the correction factor is $1/(1 + 10^{-4})$, numerically indistinguishable from He, so in practice `he_normal` is used for leaky ReLU without adjustment.

Solution to Exercise 6.8. Part 1. In Proposition 6.12 the penalty term is $q(1 - q) \sum_j w_j^2 x_j^2$ with $q = 1 - p$ the keep probability. At $p = 0.5$ we have $q = 0.5$ and

$$q(1 - q) = 0.5 \times 0.5 = \mathbf{0.25},$$

so the implied penalty is $0.25 \sum_j w_j^2 x_j^2$, and the point prediction is additionally shrunk deterministically by the factor $q = 0.5$. Averaged over the training set, the effective per-coordinate ridge coefficient is $0.25 \mathbb{E}[x_j^2]$: dropout is a ridge penalty *weighted by each feature's second moment*. Note also that $q(1 - q)$ is maximised at $q = 1/2$, so $p = 0.5$ is the most aggressive drop rate in this sense; both $p \rightarrow 0$ and $p \rightarrow 1$ send the penalty to zero, the latter for the uninteresting reason that the model is destroyed.

Part 2. Suppose feature j is left in raw KES with typical magnitude 10^6 , so $\mathbb{E}[x_j^2] \approx 10^{12}$, while a standardised feature k has $\mathbb{E}[x_k^2] = 1$. The implied penalties are then in the ratio

$$\frac{0.25 \times 10^{12}}{0.25 \times 1} = 10^{12}.$$

Coordinate j is penalised a trillion times more heavily than coordinate k , which drives w_j to essentially zero: the feature is silently deleted from the model by the regulariser, no matter how predictive it is. The reverse case is equally bad — a feature measured in millionths would be effectively unregularised and free to overfit.

This is the same lesson as Section 6.12, arriving from a different direction. There, unstandardised inputs damaged the *conditioning* of the optimisation via Proposition 6.7; here they damage the *regularisation* by making the implied penalty an arbitrary function of the measurement unit. In both cases the defect is that a modelling decision has become a consequence of whether an analyst recorded an amount in shillings or in thousands of shillings. Standardising the inputs makes $\mathbb{E}[x_j^2] \approx 1$ for every j and restores the intended uniform penalty.

Solution to Exercise 6.9. Counting. Of 120,000 agents, 71% appear fewer than five times, that is $0.71 \times 120,000 = 85,200$ low-support agents, leaving 34,800 with support of five or more.

Without collapsing, the table has one row per agent plus one reserved out-of-vocabulary row:

$$(120,000 + 1) \times 16 = \mathbf{1,920,016} \text{ parameters.}$$

Collapsing the 85,200 low-support agents into the reserved index 0 leaves 34,800 individual rows plus that shared row:

$$(34,800 + 1) \times 16 = \mathbf{556,816} \text{ parameters,}$$

a saving of 1,363,200 parameters, or 71% of the table — unsurprisingly the same proportion as the collapsed vocabulary, since the cost is linear in the number of rows.

Which variant to submit, and why. Submit the collapsed variant. The statistical argument is decisive: each low-support agent has 16 free parameters fitted from at most four labelled observations. With a 4% base rate, the overwhelming majority of those agents have *no defaults at all* in the training set, so the fitted embedding encodes the accident of which handful of borrowers happened to use that kiosk rather than any property of the agent. That is memorisation, and it will not replicate on the test fold.

Three further arguments reinforce the choice. It trains the out-of-vocabulary path: index 0 receives gradient from 85,200 agents' worth of examples, so a genuinely unseen agent at inference time is mapped to a meaningful average rather than to an untrained initialisation — the failure mode of the pitfall in Section 6.13. It reduces the parameter count from roughly 400 times the rest of the network to roughly 115 times, which materially reduces the memory footprint of the served artefact. And it is more defensible under the governance expectations of Section 6.15, since a reviewer can be shown the support threshold as an explicit, documented modelling decision.

What evidence would settle it. Fit both variants under an identical protocol — same frozen splits, same preprocessing, same search budget, same early-stopping criterion — and compare, in order: test PR-AUC and expected cost; performance restricted to the *low-support agent subgroup*, which is where the two variants differ and where the aggregate metric may hide the difference; and calibration by county. A useful diagnostic beyond the headline metrics is to plot the norm of each learned embedding row against that agent's training support in the uncollapsed model: if the rare rows have drifted far from their initialisation, the model is fitting noise, and the plot makes that visible to a non-technical reviewer. Finally, the support threshold itself should be treated as a hyperparameter — 5 is a convention, not a result — and swept over, say, $\{3, 5, 10, 25\}$ on the validation fold.

Solution to Exercise 6.10. Proposition 6.14 establishes that $z \mapsto \sigma(z/T)$ is strictly increasing, hence order-preserving. Everything follows from distinguishing quantities that depend on the scores *only through their order* from those that depend on their *values*.

1. **ROC-AUC: unchanged.** It is a functional of the ranking alone (the probability that a random positive outranks a random negative), and the ranking is preserved.
2. **PR-AUC: unchanged.** Precision and recall at each operating point depend only on which examples fall above the cut, and the family of sets obtainable by thresholding is identical before and after scaling.

3. **Brier score: changes.** It is the mean squared difference between predicted probability and outcome, a function of the values themselves. This is precisely the quantity temperature scaling is fitted to improve; with $T = 1.8 > 1$ the logits shrink toward zero and the probabilities toward 0.5, which reduces the Brier score for an over-confident model and would increase it for an under-confident one.
4. **Expected cost at the fixed threshold $\hat{p} = 0.15$: changes.** A fixed threshold on the *probability* scale is not a fixed threshold on the score scale. Before scaling, $\hat{p} = 0.15$ corresponds to the logit $\text{logit}(0.15) = -1.7346$; after scaling, $\sigma(z/1.8) = 0.15$ requires $z = 1.8 \times (-1.7346) = -3.1223$. A different set of applicants is declined, so the confusion matrix and hence the cost move.
5. **Expected cost at the re-derived cost-optimal threshold: unchanged.** Optimising the threshold means minimising cost over the family of decision sets obtainable by thresholding the score. Since the transformation is a strictly increasing bijection, that family is identical before and after, so the minimum over it is identical too. The optimal *threshold value* moves; the optimal *cost* does not.

Remark B.1. The last two items together carry the practical message. Temperature scaling cannot improve the best achievable cost — item 5 — so it is not a performance technique. What it does is make the probability scale trustworthy, so that the analytically derived threshold is the right one. Under the cost matrix of the running case study the indifference point is where $\hat{p} \cdot 10,000 = (1 - \hat{p}) \cdot 800$, that is

$$\hat{p}^* = \frac{800}{10,800} = 0.0741.$$

Applying that threshold to *uncalibrated* scores gives the cost-optimal decision only by luck; applying it after calibration is a principled derivation. That is the entire value proposition: item 5 says calibration costs nothing in ranking, and this remark says it buys correctness in the mapping from scores to decisions.

Bibliography

- Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2623–2631.
- Arthur, D. and Vassilvitskii, S. (2007). *k*-means++: The advantages of careful seeding. In *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1027–1035.
- Ba, J. L., Kiros, J. R., and Hinton, G. E. (2016). Layer normalization. *arXiv preprint arXiv:1607.06450*.
- Baydin, A. G., Pearlmutter, B. A., Radul, A. A., and Siskind, J. M. (2018). Automatic differentiation in machine learning: A survey. *Journal of Machine Learning Research*, 18(153):1–43.
- Bergstra, J., Bardenet, R., Bengio, Y., and Kégl, B. (2011). Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems*, volume 24.
- Bergstra, J. and Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305.
- Beyer, K., Goldstein, J., Ramakrishnan, R., and Shaft, U. (1999). When is “nearest neighbor” meaningful? In *International Conference on Database Theory (ICDT)*, pages 217–235.
- Bishop, C. M. and Bishop, H. (2024). *Deep Learning: Foundations and Concepts*. Springer.
- Bjorkegren, D. and Grissen, D. (2020). Behavior revealed in mobile phone usage prediction credit repayment. In *The World Bank Economic Review*, volume 34.
- Box, G. E. P. and Cox, D. R. (1964). An analysis of transformations. *Journal of the Royal Statistical Society: Series B*, 26(2):211–243.

- Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24(2):123–140.
- Breiman, L. (2001a). Random forests. *Machine Learning*, 45(1):5–32.
- Breiman, L. (2001b). Statistical modeling: The two cultures. *Statistical Science*, 16(3):199–231.
- Breiman, L., Friedman, J. H., Olshen, R. A., and Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth, Belmont, CA.
- Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3.
- Campello, R. J. G. B., Moulavi, D., and Sander, J. (2013). Density-based clustering based on hierarchical density estimates. In *Advances in Knowledge Discovery and Data Mining (PAKDD)*, pages 160–172.
- Chawla, N. V., Bowyer, K. W., Hall, L., and Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357.
- Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proc. 22nd ACM SIGKDD*, pages 785–794.
- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314.
- Davis, J. and Goadrich, M. (2006). The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 233–240.
- Dempster, A. P., Laird, N. M., and Rubin, D. B. (1977). Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society, Series B*, 39(1):1–38.
- Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7):1895–1923.
- Elkan, C. (2001). The foundations of cost-sensitive learning. In *Proceedings of the 17th International Joint Conference Artificial Intelligence (IJCAI)*, pages 973–978.
- Ester, M., Kriegel, H.-P., Sander, J., and Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 226–231.

- Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874.
- Freund, Y. and Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139.
- Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232.
- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*. O'Reilly Media, 3rd edition.
- Glorot, X. and Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 249–256.
- Grinsztajn, L., Oyallon, E., and Varoquaux, G. (2022a). Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems*, volume 35.
- Grinsztajn, L., Oyallon, E., and Varoquaux, G. (2022b). Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems 35 (NeurIPS), Datasets and Benchmarks Track*.
- Guo, C. and Berkhahn, F. (2016). Entity embeddings of categorical variables. *arXiv preprint arXiv:1604.06737*.
- Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. (2017). On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1321–1330.
- Halko, N., Martinsson, P.-G., and Tropp, J. A. (2011). Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288.
- He, K., Zhang, X., Ren, S., and Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034.
- Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2):251–257.
- Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6):417–441.
- Huyen, C. (2022). *Designing Machine Learning Systems*. O'Reilly Media.

- Ioffe, S. and Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456.
- Kaufman, S., Rosset, S., Perlich, C., and Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM TKDD*, 6(4).
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30.
- Keskar, N. S., Mudigere, D., Nocedal, J., Smelyanskiy, M., and Tang, P. T. P. (2017). On large-batch training for deep learning: Generalization gap and sharp minima. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*.
- Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33.
- Li, L., Jamieson, K., DeSalvo, G., Rostamizadeh, A., and Talwalkar, A. (2017). Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52.
- Lin, T.-Y., Goyal, P., Girshick, R., He, K., and Dollár, P. (2017). Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988.
- Liu, F. T., Ting, K. M., and Zhou, Z.-H. (2008). Isolation forest. In *Eighth IEEE International Conference on Data Mining*, pages 413–422.
- Lloyd, S. P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137.
- Loshchilov, I. and Hutter, F. (2019). Decoupled weight decay regularization. In *Proceedings of the 7th International Conference on Learning Representations (ICLR)*.
- Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30.

- McInnes, L., Healy, J., and Melville, J. (2018). UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*.
- Melis, G., Dyer, C., and Blunsom, P. (2018). On the state of the art of evaluation in neural language models. In *International Conference on Learning Representations*.
- Micci-Barreca, D. (2001). A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems. *ACM SIGKDD Explorations Newsletter*, 3(1):27–32.
- Niculescu-Mizil, A. and Caruana, R. (2005). Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, pages 625–632.
- Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11):559–572.
- Platt, J. C. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, pages 61–74. MIT Press.
- Prince, S. J. D. (2024). *Understanding Deep Learning*. MIT Press.
- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., and Gulin, A. (2018). CatBoost: unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, volume 31.
- Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408.
- Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65.
- Rubin, D. B. (1976). Inference and missing data. *Biometrika*, 63(3):581–592.
- Schölkopf, B., Smola, A., and Müller, K.-R. (1998). Nonlinear component analysis as a kernel eigenvalue problem. *Neural Computation*, 10(5):1299–1319.
- Schwarz, G. (1978). Estimating the dimension of a model. *The Annals of Statistics*, 6(2):461–464.
- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28*.

- Siddiqi, N. (2017). *Intelligent Credit Scoring*. Wiley, 2nd edition.
- Snoek, J., Larochelle, H., and Adams, R. P. (2012). Practical Bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems*, volume 25.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958.
- Sundararajan, M., Taly, A., and Yan, Q. (2017). Axiomatic attribution for deep networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 3319–3328.
- van der Maaten, L. and Hinton, G. (2008). Visualizing data using *t*-SNE. *Journal of Machine Learning Research*, 9:2579–2605.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.
- Wattenberg, M., Viégas, F., and Johnson, I. (2016). How to use *t*-SNE effectively. *Distill*.
- Wolpert, D. H. (1992). Stacked generalization. *Neural Networks*, 5(2):241–259.
- Yeo, I.-K. and Johnson, R. A. (2000). A new family of power transformations to improve normality or symmetry. *Biometrika*, 87(4):954–959.